C言語プログラミング教材

基礎からC23までを段階的に学ぶ

C言語学習教材プロジェクト

目次

本文 **4**

第1章:C言語入門 5

第2章:基本構文 7

第3章:データ型と変数 11

第4章:演算子 16

第5章:条件分岐 20

第6章:繰り返し処理 24

第7章:配列 29

第8章:ポインター 34

第9章:配列とポインター 40

第10章:文字列 47

第11章:関数 55

第12章:ビット操作 73

第13章:構造体 79

第14章:関数ポインター 88

第15章:分割コンパイルと発展技術 97

第16章:C23の機能 111

補章1:C言語規格の歴史 115

補章2:開発環境 118

演習問題 **122**

第1章:C言語入門の演習問題 123

第2章:基本構文の演習問題 126

第3章:データ型と変数の演習問題 133

第4章:演算子の演習問題 137

第5章:条件分岐の演習問題 142

第6章:繰り返し処理の演習問題 149
第7章:配列の演習問題 155
第8章:ポインターの演習問題 162
第9章:配列とポインターの演習問題 167
第10章:文字列の演習問題 172
第11章:関数の演習問題 178
第12章:ビット操作の演習問題 188
第13章:構造体の演習問題 194
第14章:関数ポインターの演習問題 201
第15章:分割コンパイルと発展技術の演習問題 208
第16章:C23の機能の演習問題 215
補章2:開発環境の演習問題 222

解答例 **224**

第1章:C言語入門の解答例 225
第2章:基本構文の解答例 227
第3章:データ型と変数の解答例 229
第4章:演算子の解答例 233
第5章:条件分岐の解答例 236
第6章:繰り返し処理の解答例 239
第7章:配列の解答例 242
第8章:ポインターの解答例 245
第10章:文字列の解答例 249
第11章:関数の解答例 253
第12章:ビット操作の解答例 256
第13章:構造体の解答例 259
第14章:関数ポインターの解答例 262
第15章:分割コンパイルと発展技術の解答例 265
第16章:C23の機能の解答例 268

本文

第1章:C言語入門

学習目標

この章では、Cプログラムを作成し、コンパイルして実行するまでの流れを学びます。

- C言語が使われる分野を説明できる
- コンパイラを準備し、動作を確認できる
- `Hello, World!`を表示するプログラムを作成できる
- コンパイルと実行を区別できる

C言語が使われる分野

C言語は、オペレーティングシステム、組込み機器、処理系、性能が求められるライブラリなどで使われています。メモリ上のデータやハードウェアに近い処理を明示的に記述できる一方、配列の境界やメモリの寿命はプログラマが管理します。

C言語を学ぶと、プログラムがコンピューター上で動く仕組みを理解しやすくなります。具体的には、変数がメモリに格納される仕組みや、ソースコードが実行可能ファイルへ変換される過程を学べます。さらに、処理系に依存する動作と、規格で保証される動作を区別できるようになります。

開発環境

Cプログラムの作成には、テキストエディタとCコンパイラが必要です。コンパイラを導入したら、ターミナルまたはコマンドプロンプトで次のコマンドを実行してください。

```
gcc --version
```

バージョン情報が表示されれば、`gcc`コマンドを実行できます。環境によっては、コンパイラのコマンド名が`clang`です。

Windows

MinGW-w64など、GCCを含む開発環境を利用できます。インストール方法と`PATH`の設定は配布元によって異なるため、利用する配布パッケージの手順に従ってください。

macOS

Command Line Toolsを利用できます。

```
xcode-select --install
```

UbuntuとDebian

`build-essential`パッケージに、GCCや`make`などの基本ツールが含まれます。

```
sudo apt update
sudo apt install build-essential
```

最初のプログラム

`hello.c`を作成し、次のコードを保存します。

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

各行には次の役割があります。

- `#include <stdio.h>`は、`printf`関数の宣言を取り込みます。
- `int main(void)`は、引数を受け取らずに`int`を返す`main`関数を定義します。
- `printf`は、標準出力へ文字列を書き出します。
- `return 0;`は、ホスト環境へ正常終了を通知します。

コンパイルと実行

コンパイルでは、Cのソースファイルから実行可能ファイルを作ります。警告を有効にすると、誤りの兆候を早い段階で確認できます。

```
gcc -std=c90 -Wall -Wextra -pedantic hello.c -o hello
```

コンパイルに成功したら、生成されたプログラムを実行します。

```
./hello
```

Windowsのコマンドプロンプトでは、実行ファイル名を指定します。

```
hello.exe
```

実行結果は次のとおりです。

```
Hello, World!
```

文字列を入力するプログラム

空白を含まない名前を読み取り、入力内容を表示する例です。`scanf`のフィールド幅を配列の要素数より1小さく指定し、終端文字`'\0'`の領域を確保します。

```
#include <stdio.h>

int main(void)
{
    char name[50];

    printf("Name: ");
    if (scanf("%49s", name) != 1) {
        return 1;
    }

    printf("Hello, %s!\n", name);
    return 0;
}
```

`%s`だけを指定すると、入力が配列に収まるかを`scanf`が判断できません。また、この例は空白で入力を区切るため、空白を含む名前全体は読み取れません。行全体を扱う方法は、文字列の章で説明します。

付属コードのビルド

このディレクトリのMakefileを使うと、付属コードをまとめてビルドできます。

```
make all
make run-hello_world
make clean
```

実装例は、次のファイルにあります。

- hello_world.c
- greeting.c

演習問題

演習問題では、出力内容の変更、複数行の表示、簡単な入力処理を練習します。

第2章：基本構文

対応するC規格

- **主要対象**：C90
- **学習内容**：プログラムの構造、main関数、printf関数、コメント、エスケープシーケンス

学習目標

この章では、短いCプログラムを構成する要素と、標準出力の基本を学びます。

- #include、関数定義、文の役割を説明できる
- 標準規格に沿ったmain関数を定義できる
- printfの書式指定と引数の型を対応させられる
- C90で使用できるコメントを記述できる

プログラムの構造

次のプログラムは、標準出力へ1行を書き出して終了します。

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

#include <stdio.h>は、標準入出力ライブラリが提供するprintf関数の宣言を取り込みます。関数を正しい宣言なしで呼び出してはいけません。

mainは、ホスト環境で実行されるCプログラムの開始点です。関数本体は{と}で囲み、処理を構成する文の末尾には、必要に応じてセミコロンを置きます。

main関数

コマンドライン引数を使わない場合は、次の形式を使用します。

```
int main(void)
{
    return 0;
}
```

コマンドライン引数を受け取る場合は、次の形式を使用できます。

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Program: %s\n", argv[0]);

    if (argc > 1) {
        printf("First argument: %s\n", argv[1]);
    }

    return 0;
}
```

argcは引数の個数で、0以上です。argcが0より大きい場合、argv[0]はプログラム名を表す文字列を指します。実行環境からプログラム名を取得できない場合、argv[0][0]はヌル文字です。

mainから0を返すと、正常終了を表します。異常終了を表す移植可能な値には、<stdlib.h>で定義されるEXIT_FAILUREを使用できます。C99以降では、mainの末尾まで到達した場合も0を返したものとして扱われますが、

教材では終了状態を明示します。

標準Cのホスト環境では、void main()は規定された形式ではありません。戻り値の型にはintを使用してください。

printf関数

printfは、書式文字列に従って標準出力へデータを書き出します。書式指定子と対応する引数の型が一致しない場合、動作は未定義です。

指定子	対応する引数	用途
%d	int	符号付き10進整数
%u	unsigned int	符号なし10進整数
%c	intへ整数拡張された文字	1文字
%s	char *	ヌル終端文字列
%f	double	固定小数点表記
%p	void *	ポインタ値

可変個引数関数では、floatはdoubleへ、charとshortは通常intへ既定の実引数拡張を受けます。このため、printfで浮動小数点数を表示するときの%fはdoubleを受け取ります。

```
#include <stdio.h>

int main(void)
{
    int age = 20;
    double height = 170.5;

    printf("Age: %d\n", age);
    printf("Height: %.1f cm\n", height);
    return 0;
}
```

printfは、書き出した文字数を返します。出力エラーが発生した場合は負の値を返すため、出力の成否が必要なプログラムでは戻り値を検査します。

書式指定の誤り

書式指定子の個数と引数の個数を一致させます。

```
printf("%d + %d = %d\n", 10, 20, 30);
```

次の呼び出しは、%dが要求するintに対してdoubleを渡しているため、未定義動作です。

```
double value = 3.14;
printf("%d\n", value); /* Wrong format: undefined behavior. */
```

正しくは%fを使用します。

```
printf("%f\n", value);
```

エスケープシーケンス

文字列リテラルの中では、バックスラッシュで始まるエスケープシーケンスを使って制御文字や引用符を表します。

シーケンス	意味
\n	改行
\t	水平タブ
\"	ダブルクォート
\\	バックスラッシュ

シーケンス	意味
\a	警告音
\b	バックスペース
\r	復帰

端末が警告音、バックスペース、復帰をどのように表示するかは、実行環境に依存します。たとえば、\bが直前の文字を画面から必ず消すとは限りません。

```
#include <stdio.h>

int main(void)
{
    printf("First line\nSecond line\n");
    printf("Name\tScore\n");
    printf("Quote: \"C\"\n");
    printf("Path: C:\\Program Files\\App\n");
    return 0;
}
```

コメント

C90では、/*から*/までがコメントです。コメントは入れ子にできません。

```
/* This is a comment. */

/*
 * A comment can span
 * multiple lines.
 */
```

//で始まる行コメントは、C99で標準化されました。C90としてコンパイルするコードでは使用しません。

コンパイル

警告と規格適合性の診断を有効にしてコンパイルします。

```
gcc -std=c90 -Wall -Wextra -pedantic examples/hello_world.c -o hello_world
./hello_world
```

Makefileを使う場合は、次のコマンドを実行します。

```
make all
make run-all
make clean
```

実装例は、次のファイルにあります。

- hello_world.c
- formatted_output.c
- escape_sequences.c

C90とC99の違い

C90では、宣言を各複合文の文より前に置き、コメントには/* ... */を使います。C99では、文の後やfor文の初期化部でも変数を宣言でき、//コメントも使用できます。

演習問題

演習問題では、標準出力、書式指定、エスケープシーケンスを練習します。

参考資料

- 第1章：はじめてのC言語

- データ型と変数

第3章：データ型と変数

対応するC規格

- **主要対象**：C90
- **学習内容**：基本型、宣言、初期化、型変換、スコープ、記憶域期間、const、sizeof

学習目標

この章では、値を表現する型と、変数を利用できる範囲・期間を学びます。

- 基本型の性質と処理系依存の範囲を区別できる
- 変数を宣言し、使用前に初期化できる
- 暗黙の型変換による値の変化を説明できる
- スコープ、リンケージ、記憶域期間を区別できる
- sizeofの結果を正しい型で扱える

オブジェクトと型

Cの**オブジェクト**は、値を格納する記憶領域です。変数の宣言では、オブジェクトの型と識別子を指定します。

```
int count;
double average;
char grade;
```

型は、値の表現、演算の規則、必要な記憶領域を決めます。同じビット列でも、型が異なれば解釈や演算結果が異なる場合があります。

整数型

標準Cの基本整数型には、char、short、int、longと、それぞれの符号付き・符号なしの型があります。ただし、charがunsigned charと同じ範囲かsigned charと同じ範囲かは処理系定義です。

型の幅は固定されていません。標準が保証するのは、各型が少なくとも必要な範囲を表現でき、次の大小関係を満たすことです。

```
sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long)
```

sizeof(char)は常に1ですが、Cにおける1バイトが必ず8ビットとは限りません。1バイトのビット数は、<limits.h>のCHAR_BITで確認できます。

各処理系における範囲は、<limits.h>のINT_MIN、INT_MAX、UINT_MAXなどで確認します。特定のビット幅が必要な場合は、C99以降の<stdint.h>が提供するint32_tなどを検討します。これらの正確な幅の型は、処理系が対応する表現を持つ場合にだけ定義されます。

整数型の選択

通常の整数計算にはintを使います。ビット集合や剰余算術には、符号なし整数型が適します。ただし、「負の値を使わない」という理由だけでunsignedを選ぶべきではありません。符号付き整数との比較で、意図しない型変換が起こることがあるためです。

文字データの格納にはcharを使います。生のバイト列や0からUCHAR_MAXまでの値を扱う場合は、unsigned charを使います。

浮動小数点型

標準Cの浮動小数点型は、float、double、long doubleの3種類です。この順に精度と表現範囲が狭くならないことは保証されます。ただし、各型のバイト数やIEEE 754への準拠は、標準Cでは規定されません。

実際の精度と範囲は、<float.h>が定義するFLT_DIG、DBL_DIG、FLT_MAX、DBL_MAXなどで確認できます。多くの10進小数は二進の浮動小数点数で正確に表せないため、計算では丸め誤差を考慮します。金額など10進単位を厳密に扱う値は、最小通貨単位に換算して整数で保持する方法が適しています。

宣言と初期化

変数は、最初に読み取る前に値を設定します。

```
int count = 0;
double ratio = 0.5;
char grade = 'A';
```

C90では、各複合文の宣言を、その複合文に含まれる文より前へ置きます。「関数の先頭だけ」でなく、内側のブロックの先頭にも宣言できます。

```
int main(void)
{
    int value = 10;

    if (value > 0) {
        int doubled = value * 2;
        value = doubled;
    }

    return 0;
}
```

初期化していない自動記憶域期間のオブジェクトは、不定値を持ちます。不定値の読み取りは、型や状況によって未定義動作になり得るため、表示して「どの値になるか」を観察する教材にはしません。

代入と算術

代入演算子は、右辺を左辺の型へ変換して格納します。

```
int number = 10;

number += 5;
number -= 3;
number *= 2;
number /= 4;
```

C99以降の整数除算では、小数部を0方向へ切り捨てます。C90では、負のオペランドを含む除算と剰余の丸め方向が処理系定義です。

```
int a = 5;
int b = 2;
int quotient = a / b;              /* 2 */
double average = (double)a / b;    /* 2.5 */
```

0による整数除算と剰余は未定義動作です。除数が入力値で決まる場合は、演算前に0でないことを確認します。

型変換

代入、関数呼び出し、算術式などでは、必要に応じて値が別の型へ変換されます。変換を明示する構文がキャストです。

```
int total = 100;
int count = 3;
double average = (double)total / count;
```

整数型と浮動小数点型の変換規則は、単純な「小さい型から大きい型への順位」では説明できません。整数拡張、整数変換の順位、符号の有無、各型が表現できる範囲に基づいて決まります。

値が範囲外になる変換

符号なし整数型への変換結果は、その型が表現できる値の個数を法として繰り返し加減した値です。

```
unsigned int wrapped = (unsigned int)-1;
```

符号付き整数型への変換で値を表現できない場合、結果は処理系定義になるか、処理系定義のシグナルが発生します。「常に-1になる」など、特定の結果を前提にしてはいけません。

浮動小数点型から整数型への変換では、小数部が0方向へ切り捨てられます。切り捨て後の値を変換先の整数型で表現できない場合、動作は未定義です。

符号付き整数演算で結果を型の範囲内に表現できない場合も、動作は未定義です。一方、符号なし整数演算は、その型の最大値に1を加えた数を法とする算術です。

スコープ、リンケージ、記憶域期間

スコープは、識別子を名前で参照できる範囲です。**リンケージ**は、別の宣言が同じ実体を指すかどうかを決めます。**記憶域期間**は、オブジェクトの生存期間です。

ファイルスコープとブロックスコープ

関数の外側で宣言した識別子は、宣言位置から翻訳単位の末尾までファイルスコープを持ちます。関数本体やif文のブロック内で宣言した変数は、ブロックスコープを持ちます。

```
static int file_private = 100;

int get_value(void)
{
    int local = file_private;
    return local;
}
```

Cで関数スコープを持つのはラベルです。関数内で宣言した通常の変数を「関数スコープ」と呼ぶのは正確ではありません。

ファイルスコープのオブジェクトへstaticを指定すると、内部リンケージを持ち、同じ翻訳単位の外部から同じ識別子で参照できなくなります。

自動記憶域期間

通常のブロック内変数は、自動記憶域期間を持ちます。宣言を含むブロックへ入るたびに生存期間が始まり、ブロックから出ると終了します。標準Cは、このオブジェクトが「スタック」に置かれるとは規定していません。

```
void count_once(void)
{
    int count = 0;
    count++;
}
```

静的記憶域期間

ファイルスコープのオブジェクトと、staticを指定したブロック内オブジェクトは、静的記憶域期間を持ちます。プログラムの開始前に初期化され、実行中は値を保持します。明示的な初期化子がない場合は、型に応じた0またはヌルポインタへ初期化されます。

```
int next_id(void)
{
    static int id;
    return ++id;
}
```

register指定子

registerを指定したオブジェクトには、単項&演算子を適用できません。処理系がCPUレジスタへ格納する保証はなく、格納場所は実装が決めます。

const修飾子

constで修飾した型の左辺値を通じて、オブジェクトを変更することはできません。

```
const int max_count = 100;
const char *text = "Hello";
```

C90では、ブロックスコープのconst intは整数定数式ではないため、固定長配列の要素数には使えません。固定長配列の上限には、列挙定数や整数定数式になるマクロを使用できます。

```
enum { BUFFER_SIZE = 64 };
char buffer[BUFFER_SIZE];
```

マクロはプリプロセス時のトークン置換であり、型を持ちません。式形式のマクロでは、引数と展開全体を括弧で囲み、副作用を持つ引数が複数回評価される点に注意します。

```
#define SQUARE(x) ((x) * (x))
```

SQUARE(i++)のような呼び出しは、同じオブジェクトを複数回更新するため使用しません。

sizeof演算子

sizeofは、型または式の型が占めるバイト数をsize_t型で返します。型名には括弧が必要ですが、式の括弧は任意です。

```
sizeof(int)
sizeof value
sizeof(value)
```

size_tの値をprintfで表示する%zuはC99で追加されました。C90では、表示可能な範囲であることを確認したうえでunsigned longへ変換し、%luを使用します。

```
printf("int: %lu bytes\n", (unsigned long)sizeof(int));
```

通常、sizeofのオペランドの式は評価されません。ただし、C99以降の可変長配列型の大きさを決める必要がある場合は、実行時に評価されます。したがって、「sizeofのオペランドには常に副作用がない」と一般化してはいけません。

配列の要素数は、配列そのものが見える場所で次のように求められます。

```
int values[10];
size_t count = sizeof values / sizeof values[0];
```

この式を関数仮引数に対して使うと、仮引数の配列宣言がポインタ型へ調整されるため、要素数は得られません。

コンパイル

```
gcc -std=c90 -Wall -Wextra -pedantic examples/basic_types.c -o basic_types
./basic_types
```

Makefileを使う場合は、次のコマンドを実行します。

```
make all
make run-all
make clean
```

実装例は、次のファイルにあります。

- basic_types.c
- arithmetic_operations.c

- sizeof_demo.c
- data_types_demo.c

C99で追加された主な機能

C99では、`long long`、`_Bool`、複素数型、文の後での宣言、可変長配列などが追加されました。C11とC17では可変長配列への対応が処理系の任意機能であるため、対象とする規格と処理系で利用可能かを確認します。

演習問題

演習問題では、型の選択、変換、スコープ、`sizeof`を練習します。

第4章：演算子

対応するC規格

- **主要対象**：C90
- **学習内容**：算術、代入、比較、論理、ビット、条件、sizeof、優先順位

学習目標

この章では、値を計算し、比較し、組み合わせるための演算子を学びます。

- 整数演算と浮動小数点演算を区別できる
- 前置・後置の++と--を使い分けられる
- 論理演算子の短絡評価を説明できる
- ビットシフトの未定義動作と処理系定義の動作を避けられる
- 優先順位と評価順序を区別できる

算術演算子

演算子	意味	例
+	加算	a + b
-	減算	a - b
*	乗算	a * b
/	除算	a / b
%	剰余	a % b

整数同士の演算結果は、原則として整数型です。

```
int total = 7;
int count = 2;
int quotient = total / count;           /* 3 */
double average = (double)total / count; /* 3.5 */
```

0による整数除算と剰余は未定義動作です。符号付き整数の加算、減算、乗算で結果を型の範囲内に表現できない場合も未定義動作です。

C99以降の整数除算は、小数部を0方向へ切り捨てます。C90では、負のオペランドを含む除算と剰余の丸め方向が処理系定義だったため、移植性が必要なC90コードでは負数を含む結果に依存しません。

剰余は、周期的な処理や倍数の判定に使えます。

```
if (number % 2 == 0) {
    printf("even\n");
}
```

代入演算子

単純代入=は、右辺の値を左辺の型へ変換して格納します。複合代入は、演算と代入をまとめた構文です。

```
int score = 100;

score += 20;
score -= 5;
score *= 2;
score /= 5;
score %= 7;
```

E1 op= E2は、おおむねE1 = E1 op E2に相当しますが、E1は1回だけ評価される点が異なります。左辺が配列要素や間接参照を含む場合、この違いが重要です。

インクリメントとデクリメント

前置形式は更新後の値を、後置形式は更新前の値を式の値とします。

```
int a = 5;
int b = 5;
int x = ++a; /* a == 6, x == 6 */
int y = b++; /* b == 6, y == 5 */
```

更新後の値をその式で使わない場合、単独の文としてi++または++iを使えます。読みやすさのため、1つの式の中で同じオブジェクトを複数回更新しません。

```
int n = 0;
int result = n++ + n++; /* Undefined behavior. */
```

C90とC99の用語では、次のシーケンスポイントまでに同じオブジェクトを複数回更新するため未定義動作です。

関係演算子と等価演算子

演算子	意味
<	より小さい
<=	以下
>	より大きい
>=	以上
==	等しい
!=	等しくない

比較結果の型はintで、真なら1、偽なら0です。C90では専用のブール型を使わず、条件式では0を偽、0以外を真として扱います。

```
if (count == 0) {
    printf("empty\n");
}
```

=は代入、==は比較です。条件式の中で代入が必要な場合は、意図を明確にするため括弧で囲みます。

```
if ((ch = getchar()) != EOF) {
    putchar(ch);
}
```

浮動小数点数は丸め誤差を含むことがあるため、計算結果を==で比較できるかは計算内容によって判断します。許容誤差を使う場合は、固定値を無条件に選ばず、値の尺度と要求精度を考慮します。

論理演算子

演算子	意味
&&	論理積
||	論理和
!	論理否定

論理演算子の結果はintの0または1です。!valueは、valueが0なら1、0以外なら0になります。

短絡評価

`left && right`では、`left`が0なら`right`を評価しません。`left || right`では、`left`が0以外なら`right`を評価しません。

この規則により、危険な演算の前提条件を先に検査できます。

```
if (divisor != 0 && value / divisor > 10) {
    printf("large quotient\n");
}
```

```
if (ptr != NULL && *ptr == 'A') {
    printf("match\n");
}
```

一方、右オペランドの副作用に依存すると、左オペランドの値によって更新が実行されない場合があります。

```
int x = 0;
int y = 0;
int result = x && ++y; /* y remains 0. */
```

副作用が処理の目的なら、独立した文として記述する方が意図を確認しやすくなります。

ビット演算子

演算子	意味
&	ビット単位のAND
|	ビット単位のOR
^	ビット単位のXOR
~	ビット反転
<<	左シフト
>>	右シフト

ビット集合には、符号なし整数型を使うと規則を説明しやすくなります。

```
unsigned int read_flag = 1u << 0;
unsigned int write_flag = 1u << 1;
unsigned int flags = read_flag | write_flag;

if ((flags & read_flag) != 0u) {
    printf("read enabled\n");
}
```

`char`や`short`のオペランドには整数拡張が適用されます。たとえば`~a`の計算結果は、`a`が`unsigned char`でも通常`int`です。必要なら、目的の型へ明示的に変換します。

```
unsigned char a = 5u;
unsigned char inverted = (unsigned char)~a;
```

シフトの条件

シフト回数が負の場合、または整数拡張後の左オペランドの幅以上の場合、動作は未定義です。符号付き負数の左シフトも未定義動作です。符号付き非負数の左シフトは、結果をその型で表現できる場合にだけ定義されます。

符号なし整数の右シフトでは、空いた上位ビットに0が入ります。負の符号付き整数の右シフト結果は処理系定義です。

条件演算子

条件演算子`?:`は、条件に応じて2つの式の一方だけを評価します。

```
int max = a > b ? a : b;
```

短い値の選択には適しますが、副作用を伴う長い処理はif文で記述します。

sizeof演算子

sizeofは、型またはオブジェクトが占めるバイト数をsize_t型で返します。C90で表示する場合は、値がunsigned longに収まることを前提に変換します。

```
int values[10];

printf("int: %lu bytes\n", (unsigned long)sizeof(int));
printf("array: %lu bytes\n", (unsigned long)sizeof values);
```

優先順位と評価順序

優先順位は、演算子がどのオペランドと結び付くかを決めます。オペランドを実際に評価する順序を決めるものではありません。

```
int result = a + b * c; /* a + (b * c) */
```

関数引数など、多くの式では評価順序が規定されていません。副作用の順序が必要な処理は、複数の文に分けます。

```
int first = f();
int second = g();
int result = combine(first, second);
```

論理式では!、&&、||の順に強く結び付きます。優先順位を覚えていても、条件のまとまりが読み取りにくい場合は括弧を使います。

```
if (is_admin || (is_member && has_permission)) {
    printf("allowed\n");
}
```

コンパイル

```
gcc -std=c90 -Wall -Wextra -pedantic examples/operators_demo.c -o operators_demo
./operators_demo
```

Makefileを使う場合は、次のコマンドを実行します。

```
make all
make run-all
make clean
```

実装例は、次のファイルにあります。

- operators_demo.c
- comparison_operators.c
- logical_operators.c
- bitwise_demo.c
- precedence_demo.c

演習問題

演習問題では、算術式、比較、短絡評価、ビット演算、優先順位を練習します。

第5章：条件分岐

対応するC規格

- **主要対象**：C90
- **学習内容**：if、else、switch、条件演算子、条件式の注意点

学習目標

この章では、条件に応じて実行する処理を選ぶ方法を学びます。

- if文とif-else文を記述できる
- 複数の条件を検査する順序を設計できる
- switch文のcase、break、フォールスルーを説明できる
- 代入と比較、浮動小数点比較の誤りを避けられる

条件式

C90では、条件式の値が0なら偽、0以外なら真です。関係演算子と論理演算子の結果は、intの0または1になります。

```
if (count > 0) {
    printf("items available\n");
}
```

if文

if文は、条件が真の場合に文を実行します。

```
if (condition) {
    statement;
}
```

本体が1文だけでも波括弧を付けると、後から文を追加したときの誤りを防ぎやすくなります。

```
if (score >= 60) {
    printf("passed\n");
    passed_count++;
}
```

波括弧がない場合、ifが制御するのは直後の1文だけです。

```
if (score >= 60)
    printf("passed\n");
passed_count++; /* Always executed. */
```

if-else文

if-else文は、条件が真なら最初の文を、偽ならelse側の文を実行します。

```
if (age >= 18) {
    printf("adult\n");
} else {
    printf("minor\n");
}
```

入力値を前提とする処理では、正常系より先に不正値を処理すると、入れ子を浅くできます。

```
if (amount <= 0) {
    printf("invalid amount\n");
} else if (amount > balance) {
    printf("insufficient balance\n");
} else {
    balance -= amount;
```

```
    printf("withdrawal completed\n");
}
```

else ifによる複数分岐

条件は上から順に評価され、最初に真になった分岐だけを実行します。範囲が重なる場合は、より限定された条件または大きい境界から並べます。

```
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else if (score >= 70) {
    grade = 'C';
} else if (score >= 60) {
    grade = 'D';
} else {
    grade = 'F';
}
```

score >= 60を最初に置くと、90点もその分岐で処理されます。各条件がどの範囲を担当するかを確認してください。

入れ子のif文

条件の中で追加条件を検査する場合は、if文を入れ子にできます。

```
if (temperature >= 25) {
    if (is_raining) {
        printf("warm and rainy\n");
    } else {
        printf("warm and dry\n");
    }
}
```

波括弧を省略した入れ子では、elseは対応可能な最も近いifへ結び付きます。意図を明確にするため、入れ子では波括弧を省略しません。

条件を論理演算子でまとめた方が読みやすい場合もあります。

```
if (age >= 18 && has_license) {
    printf("eligible\n");
}
```

ただし、条件ごとに異なるエラーを示す必要がある場合は、分岐を分けます。

switch文

switch文は、整数型または列挙型の式の値に応じて処理を選びます。各caseには、変換後に重複しない整数定数式を指定します。

```
switch (choice) {
case 1:
    printf("option 1\n");
    break;
case 2:
    printf("option 2\n");
    break;
default:
    printf("unknown option\n");
    break;
}
```

breakを実行すると、最も内側のswitch文から抜けます。breakがなければ、次のcaseへ処理が続きます。

意図的なフォールスルー

複数の値を同じ処理へまとめる場合は、文を置かずにcaseを連続させます。

```
switch (letter) {
```

```
case 'a':
case 'e':
case 'i':
case 'o':
case 'u':
    printf("vowel\n");
    break;
default:
    printf("not a lowercase vowel\n");
    break;
}
```

処理を実行した後で次のcaseへ続ける場合は、その意図をコメントで示します。

```
switch (level) {
case 2:
    enable_advanced_mode();
    /* Fall through. */
case 1:
    enable_basic_mode();
    break;
default:
    break;
}
```

C90では宣言そのものは文ではないため、caseラベルの直後で変数を宣言したい場合はブロックを作ります。

```
switch (choice) {
case 1: {
    int value = read_value();
    use_value(value);
    break;
}
default:
    break;
}
```

条件演算子

条件演算子?:は、条件に応じて2つの式の一方を評価します。

```
int max = a > b ? a : b;
```

値を選ぶ短い式に適しています。複数の文や副作用を含む処理は、if-else文で記述します。

条件式の注意点

代入と比較

=は代入、==は比較です。

```
if (x = 10) {  /* Assigns 10, then tests it as true. */
    printf("executed\n");
}
```

比較する場合は==を使います。

```
if (x == 10) {
    printf("x is 10\n");
}
```

浮動小数点数の比較

0.1や0.2のような値は、一般的な2進浮動小数点形式で正確に表現できない場合があります。計算結果を比較するときは、要求精度と値の大きさに応じた方法を選びます。

```
#include <math.h>

if (fabs(a - b) <= tolerance) {
    printf("close\n");
}
```

固定の許容誤差だけでは、非常に大きい値や非常に小さい値に適さない場合があります。用途に応じて絶対誤差と

相対誤差を組み合わせます。

条件式の末尾に置くセミコロン

次のコードでは、ifが制御する文は空文です。後続のブロックは条件に関係なく実行されます。

```
if (condition); /* Empty statement. */
{
    printf("always executed\n");
}
```

コンパイル

```
gcc -std=c90 -Wall -Wextra -pedantic examples/basic_if.c -o basic_if
./basic_if
```

Makefileを使う場合は、次のコマンドを実行します。

```
make all
make run-all
make clean
```

実装例は、次のファイルにあります。

- basic_if.c
- if_else.c
- grade_system.c
- switch_basic.c
- conditional_operator.c

C99以降との違い

C99以降では、文の後やfor文の初期化部でも変数を宣言できます。条件分岐そのものの基本構文と、0を偽・0以外を真とする規則はC90と共通です。

演習問題

演習問題では、範囲判定、入力検査、switch文、条件演算子を練習します。

第6章：繰り返し処理

対応するC規格

- **主要対象**：C90
- **学習内容**：for、while、do-while、break、continue、入れ子のループ

学習目標

この章では、同じ処理を条件に応じて繰り返す方法を学びます。

- 反復回数と終了条件に応じてループを選べる
- for文の各式が評価される順序を説明できる
- breakとcontinueの制御先を説明できる
- 配列の境界を守るループを記述できる

for文

for文は、初期化、継続条件、反復式を1か所にまとめます。

```
for (initialization; condition; iteration) {
    statement;
}
```

実行順序は次のとおりです。

1. 初期化を1回だけ実行する
2. 継続条件を評価する
3. 条件が0ならループを終了する
4. 本体を実行する
5. 反復式を評価し、手順2へ戻る

C90では、ループ変数をfor文より前で宣言します。

```
#include <stdio.h>

int main(void)
{
    int i;

    for (i = 0; i < 5; i++) {
        printf("%d\n", i);
    }

    return 0;
}
```

配列を先頭から末尾まで処理するときは、添字の開始値と終了条件を配列の要素数に対応させます。

```
int values[5] = { 10, 20, 30, 40, 50 };
int i;

for (i = 0; i < 5; i++) {
    printf("%d\n", values[i]);
}
```

`i <= 5`とすると、最後の反復で`values[5]`へアクセスし、配列の範囲を超えます。

while文

while文は、本体を実行する前に継続条件を評価します。最初の評価が偽なら、本体は一度も実行されません。

```
while (condition) {
    statement;
}
```

入力の成否と終了条件を同時に検査する例です。

```
#include <stdio.h>

int main(void)
{
    int number;

    while (scanf("%d", &number) == 1 && number > 0) {
        printf("value: %d\n", number);
    }

    return 0;
}
```

scanfが1を返すのは、1個の変換に成功した場合です。数値へ変換できない入力では0を、入力の終端または読み取りエラーではEOFを返します。

do-while文

do-while文は、本体を実行した後で継続条件を評価します。文へ到達した場合、本体は少なくとも1回実行されます。

```
do {
    statement;
} while (condition);
```

while (condition)の末尾にはセミコロンが必要です。

```
#include <stdio.h>

int main(void)
{
    int choice;

    do {
        printf("1: continue, 0: exit\n");

        if (scanf("%d", &choice) != 1) {
            return 1;
        }
    } while (choice != 0);

    return 0;
}
```

break文

breakは、最も内側にあるループまたはswitch文を終了します。

```
int i;

for (i = 0; i < 100; i++) {
    if (i == 5) {
        break;
    }

    printf("%d\n", i);
}
```

入れ子のループでbreakを実行しても、終了するのは最も内側のループだけです。外側のループも終了する必要がある場合は、フラグを検査する、処理を関数へ分けてreturnする、または終了先を明示した構造にします。

```
int row;
int column;
int found = 0;
```

```
for (row = 0; row < 3 && !found; row++) {
    for (column = 0; column < 3; column++) {
        if (matrix[row][column] == target) {
            found = 1;
            break;
        }
    }
}
```

continue文

continueは、現在の反復の残りを飛ばして次の反復へ進みます。for文では反復式へ、while文とdo-while文では継続条件の評価へ移ります。

```
int i;

for (i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue;
    }

    printf("%d\n", i);
}
```

while文で更新処理より前にcontinueすると、更新が実行されず無限ループになることがあります。

```
int i = 0;

while (i < 10) {
    i++;

    if (i % 2 == 0) {
        continue;
    }

    printf("%d\n", i);
}
```

入れ子のループ

2次元の表や配列では、外側のループで行を、内側のループで列を処理します。

```
int row;
int column;

for (row = 1; row <= 3; row++) {
    for (column = 1; column <= 3; column++) {
        printf("%d ", row * column);
    }

    printf("\n");
}
```

外側がm回、内側が各回n回実行される場合、本体の実行回数はm * n回です。データ量が大きい場合は、この回数が処理時間へ与える影響を確認します。

無限ループ

継続条件を省略したfor文と、0以外の定数を条件にしたwhile文は、明示的に終了しない限り反復を続けます。

```
for (;;) {
    if (should_stop()) {
        break;
    }

    process_one_item();
}
```

```
while (1) {
    if (should_stop()) {
        break;
    }
```

```
    process_one_item();
}
```

イベントループや待受処理では無限ループが必要な場合があります。終了条件、エラー処理、外部から停止する方法を設計してください。

ループを安全に書く

境界を式で対応させる

配列の要素数を配列から求められる場所では、上限を重複して記述しない方法があります。

```
#include <stddef.h>

int values[10];
size_t i;
size_t count = sizeof values / sizeof values[0];

for (i = 0; i < count; i++) {
    values[i] = 0;
}
```

終了条件を更新する

ループ内で条件に関わる値が変化しない場合、ループが終了しない可能性があります。

```
int i = 0;

while (i < 10) {
    printf("%d\n", i);
    i++;
}
```

空文を確認する

次のfor文の本体は、末尾のセミコロンによる空文です。

```
for (i = 0; i < 10; i++); /* Empty loop body. */
{
    printf("executed once\n");
}
```

コンパイル

```
gcc -std=c90 -Wall -Wextra -pedantic examples/basic_for.c -o basic_for
./basic_for
```

Makefileを使う場合は、次のコマンドを実行します。

```
make all
make run-all
make clean
```

実装例は、次のファイルにあります。

- basic_for.c
- while_loop.c
- while_dowhile.c
- break_continue.c
- nested_loops.c

C99以降との違い

C99以降では、for文の初期化部で変数を宣言できます。

```
for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
}
```

このiのスコープはfor文を含む範囲に限定されます。

演習問題

演習問題では、集計、入力ループ、入れ子のループ、終了条件を練習します。

第7章：配列

対応するC規格

- **主要対象**：C90
- **学習内容**：1次元配列、多次元配列、初期化、走査、検索、集計

学習目標

この章では、同じ型のオブジェクトを連続して保持する配列を学びます。

- 配列を宣言し、初期化できる
- 有効な添字の範囲を説明できる
- ループで配列を検索・集計できる
- 2次元配列の行と列を正しく処理できる
- 配列とポインタを同一視せず、変換が起こる場面を説明できる

配列とは

配列は、同じ型の要素を連続して並べたオブジェクトです。次の宣言は、`int`型の要素を5個持つ配列`values`を定義します。

```
int values[5];
```

要素の添字は0から始まります。要素数が5なら、有効な添字は0から4までです。

```
values[0] = 10;
values[4] = 50;
```

`values[5]`へのアクセスは配列の範囲外であり、動作は未定義です。Cは実行時に添字を自動検査しません。

配列の宣言

C90の固定長配列では、要素数に0より大きい整数定数式を指定します。

```
#define SCORE_COUNT 5

int scores[SCORE_COUNT];
```

配列の要素型は完全オブジェクト型である必要があります。要素数を後から変更することはできません。

配列の初期化

すべての要素を列挙できます。

```
int values[5] = { 10, 20, 30, 40, 50 };
```

初期化子が要素数より少ない場合、残りの要素は0で初期化されます。

```
int values[5] = { 1, 2 }; /* { 1, 2, 0, 0, 0 } */
```

要素数を省略すると、初期化子から要素数が決まります。

```
int values[] = { 10, 20, 30 };
```

すべての要素を0で初期化する一般的な書き方です。

```
int values[100] = { 0 };
```

初期化後の配列へ別の配列を代入することはできません。

```
int a[3] = { 1, 2, 3 };
```

```
int b[3];

/* b = a; */ /* Constraint violation. */
```

要素をコピーするには、ループまたは適切なライブラリ関数を使います。

```
int i;

for (i = 0; i < 3; i++) {
    b[i] = a[i];
}
```

配列を走査する

配列全体を処理するループでは、添字を0から要素数未満まで進めます。

```
#include <stddef.h>
#include <stdio.h>

int main(void)
{
    int values[] = { 10, 20, 30, 40, 50 };
    size_t count = sizeof values / sizeof values[0];
    size_t i;

    for (i = 0; i < count; i++) {
        printf("%d\n", values[i]);
    }

    return 0;
}
```

`sizeof values / sizeof values[0]`は、valuesが配列として見えている場所で要素数を求めます。関数仮引数として宣言した配列には使えません。

線形探索

配列を先頭から調べ、目的の値が見つかった位置を返します。見つからない場合は、添字として使わない負の値を返します。

```
int find_value(const int values[], int count, int target)
{
    int i;

    for (i = 0; i < count; i++) {
        if (values[i] == target) {
            return i;
        }
    }

    return -1;
}
```

関数仮引数の`const int values[]`は、`const int *values`へ調整されます。配列全体の要素数は関数へ自動では渡らないため、countを別の引数として渡します。

最大値、最小値、平均

最初の要素を初期値に使う場合、要素数が1以上であることを先に確認します。

```
#include <stddef.h>

int max_value(const int values[], int count, int *result)
{
    int i;
    int max;

    if (values == NULL || result == NULL || count <= 0) {
        return 0;
    }

    max = values[0];
```

```
    for (i = 1; i < count; i++) {
        if (values[i] > max) {
            max = values[i];
        }
    }

    *result = max;
    return 1;
}
```

平均を求める場合は、0による除算を避けます。加算結果が型の範囲を超える可能性も考慮し、データの範囲に応じて集計用の型を選びます。

```
#include <stddef.h>

double average(const int values[], int count)
{
    long sum = 0;
    int i;

    if (values == NULL || count <= 0) {
        return 0.0;
    }

    for (i = 0; i < count; i++) {
        sum += values[i];
    }

    return (double)sum / count;
}
```

この例でも、要素数と値の範囲によってはlongの加算がオーバーフローします。実際の入力上限を定めてから型を選んでください。

要素を逆順に並べる

配列の両端から要素を交換します。

```
void reverse(int values[], int count)
{
    int left = 0;
    int right = count - 1;

    while (left < right) {
        int temporary = values[left];
        values[left] = values[right];
        values[right] = temporary;
        left++;
        right--;
    }
}
```

このコードはC90でも有効です。temporaryはwhile本体の複合文で、文より前に宣言されています。

2次元配列

2次元配列は、配列を要素に持つ配列です。次のmatrixは、3個の「intを4個持つ配列」を要素に持ちます。

```
int matrix[3][4];
```

matrix[row][column]で要素へアクセスします。有効な行添字は0から2、列添字は0から3です。

```
int matrix[2][3] = {
    { 1, 2, 3 },
    { 4, 5, 6 }
};
```

Cの多次元配列は行優先で連続配置されます。この例では、matrix[0][0]からmatrix[0][2]が並び、その後に2行目が続きます。

```
int row;
int column;
```

```
for (row = 0; row < 2; row++) {
    for (column = 0; column < 3; column++) {
        printf("%d ", matrix[row][column]);
    }

    printf("\n");
}
```

2次元配列を関数へ渡すときは、ポインタ演算に必要な列数を型に含めます。

```
void print_matrix(const int matrix[][3], int rows)
{
    int row;
    int column;

    for (row = 0; row < rows; row++) {
        for (column = 0; column < 3; column++) {
            printf("%d ", matrix[row][column]);
        }

        printf("\n");
    }
}
```

配列からポインタへの変換

配列式は、多くの式の中で先頭要素へのポインタに変換されます。ただし、sizeofのオペランドや単項&のオペランドなどでは、この変換は起こりません。

```
int values[5];
int *first = values;       /* &values[0] */
size_t bytes = sizeof values;
```

配列とポインタは別の型であり、同じオブジェクトではありません。この違いは、次章で詳しく扱います。

よくある誤り

範囲外アクセス

```
int values[5] = { 0 };
values[5] = 10; /* Undefined behavior. */
```

初期化前の読み取り

```
int values[5];
printf("%d\n", values[0]); /* Indeterminate value. */
```

バイト数と要素数の混同

```
int values[10];
size_t bytes = sizeof values;
size_t count = sizeof values / sizeof values[0];
```

bytesはバイト数、countは要素数です。

コンパイル

```
gcc -std=c90 -Wall -Wextra -pedantic examples/array_basics.c -o array_basics
./array_basics
```

Makefileを使う場合は、次のコマンドを実行します。

```
make all
make run-all
make clean
```

実装例は、次のファイルにあります。

- array_basics.c

- multidimensional_arrays.c

C99で追加された配列機能

C99では、指定初期化子、複合リテラル、可変長配列が追加されました。C11とC17では可変長配列への対応が処理系の任意機能です。

```
int values[10] = { [2] = 5, [7] = 9 };
```

演習問題

演習問題では、探索、集計、並べ替え、多次元配列を練習します。

第8章：ポインター

対応するC規格

- **主要対象**：C90
- **学習内容**：ポインタ、間接参照、配列との関係、関数引数、動的メモリ確保

学習目標

この章では、オブジェクトや関数を指すポインタを学びます。

- ポインタを宣言し、オブジェクトのアドレスを格納できる
- 間接参照が有効になる条件を説明できる
- 配列からポインタへの変換とポインタ演算の範囲を説明できる
- ポインタを関数へ渡して呼び出し元の値を変更できる
- 動的に確保したメモリの所有権と寿命を管理できる

ポインタとは

ポインタ型のオブジェクトは、別のオブジェクトまたは関数を指す値を格納します。指す対象の型は、宣言の*より左側に記述します。

```
int *int_pointer;
double *double_pointer;
```

未初期化の自動変数であるポインタは不定値を持ちます。有効なアドレスまたはヌルポインタで初期化してから使用します。

```
int *pointer = NULL;
```

ヌルポインタは、どのオブジェクトも指していないことを表します。ヌルポインタを間接参照すると、動作は未定義です。

アドレス演算子と間接演算子

単項&演算子は、オブジェクトまたは関数を指すポインタを作ります。単項*演算子は、有効なポインタが指す対象へアクセスします。

```
#include <stdio.h>

int main(void)
{
    int value = 42;
    int *pointer = &value;

    printf("value: %d\n", value);
    printf("through pointer: %d\n", *pointer);
    printf("address: %p\n", (void *)pointer);
    return 0;
}
```

printfの%pに対応する引数はvoid *です。オブジェクトへのポインタは、表示時にvoid *へ変換します。表示形式は処理系定義であり、常に16進表記になるとは規定されていません。

間接参照した式は、指しているオブジェクトを表す左辺値になります。

```
int value = 10;
int *pointer = &value;

*pointer = 20; /* value becomes 20. */
```

ポインタと配列

配列はポインタではありません。ただし、配列式は多くの式の中で、先頭要素を指すポインタへ変換されます。

```
int values[5] = { 10, 20, 30, 40, 50 };
int *pointer = values; /* Same value as &values[0]. */
```

values[i]は*(values + i)と同じ規則で定義されます。

```
int i;

for (i = 0; i < 5; i++) {
    printf("%d\n", *(pointer + i));
}
```

配列とポインタが異なる場面

sizeofのオペランドでは、配列からポインタへの変換は起こりません。

```
int values[5];
int *pointer = values;

sizeof values;  /* Size of the entire array. */
sizeof pointer; /* Size of the pointer object. */
```

単項&のオペランドでも変換は起こりません。&valuesの型は「5個のintの配列へのポインタ」であり、&values[0]の型であるint *とは異なります。

ポインタ演算

オブジェクトへのポインタには、整数を加減できます。演算結果は、指している配列要素を基準に要素単位で移動します。

```
int values[5];
int *begin = &values[0];
int *third = begin + 2;
```

ポインタ演算で定義される範囲は、同じ配列オブジェクトの要素と、その末尾の1つ後ろまでです。末尾の1つ後ろを指すポインタは比較やループ終了判定に使えますが、間接参照できません。

```
int *current;
int *end = values + 5;

for (current = values; current != end; current++) {
    printf("%d\n", *current);
}
```

別々の配列を指すポインタ同士の減算や、順序関係の比較には依存しません。同じ配列内の2つのポインタを減算した結果の型はptrdiff_tです。

ポインタを関数へ渡す

Cの関数引数は値渡しです。ポインタを値として渡すと、関数はそのポインタが指すオブジェクトを変更できます。

```
void swap(int *left, int *right)
{
    int temporary;

    if (left == NULL || right == NULL) {
        return;
    }

    temporary = *left;
    *left = *right;
    *right = temporary;
}
```

```
int a = 10;
int b = 20;

swap(&a, &b);
```

関数がヌルポインタを受け付けない設計なら、その事前条件を関数の契約として文書化します。ヌルポインタをエラーとして処理する設計なら、間接参照より前に検査します。

配列を関数へ渡す

関数仮引数の配列宣言は、対応するポインタ型へ調整されます。次の2つの宣言は、関数型として同じです。

```
void process(int values[], int count);
void process(int *values, int count);
```

要素数は自動では渡らないため、別の引数で渡します。

```
int sum_array(const int values[], int count)
{
    int sum = 0;
    int i;

    if (values == NULL || count < 0) {
        return 0;
    }

    for (i = 0; i < count; i++) {
        sum += values[i];
    }

    return sum;
}
```

この例では、加算結果が`int`の範囲を超えないことを呼び出し側の前提としています。エラーと正しい合計値0を区別する必要がある場合は、戻り値と出力引数を分けます。

関数からポインタを返す

関数から返すポインタは、関数終了後も生存しているオブジェクトを指す必要があります。自動記憶域期間のローカル変数へのポインタを返してはいけません。

```
int *invalid_pointer(void)
{
    int local = 10;
    return &local; /* Invalid after the function returns. */
}
```

静的記憶域期間のオブジェクトへのポインタは、関数終了後も有効です。ただし、呼び出しごとに同じオブジェクトを共有するため、再入可能性や並行実行に注意が必要です。

```
int *shared_value(void)
{
    static int value;
    return &value;
}
```

呼び出し側へ所有権を渡す必要がある場合は、動的メモリ確保を使う方法もあります。

文字列とポインタ

文字列リテラルから配列を初期化すると、変更可能な配列が作られます。

```
char text[] = "Hello";
text[0] = 'h';
```

文字列リテラルを指すポインタを通じて文字を変更すると、動作は未定義です。C90でも、変更しない意図を`const char *`で表します。

```
const char *text = "Hello";
```

文字列はヌル文字`'\0'`で終わります。文字列処理では、配列の容量と終端文字の領域を管理します。

2次元配列へのポインタ

2次元配列の各要素は行を表す配列です。

```
int matrix[3][4];
int (*row_pointer)[4] = matrix;
```

row_pointer + 1は、int1個ではなく、int4個からなる次の行へ進みます。

```
matrix[2][3] == *(*(matrix + 2) + 3)
```

関数へ渡す場合も、最初の次元以外の要素数を型に含めます。

```
void clear_matrix(int matrix[][4], int rows)
{
    int row;
    int column;

    for (row = 0; row < rows; row++) {
        for (column = 0; column < 4; column++) {
            matrix[row][column] = 0;
        }
    }
}
```

動的メモリ確保

malloc、calloc、realloc、freeは<stdlib.h>で宣言されています。Cでは、mallocが返すvoid *をオブジェクトへのポインタへ暗黙に変換できるため、キャストは不要です。

```
#include <stdlib.h>

int *allocate_values(size_t count)
{
    int *values;

    if (count == 0 || count > (size_t)-1 / sizeof *values) {
        return NULL;
    }

    values = malloc(count * sizeof *values);
    return values;
}
```

mallocは、要求した大きさの領域を確保できない場合にヌルポインタを返します。成功時に得られる領域の内容は不定です。

```
int *values = allocate_values(100);

if (values == NULL) {
    return 1;
}

/* Use values here. */
free(values);
values = NULL;
```

freeへ渡せるのは、動的メモリ確保関数が返したポインタ、またはヌルポインタです。同じ領域を2回解放したり、配列の途中を指すポインタを渡したりすると、動作は未定義です。

calloc

callocは、要素数と1要素の大きさを受け取り、確保した全ビットを0にします。これは整数型の0を作る用途には使えますが、すべての型についてヌルポインタや浮動小数点の0.0と同じ表現になることを標準Cは保証していません。

```
int *values = calloc(count, sizeof *values);
```

要素数と要素サイズの積の処理はcalloc側が行いますが、確保失敗を示すヌルポインタは検査します。

realloc

reallocが失敗した場合、元の領域は解放されず、そのまま有効です。戻り値を元のポインタへ直接代入すると、失敗時に元のアドレスを失います。

```
int *temporary;

temporary = realloc(values, new_count * sizeof *values);

if (temporary == NULL) {
    /* values still refers to the original allocation. */
} else {
    values = temporary;
}
```

この例では、new_count * sizeof *valuesがsize_tの範囲を超えないことを事前に確認する必要があります。大きさ0のreallocには規格版や処理系による差があるため、解放したい場合はfreeを明示的に使います。

二重ポインタ

二重ポインタは、ポインタオブジェクトを指します。関数内で呼び出し元のポインタ値そのものを変更するときに使えます。

```
int allocate_one(int **result)
{
    int *value;

    if (result == NULL) {
        return 0;
    }

    value = malloc(sizeof *value);

    if (value == NULL) {
        return 0;
    }

    *value = 0;
    *result = value;
    return 1;
}
```

呼び出し側は、成功時に受け取った領域をfreeします。

関数ポインタ

関数ポインタは、互換性のある関数型を持つ関数を指します。

```
int add(int left, int right)
{
    return left + right;
}

int main(void)
{
    int (*operation)(int, int) = add;
    int result = operation(2, 3);

    return result == 5 ? 0 : 1;
}
```

オブジェクトへのポインタと関数へのポインタは別の分類です。標準Cは、関数ポインタとvoid *を相互変換できるとは保証していません。

よくある誤り

未初期化ポインタ

```
int *pointer;
*pointer = 10; /* Undefined behavior. */
```

解放後の使用

```
int *pointer = malloc(sizeof *pointer);

if (pointer != NULL) {
    free(pointer);
    /* *pointer = 10; */ /* Undefined behavior. */
    pointer = NULL;
}
```

配列の範囲外への移動

ポインタを同じ配列の末尾よりさらに先へ進める演算自体が未定義です。

```
int values[5];
int *invalid = values + 6; /* Undefined behavior. */
```

メモリリーク

所有している動的領域を指す最後のポインタを失うと、その領域を解放できなくなります。どの関数が解放責任を持つかをインターフェースで決めます。

コンパイル

```
gcc -std=c90 -Wall -Wextra -pedantic examples/pointer_basic.c -o pointer_basic
./pointer_basic
```

Makefileを使う場合は、次のコマンドを実行します。

```
make all
make run-all
make clean
```

実装例は、次のファイルにあります。

- pointer_basic.c
- pointer_arrays.c
- pointer_functions.c

C99で追加された関連機能

C99では、`restrict`修飾子、可変長配列、複合リテラル、`<stdint.h>`などが追加されました。`restrict`は、ポインタが指すオブジェクトへのアクセス関係について、プログラマが処理系へ契約を示す機能です。契約に違反した場合は未定義動作になるため、単なる最適化の要望として追加してはいけません。

演習問題

演習問題では、間接参照、配列走査、関数引数、動的メモリ管理を練習します。

第9章：配列とポインター

対応C規格

- **主要対象：** C90
- **学習内容：** 配列とポインターの関係、ポインターを使った配列操作、関数への配列渡し

学習目標

この章を完了すると、以下のことができるようになります。

- 配列式が先頭要素へのポインターに変換される条件を理解する
- ポインター演算を使った配列操作ができる
- 配列を関数に渡す仕組みを理解する
- 配列とポインターの型やサイズの違いを説明できる

概要と詳細

前提知識

この章は、次の内容を理解していることを前提とします。

- 第7章：配列（基本編）
- 第8章：ポインター基礎

配列とポインターの関係

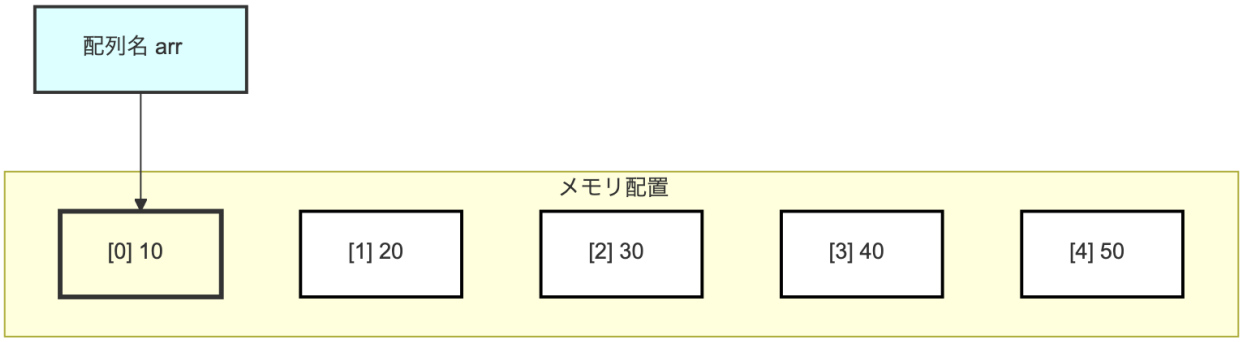
Cでは、配列式が多くの文脈で先頭要素へのポインターに変換されます。この変換によって同じ添字記法を使えますが、配列とポインターは異なる型のオブジェクトです。

配列式からポインターへの変換

配列を表す式は、`sizeof`のオペランドまたは単項`&`のオペランドになる場合を除き、通常は先頭要素へのポインターに変換されます。次の例では、`arr`と`&arr[0]`は同じアドレスを表します。一方、`&arr`の値として表現されるアドレスは同じでも、型は配列全体へのポインター`int (*)[5]`です。

```
int arr[5] = {10, 20, 30, 40, 50};

/* 3つの値は同じアドレスとして表示されるが、式の型は異なる */
printf("arr      = %p\n", (void *)arr);
printf("&arr[0]  = %p\n", (void *)&arr[0]);
printf("&arr     = %p\n", (void *)&arr);
```

視覚的な理解



配列int arr[5] = {10, 20, 30, 40, 50}の要素は、添字の順に連続して配置されます。次のアドレスは、sizeof(int) == 4で先頭アドレスを1000と仮定した例です。

- **メモリアドレス**:配列の要素は連続したメモリ領域に配置されます
  - arr[0]:アドレス1000、値10
  - arr[1]:アドレス1004、値20
  - arr[2]:アドレス1008、値30
  - arr[3]:アドレス1012、値40
  - arr[4]:アドレス1016、値50
- **配列式の変換後の値**:arrは、通常の式では最初の要素arr[0]へのポインターに変換されます
- **アドレスの増分**:この例ではsizeof(int) == 4なので、各要素のアドレスは4ずつ増加します

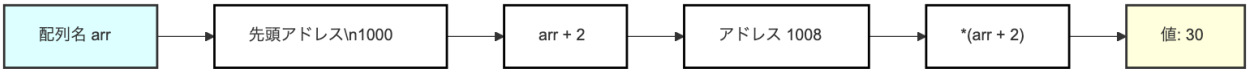
配列要素へのアクセス方法の等価性

2つの記法

C言語では、配列要素にアクセスする方法が2つあります。

```
int arr[5] = {10, 20, 30, 40, 50};
int i;

/* 添字演算とポインター演算は同じ要素を参照する */
for (i = 0; i < 5; i++) {
    printf("arr[%d] = %d\n", i, arr[i]);        /* 配列記法 */
    printf("*(arr+%d) = %d\n", i, *(arr+i));    /* ポインター記法 */
}

/* arr[i] は *((arr) + (i)) と定義される */
```

添字演算の仕組み



添字演算E1[E2]は*((E1) + (E2))と定義されています。arr[2]では、次の順序で参照先が決まります。

1. **配列名は先頭アドレス**
   - 式arrが、配列の最初の要素へのポインターに変換されます
2. **インデックスによるアドレス計算**
   - arr[2]にアクセスする場合、i = 2です
   - arr + iは「先頭からi要素分進んだアドレス」を計算します
3. **実際のアドレス計算**
   - sizeof(int) == 4と仮定すると、arr + 2が表すアドレスは1000 + (2 × 4) = 1008です
   - これは3番目の要素(インデックス2)のアドレスです
4. **値の取得**
   - *(arr + i) でそのアドレスに格納された値を取得します
   - この定義により、arr[2]と*(arr + 2)は同じ要素を表します

配列とポインターの違い

配列式はポインターに変換されることがありますが、配列そのものがポインターになるわけではありません。

配列には代入できない

```
int arr[5] = {1, 2, 3, 4, 5};
int *ptr = arr;  /* OK: 配列の先頭アドレスをポインターに代入 */

/* 配列は変更可能な左辺値ではないため、代入やインクリメントはできない */
arr = ptr;       /* 制約違反 */
arr++;           /* 制約違反 */

/* ポインターは変数（変更可能） */
ptr++;           /* OK: ポインターを次の要素に移動 */
ptr = &arr[3];   /* OK: ポインターに別のアドレスを代入 */
```

サイズの違い

```
int arr[5] = {1, 2, 3, 4, 5};
int *ptr = arr;

printf("sizeof(arr) = %lu\n", (unsigned long)sizeof(arr));
printf("sizeof(ptr) = %lu\n", (unsigned long)sizeof(ptr));
```

`sizeof(arr)`は配列全体のバイト数であり、`5 * sizeof(int)`です。`sizeof(ptr)`はポインター自体のバイト数なので、両者が同じ値になる保証はありません。

ポインター演算による配列操作

ポインターの移動

```
#include <stdio.h>

int main(void) {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;

    printf("Value: %d\n", *ptr);      /* 10 */

    ptr++;  /* 次の要素へ移動 */
    printf("Value: %d\n", *ptr);      /* 20 */

    ptr += 2;  /* 2つ先の要素へ移動 */
    printf("Value: %d\n", *ptr);      /* 40 */

    return 0;
}
```

ポインターを使った配列の走査

```
#include <stdio.h>

int main(void) {
    int arr[5] = {1, 2, 3, 4, 5};
    int *ptr;

    /* 方法1: 終端の一つ後ろを表すポインターと比較 */
    printf("Compare with end pointer:\n");
    for (ptr = arr; ptr < arr + 5; ptr++) {
        printf("%d ", *ptr);
    }
    printf("\n");

    /* 方法2: 最後の要素へのポインターと比較 */
    printf("Compare with last element:\n");
    ptr = arr;
    while (ptr <= &arr[4]) {
        printf("%d ", *ptr);
        ptr++;
    }
    printf("\n");

    return 0;
}
```

関数への配列の渡し方

関数の仮引数に書いた配列型は、対応するポインター型に調整されます。呼び出し側の配列式も先頭要素へのポインターへ変換され、そのポインター値が値渡しされます。配列全体がコピーされるわけではありません。

関数の宣言方法

```
/* 関数型としては、いずれも仮引数がint *に調整される */
void process_array(int arr[]);
void process_array(int *arr);
void process_array(int arr[10]);
```

3番目の10は、この宣言だけでは要素数の検査や受け渡しを行いません。関数が処理する要素数は、別の引数で明示します。

配列を受け取る関数の例

```
#include <stdio.h>

/* 配列の合計を計算する関数 */
int array_sum(int *arr, int size) {
    int sum = 0;
    int i;

    for (i = 0; i < size; i++) {
        sum += arr[i];  /* または sum += *(arr + i); */
    }

    return sum;
}

/* 配列の要素をすべて2倍にする関数 */
void double_array(int arr[], int size) {
    int i;

    for (i = 0; i < size; i++) {
        arr[i] *= 2;  /* 元の配列が変更される */
    }
}

int main(void) {
    int numbers[5] = {1, 2, 3, 4, 5};
    int total;
    int i;

    /* 合計を計算 */
    total = array_sum(numbers, 5);
    printf("Total: %d\n", total);

    /* 配列を2倍に */
    printf("Before: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    double_array(numbers, 5);

    printf("After: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    return 0;
}
```

多次元配列と行へのポインター

2次元配列のメモリ配置

```
int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

/* 要素は行優先で連続して配置される */
/* [1][2][3][4][5][6][7][8][9][10][11][12] */
```

2次元配列へのポインター

```
int matrix[3][4];
int (*ptr)[4] = matrix;  /* 4要素のint配列へのポインター */

/* 以下はすべて同じ要素にアクセス */
matrix[1][2];          /* 通常のアクセス */
*(*(matrix + 1) + 2); /* ポインター演算 */
ptr[1][2];            /* ポインター経由のアクセス */
```

式matrixは、先頭行へのポインターint (*)[4]に変換されます。列数の4はポインター演算で1行分進むために必要なので、省略できません。

実践的な応用例

配列の最大値を見つける（ポインター版）

```
#include <stdio.h>

int *find_max(int *arr, int size) {
    int *max_ptr = arr;
    int i;

    for (i = 1; i < size; i++) {
        if (*(arr + i) > *max_ptr) {
            max_ptr = arr + i;
        }
    }

    return max_ptr;  /* 最大値へのポインターを返す */
}

int main(void) {
    int numbers[5] = {34, 67, 12, 89, 45};
    int *max_ptr;

    max_ptr = find_max(numbers, 5);

    printf("Maximum: %d\n", *max_ptr);
    printf("Index: %ld\n", (long)(max_ptr - numbers));

    return 0;
}
```

find_maxは、arrが少なくともsize個の要素を指し、size > 0であることを前提とします。

文字列操作の基礎

```
#include <stdio.h>

/* 文字列の長さを計算（ポインター版） */
int string_length(const char *str) {
    const char *start = str;

    while (*str != '\0') {
        str++;
    }

    return str - start;  /* ポインターの差が文字数 */
}

int main(void) {
    char message[] = "Hello, World!";
    int len;

    len = string_length(message);
    printf("Length: %d\n", len);

    return 0;
}
```

よくある間違いと注意点

1. 配列の境界を超えたアクセス

```
int arr[5] = {1, 2, 3, 4, 5};
int *ptr = arr;

ptr += 10;  /* このポインター演算自体が未定義動作 */
```

同じ配列内の要素、または終端の一つ後ろを超えるポインターを生成してはいけません。終端の一つ後ろを表すポインターは比較には使えますが、間接参照はできません。

2. ポインターと配列の混同

```
int arr[5];
int *ptr;

sizeof(arr);  /* 5 * sizeof(int): 配列全体のサイズ */
sizeof(ptr);  /* ポインター自体のサイズ */
```

3. 関数での配列サイズ

```
void process(int arr[]) {
    /* 仮引数arrの型はint *に調整されるため、sizeof(arr)はポインターのサイズ */
    /* 配列のサイズは別途引数で渡す必要がある */
}
```

実例コード

完全な実装例は以下のファイルを参照してください。

配列とポインターの基本

- array_pointer_basics.c - C90準拠版
- array_pointer_basics_c99.c - C99準拠版

高度な配列操作

- advanced_array_ops.c - C90準拠版
- advanced_array_ops_c99.c - C99準拠版

学習のポイント

1. **配列式の変換**：配列式は、多くの文脈で先頭要素へのポインターに変換されます

2 **添字演算**:`arr[i]`は`*(arr + i)`と定義されています

3 **仮引数の型調整**:関数仮引数の`int arr[]`は`int *arr`に調整されます

4 **値渡し**:関数には変換後のポインター値が渡され、要素数は別に渡します

参考資料

- C90規格書
- ポインターと配列の詳細

演習問題

演習問題では、配列式の変換、添字演算、関数仮引数の型調整をコードで確認できます。

- 基礎問題:基本的な文法や概念の確認
- 応用問題:より実践的なプログラムの作成
- チャレンジ問題:高度な理解と実装力が必要な問題

第10章：文字列

対応C規格

- **主要対象：** C90
- **学習内容：** 文字列の基本、文字列操作関数、文字列配列の詳細操作、文字列の比較・検索・変換

学習目標

この章を完了すると、以下のことができるようになります。

- null文字で終端された文字列の表現を説明できる
- 標準ライブラリの文字列操作関数を使える
- 2次元文字配列と文字列へのポインター配列を使い分けられる
- 配列の容量を考慮して文字列を入出力できる

概要と詳細

文字列処理とは

文字列処理には、長さの取得、コピー、比較、検索、入力の検証などがあります。Cの文字列は長さを別に保持しないため、null文字までを参照できる配列領域が必要です。

文字列の基本概念

Cの文字列は、最初のnull文字`'\0'`までを含む文字の並びです。文字列を格納する配列には、内容に加えて終端のnull文字を置く領域が必要です。

文字列のメモリ表現

文字列`"Hello"`は、次の6個の文字として表現されます。

```
文字列 "Hello" のメモリ上の配置

インデックス: [0] [1] [2] [3] [4] [5]
内容:        'H' 'e' 'l' 'l' 'o' '\0'
             ↑                   ↑
             最初の文字           null終端文字（文字列の終わり）
```

文字列の4つの表現方法

```
/* 1. 文字配列として明示的に初期化 */
char str1[] = {'H', 'e', 'l', 'l', 'o', '\0'};
/* 各文字を個別に指定。最後に必ず'\0'が必要 */

/* 2. 文字列リテラルで初期化（最も一般的） */
char str2[] = "Hello";
/* コンパイラが自動的に'\0'を追加。サイズは6バイト */

/* 3. 固定サイズ配列に文字列を格納 */
char str3[10] = "Hello";
/* 10バイト確保。"Hello\0"の後の4バイトは'\0'で初期化 */

/* 4. 文字列リテラルの先頭を指すポインター */
const char *str4 = "Hello";
/* 文字列リテラルを変更しないことを型で示す */
```

null文字(`'\0'`)による終端

null終端文字は、文字列の終わりを示す特別な文字です。

```
char str[] = "ABC";
/* メモリ上では: 'A' 'B' 'C' '\0' と格納される */
```

```
/* null終端がない場合の問題 */
char bad_str[3] = {'A', 'B', 'C'};  /* '\0'がない */
printf("%s", bad_str);  /* 配列外まで読み得るため未定義動作 */
```

文字と文字列の違い

文字定数はシングルクォート、文字列リテラルはダブルクォートで表します。

```
/* 文字（1バイト） */
char ch = 'A';        /* シングルクォート */

/* 文字列（2バイト以上） */
char str[] = "A";     /* ダブルクォート */
/* 実際は 'A' と '\0' の2バイト */

/* 間違いやすい例 */
char wrong1 = "A";    /* エラー：文字列を文字型に代入 */
char wrong2[] = 'A';  /* エラー：文字で配列を初期化 */
```

文字列のサイズと長さ

```
char str[10] = "Hello";

/* サイズ：配列として確保されたメモリ量 */
sizeof(str)   /* 結果：10（バイト） */

/* 長さ：実際の文字数（'\0'を除く） */
strlen(str)   /* 結果：5（文字） */

/* メモリ配置 */
/* [H][e][l][l][o][\0][\0][\0][\0][\0] */
/*  0  1  2  3  4  5   6   7   8   9   */
```

文字列リテラルの特殊性

```
/* 文字列リテラルを変更すると未定義動作 */
const char *ptr = "Hello";

/* 配列にコピーすれば変更可能 */
char arr[] = "Hello";
arr[0] = 'h';  /* OK: "hello"になる */

/* 同じ文字列リテラルは共有される可能性 */
const char *p1 = "Hello";
const char *p2 = "Hello";
/* p1とp2は同じアドレスを指す可能性がある */
```

基本的な文字列操作

文字列の長さを取得

```
#include <string.h>
char str[] = "Hello";
size_t len = strlen(str);  /* 結果: 5 */
```

文字列のコピー

```
char src[] = "Hello";
char dest[20];
strcpy(dest, src);  /* srcをdestにコピー */
```

文字列の比較

```
char str1[] = "Hello";
char str2[] = "World";
int result = strcmp(str1, str2);  /* 0でない値が返される */
```

文字列の連結

```
char str1[20] = "Hello";
char str2[] = " World";
strcat(str1, str2);  /* str1が "Hello World" になる */
```

標準ライブラリの文字列関数

<string.h>の主要関数

1. **strlen()** - 文字列の長さを取得

```
size_t strlen(const char *s);
```

2. **strcpy()** - 文字列をコピー

```
char *strcpy(char *dest, const char *src);
```

3. **strncpy()** - 指定した個数の文字を配列へ書き込む

```
char *strncpy(char *dest, const char *src, size_t n);
```

4. **strcmp()** - 文字列を比較

```
int strcmp(const char *s1, const char *s2);
```

5. **strncmp()** - 指定長で比較

```
int strncmp(const char *s1, const char *s2, size_t n);
```

6. **strcat()** - 文字列を連結

```
char *strcat(char *dest, const char *src);
```

7. **strncat()** - 連結元から最大n文字を連結する

```
char *strncat(char *dest, const char *src, size_t n);
```

8. **strchr()** - 文字を検索

```
char *strchr(const char *s, int c);
```

9. **strstr()** - 文字列を検索

```
char *strstr(const char *haystack, const char *needle);
```

文字列配列の詳細操作

2次元文字配列(char配列)

```
char names[5][20];  /* 5つの名前、各最大19文字と終端のnull文字 */
char fruits[5][20] = {
    "Apple",
    "Banana",
    "Orange",
    "Grape",
    "Strawberry"
};
```

特徴:

- 固定サイズのメモリ領域を確保
- 各文字列は固定長で格納される
- メモリ使用量が予測可能
- 文字列の変更が可能
- サイズ = 行数 × 列数 × sizeof(char)

文字列へのポインター配列(const char *配列)

```
const char *fruits[] = {"Apple", "Banana", "Orange"};
const char *months[] = {
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
};
```

特徴:

- 文字列リテラルへのポインターを格納
- 各要素にはポインターの領域が必要
- 文字列の長さは可変
- 文字列リテラルの変更は不可
- サイズ = ポインター数 × sizeof(char*)

char配列とconst char *配列の使い分け

char配列を選ぶ場合:

- 文字列を変更する必要がある
- メモリ使用量を予測したい
- 全ての文字列が同程度の長さ

const char *配列を選ぶ場合:

- 文字列を変更しない(読み取り専用)
- 文字列ごとに異なる長さを扱う
- 文字列の長さがバラバラ

文字列配列の操作例

配列の検索

```
int search_string(char arr[][20], int size, const char *target) {
    int i;
    for (i = 0; i < size; i++) {
        if (strcmp(arr[i], target) == 0) {
            return i;  /* 見つかった位置 */
        }
    }
    return -1;  /* 見つからなかった */
}
```

文字列配列のソート(選択ソート)

```
void sort_strings(char arr[][20], int size) {
    int i, j, min_idx;
    char temp[20];

    for (i = 0; i < size - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < size; j++) {
            if (strcmp(arr[j], arr[min_idx]) < 0) {
                min_idx = j;
            }
        }
        if (min_idx != i) {
            strcpy(temp, arr[i]);
            strcpy(arr[i], arr[min_idx]);
            strcpy(arr[min_idx], temp);
        }
    }
}
```

文字列の検索と置換

文字の検索

- **strchr()** - 最初に見つかった文字の位置
- **strrchr()** - 最後に見つかった文字の位置

文字列の検索

- **strstr()** - 部分文字列の検索
- **大文字小文字を無視した検索**は標準関数では提供されていない

文字列の置換

C標準ライブラリには置換関数はないため、自分で実装する必要があります。

文字列の変換

大文字と小文字の変換

```
#include <ctype.h>
unsigned char c = 'a';
int upper = toupper(c);  /* 'A' */
int lower = tolower((unsigned char)'B'); /* 'b' */
```

toupperとtolowerの戻り値の型はintです。引数にはEOF、またはunsigned charで表現できる値を渡します。

数値との変換

```
#include <stdlib.h>
long num = strtol("123", NULL, 10);  /* 文字列からlongへ */
double d = strtod("3.14", NULL);     /* 文字列からdoubleへ */
```

atoiやatofでは変換エラーを判別できません。入力を検証する場合は、終了位置や範囲エラーを確認できるstrtolやstrtodを使います。

文字列の入力と出力

標準入力からの文字列読み込み

```
char buffer[100];
scanf("%99s", buffer);  /* 空白までを最大99文字読み込む */
fgets(buffer, sizeof(buffer), stdin);  /* 改行までを配列の容量内で読み込む */
```

安全な文字列入力

- scanf:%sを使う場合は、配列容量に合わせた最大フィールド幅を指定します
- fgets:最大読み取り数を指定でき、改行を配列に格納する場合があります

文字列処理のベストプラクティス

安全性の確保

1. **バッファーオーバーフローの防止**
   - コピー元の長さとコピー先の容量を確認する
   - C99以降では、書式付き出力にsnprintfを使用できる
2. **null終端の確保**
   - strncpyで切り詰めが起きる場合は、終端のnull文字を明示する
   - メモリ初期化を適切に行う
3. **戻り値のチェック**
   - 文字列関数の戻り値を適切にチェック
   - NULLポインターの可能性を考慮

効率性の向上

1. **適切なデータ構造の選択**
   - 固定長vs可変長の選択

- メモリ使用量の最適化

2. **文字列操作の最小化**
   - 不必要なコピーを避ける
   - インプレース操作の活用

よくある間違いとデバッグ

1. バッファーオーバーフロー

問題: 文字列が配列のサイズを超える

```
char buffer[5];
strcpy(buffer, "Hello World");  /* 配列の境界を越えて書き込むため未定義動作 */
```

対策:

- 書き込む文字列と終端のnull文字が配列に収まるか確認する
- C99以降では、書式付き出力にsnprintfを使用する

2. null終端の忘れ

問題: null文字がない文字列

```
char str[5] = {'H', 'e', 'l', 'l', 'o'};  /* null終端なし */
printf("%s", str);  /* 未定義動作 */
```

対策:

- 文字列リテラルを使用
- 明示的にnull文字を設定

3. 文字列リテラルの変更

問題: 読み取り専用の文字列を変更

```
char *str = "Hello";
str[0] = 'h';  /* 未定義動作 */
```

対策:

- 変更が必要な場合は配列を使用
- const修飾子を適切に使用

4. ポインターの誤用

問題: 無効なポインターの使用

```
char *str;
strcpy(str, "Hello");  /* 未初期化ポインター */
```

対策:

- ポインターを適切に初期化
- メモリ確保を忘れずに

実例コード

完全な実装例は以下のファイルを参照してください。

基本的な文字列操作

- string_basics.c - C90準拠版

- string_basics_c99.c - C99準拠版

文字列操作関数の活用

- string_functions.c - C90準拠版
- string_functions_c99.c - C99準拠版

文字列配列の実践と詳細操作

- string_array_operations.c - C90準拠版
- string_array_operations_c99.c - C99準拠版

文字列配列の完全活用例

- string_arrays.c - C90準拠版(配列章から移行)
- string_arrays_c99.c - C99準拠版(配列章から移行)

コンパイル方法

基本的なコンパイル(C90準拠)

```
gcc -std=c90 -Wall -Wextra -pedantic string_basics.c -o string_basics
```

Makefileを使用した場合

```
# 全てのプログラムをコンパイル

make all

# 特定のプログラムをコンパイル

make string_basics

# C99版をコンパイル

make string_basics_c99

# プログラムを実行

make run-all

# クリーンアップ

make clean
```

学習フローとコンパイル方法

推奨学習順序

1. **理論学習**: README.mdで基本概念を理解
2. **サンプルコード**: examples/の基本例を確認
3. **演習課題**: exercises/README.mdで課題を確認
4. **実装練習**: solutions/の解答例を参考に自分で実装

実践的な学習方法

1. **基本関数の理解** - strlen, strcpy, strcmp等の基本動作を確認
2. **安全な関数の活用** - strncpy, strncat等の安全版を使用
3. **実際の問題解決** - 具体的な文字列処理問題に取り組む
4. **エラー処理の実装** - 例外的なケースへの対応を学習

C90とC99の違い

文字列処理での主な違い

- **C90**: 変数宣言は関数の先頭でまとめて行う
- **C99**: 変数を使用する場所で宣言可能
- **コメント**: C90は /* */のみ、C99は // も使用可能
- **書式付き出力**:C99では`snprintf`が標準化された

注意事項とベストプラクティス

セキュリティの考慮

1. **入力検証の徹底**
   - ユーザー入力は常に検証
   - バッファーサイズを超える入力の制御
2. **容量を明示した操作**
   - `strcpy`や`strcat`を使う前に、書き込み先の容量を確認する
   - `strncpy`は必ずnull終端する関数ではなく、`strncat`の`n`も書き込み先の容量ではない点に注意する
   - C99以降では、書式付き出力に`snprintf`を利用できる
3. **メモリ管理**
   - 動的に確保したメモリの適切な解放
   - ダングリングポインターの回避

参考資料

- C標準ライブラリリファレンス
- セキュアコーディング標準

演習問題

演習問題では、null終端、配列容量、標準文字列関数の事前条件を確認できます。

- 基礎問題:基本的な文法や概念の確認
- 応用問題:より実践的なプログラムの作成
- チャレンジ問題:高度な理解と実装力が必要な問題

第11章:関数

対応C規格

- **主要対象:** C90
- **学習内容:** 関数の基本、引数と戻り値、関数のスコープ、再帰関数、関数プロトタイプ

学習目標

この章を完了すると、以下のことができるようになります。

- 関数の定義と呼び出しができる
- 引数と戻り値を適切に使える
- 関数のスコープを理解する
- 再帰関数を作成できる
- 関数プロトタイプの重要性を理解する

概要と詳細

関数とは

関数は、プログラムを「部品」に分けるための仕組みです。大きな問題を小さな部分に分割して解決する「分割統治」の考え方を実現します。

関数の基本概念

関数は、名前を付けた処理の単位です。入力を仮引数で受け取り、必要に応じて戻り値を返します。処理を関数へ分けると、同じ処理を再利用でき、修正やテストの範囲も限定できます。

関数を使う理由

1. **再利用:** 同じ処理を複数の場所から呼び出せます
2. **分割:** 大きな処理を、入力と出力が明確な単位へ分けられます
3. **保守:** 修正やテストの対象を関数単位で絞れます
4. **可読性:** 関数名によって処理の目的を示せます

関数の構成要素

```
戻り値の型 関数名(引数リスト)
{
    /* 関数本体 */
    return 戻り値;  /* 戻り値がある場合 */
}
```

関数の定義と宣言

関数を使うには、「宣言」と「定義」を理解する必要があります。

関数プロトタイプが必要な理由

関数プロトタイプは、関数の戻り値の型と仮引数の型を呼び出しより前にコンパイラへ知らせます。コンパイラはこの情報を使って、引数の個数や型を検査します。

関数プロトタイプ(前方宣言)

```
/* 関数プロトタイプ */
int add(int a, int b);
void print_message(void);
```

関数の定義

```
/* 関数の実装 */
int add(int a, int b)
{
    return a + b;
}

void print_message(void)
{
    printf("Hello, World!\n");
}
```

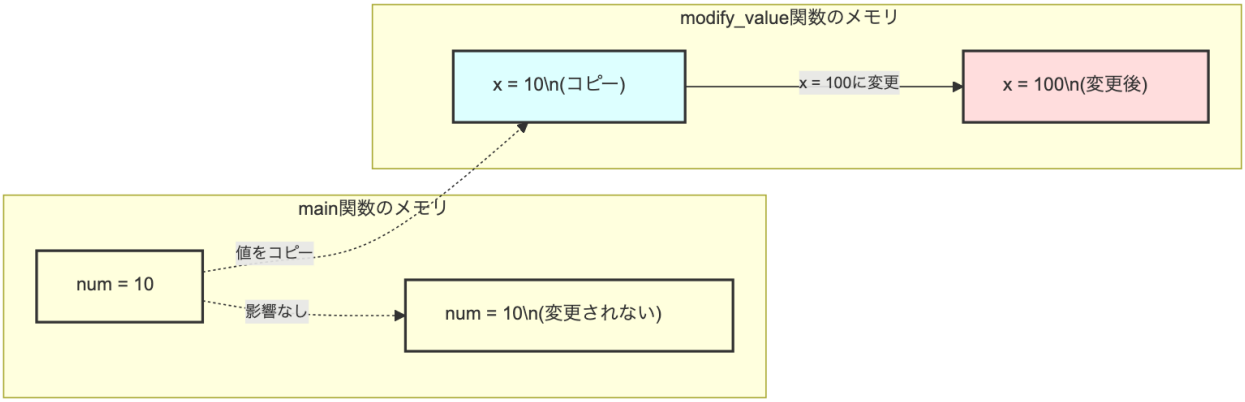
引数は常に値渡し

Cの関数呼び出しでは、すべての実引数が値渡しされます。ポインターを実引数にした場合も、関数が受け取るのはポインター値のコピーです。コピーしたポインターを間接参照すると、呼び出し側のオブジェクトを変更できます。

値渡し(Call by Value)

整数などの値を渡すと、仮引数はその値で初期化されます。仮引数を変更しても、呼び出し側のオブジェクトには影響しません。

値渡しのメカニズム



値渡しの実例

```
void modify_value(int x)
{
    printf("Inside before: x = %d\n", x);
    x = 100;  /* 元の変数には影響しない */
    printf("Inside after: x = %d\n", x);
}

int main(void)
{
    int num = 10;
    printf("Before call: num = %d\n", num);

    modify_value(num);

    printf("After call: num = %d\n", num);  /* 10のまま */
    return 0;
}
```

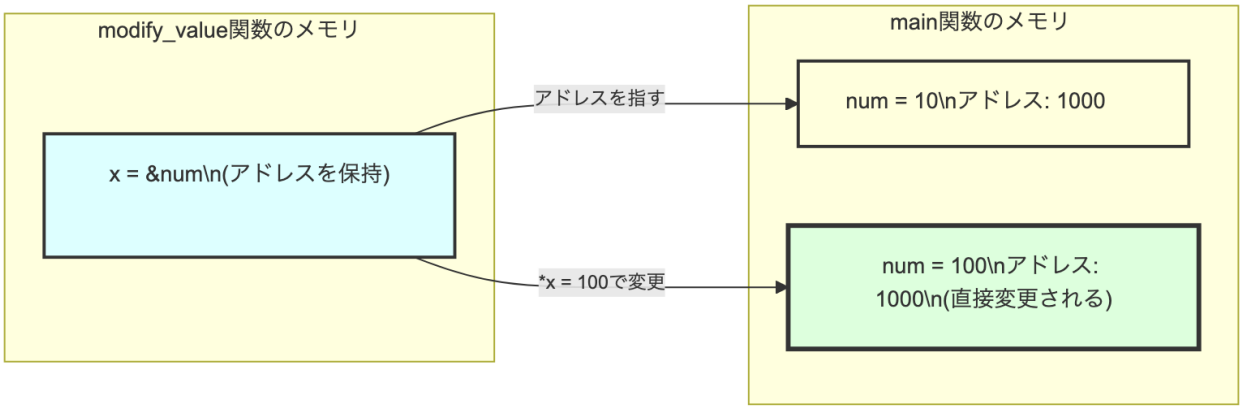
値渡しの性質

値渡しでは、仮引数への代入が呼び出し側のオブジェクトを直接変更しません。構造体も値渡しできますが、値が大きい場合はコピーのコストが判断材料になります。複数の結果は、構造体を戻り値にするか、出力先を指すポインターを渡して返せます。

ポインター値を渡す場合

オブジェクトのアドレスを表すポインター値を渡すと、関数は間接参照によってそのオブジェクトを読み書きできます。これは参照渡しという別の呼び出し方式ではなく、ポインター値の値渡しです。

ポインター値がコピーされる仕組み



ポインターを使った変更例

```
void modify_value(int *x)
{
    printf("Inside before: *x = %d\n", *x);
    *x = 100;  /* 元の変数を変更 */
    printf("Inside after: *x = %d\n", *x);
}

int main(void)
{
    int num = 10;
    printf("Before call: num = %d\n", num);

    modify_value(&num);  /* &でアドレスを渡す */

    printf("After call: num = %d\n", num);  /* 100に変更されている */
    return 0;
}
```

ポインターを渡す場合の性質

ポインターを介すと、呼び出し側のオブジェクトを変更したり、複数の出力先へ結果を書き込んだりできます。一方、関数はポインターが指す範囲や有効期間を型だけでは判定できません。呼び出し側と関数側で、NULLを許すか、何要素を参照できるか、書き込み可能かという契約をそろえる必要があります。

配列の引数渡し

配列を表す式は、関数呼び出しでは通常、先頭要素へのポインターに変換されます。

配列式と仮引数の型調整

```
void modify_array(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++) {
        arr[i] *= 2;  /* 元の配列を変更 */
    }
}

int main(void)
{
    int numbers[5] = {1, 2, 3, 4, 5};
    int i;

    printf("Before: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    modify_array(numbers, 5);

    printf("After: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);  /* 値が2倍になっている */
    }
    printf("\n");

    return 0;
}
```

関数が受け取る値

```
配列式は先頭要素へのポインターに変換される

numbers[5] = {1, 2, 3, 4, 5}

numbers  [1][2][3][4][5]
          
     先頭要素のアドレス

関数には、このポインター値のコピーが渡される
```

関数仮引数に書いた`int arr[]`は`int *arr`に調整されます。配列全体や要素数は渡されないため、要素数は別の引数で指定します。

構造体の引数渡し

構造体そのものを渡すと、構造体の値がコピーされます。構造体へのポインターを渡すと、ポインター値がコピーされ、間接参照によって呼び出し側の構造体を操作できます。

構造体の値渡し

```
struct Point {
    int x;
    int y;
};

void move_point(struct Point p)
{
    p.x += 10;  /* コピーを変更（元の構造体は変わらない） */
    p.y += 10;
}
```

構造体へのポインターを渡す場合

```
void move_point_ref(struct Point *p)
{
    p->x += 10;  /* 元の構造体を変更 */
    p->y += 10;
}
```

ポインターを引数にする例

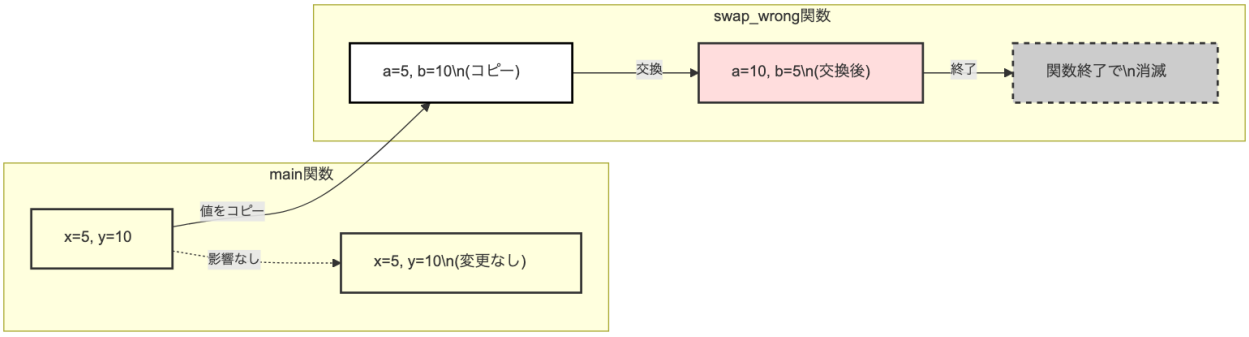
呼び出し側のオブジェクトを変更する関数や、複数の結果を書き込む関数は、出力先へのポインターを引数にします。

1. swap関数(値の交換)

2つの変数を交換するには、それぞれの変数へのポインターを関数へ渡します。

```
/* 値渡しでは交換できない（間違い例） */
void swap_wrong(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
    /* 関数内でコピーを交換しただけ */
}

/* 変数へのポインターを渡して交換 */
void swap_correct(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void)
{
    int x = 5, y = 10;

    printf("Before: x=%d, y=%d\n", x, y);

    swap_wrong(x, y);
    printf("After swap_wrong: x=%d, y=%d\n", x, y);  /* 5, 10のまま */

    swap_correct(&x, &y);
    printf("After swap_correct: x=%d, y=%d\n", x, y);  /* 10, 5に交換 */

    return 0;
}
```

swap_wrongが呼び出し側を変更できない理由



2. 複数の値を返す関数

Cの関数が直接返す戻り値は一つです。複数の出力先へのポインターを渡すと、それぞれのオブジェクトへ結果を書き込めます。

```
#include <limits.h>
#include <stddef.h>
#include <stdio.h>

/* 商と余りを出力引数へ書き込む */
int divide_with_remainder(int dividend, int divisor,
                          int *quotient, int *remainder)
{
    if (quotient == NULL || remainder == NULL) {
        return -1;
    }
    if (divisor == 0 || (dividend == INT_MIN && divisor == -1)) {
        return -2;
```

```
    }

    *quotient = dividend / divisor;
    *remainder = dividend % divisor;
    return 0;
}

/* 統計情報を一度に計算する */
int calculate_stats(const int arr[], int size,
                    int *min, int *max, double *average)
{
    int i;
    double sum = 0.0;

    if (size <= 0 || arr == NULL || min == NULL ||
        max == NULL || average == NULL) {
        return -1;
    }

    *min = *max = arr[0];

    for (i = 0; i < size; i++) {
        if (arr[i] < *min) *min = arr[i];
        if (arr[i] > *max) *max = arr[i];
        sum += arr[i];
    }

    *average = (double)sum / size;
    return 0;
}

int main(void)
{
    int q, r;
    int numbers[] = {23, 67, 12, 89, 45};
    int minimum, maximum;
    double avg;

    /* 除算の例 */
    if (divide_with_remainder(17, 5, &q, &r) == 0) {
        printf("17 / 5 = %d, remainder %d\n", q, r);  /* 3 余り 2 */
    }

    /* 統計計算の例 */
    if (calculate_stats(numbers, 5, &minimum, &maximum, &avg) == 0) {
        printf("Minimum: %d, maximum: %d, average: %.1f\n",
               minimum, maximum, avg);
    }

    return 0;
}
```

3. ポインター引数の検証とエラー処理

NULLを受け取る可能性がある関数は、間接参照の前にNULLかどうかを検査します。NULLを許可しない契約にする場合は、その事前条件をAPIの利用者へ明示します。

```
#include <limits.h>

/* 基本的なNULLチェック */
void safe_increment(int *ptr)
{
    if (ptr != NULL) {  /* NULLポインターチェック */
        (*ptr)++;
    }
}

/* より実践的な例：文字列の安全なコピー */
int safe_string_copy(char *dest, const char *src, int dest_size)
{
    int i;

    /* 引数の検証 */
    if (dest == NULL || src == NULL || dest_size <= 0) {
        return -1;  /* エラー */
    }

    /* 安全にコピー */
```

```
    for (i = 0; i < dest_size - 1 && src[i] != '\0'; i++) {
        dest[i] = src[i];
    }
    dest[i] = '\0';  /* null終端を保証 */

    return i;  /* コピーした文字数を返す */
}

/* エラーコードを返す関数の例 */
int safe_divide(int a, int b, int *result)
{
    if (result == NULL) {
        return -1;  /* 引数エラー */
    }

    if (b == 0 || (a == INT_MIN && b == -1)) {
        return -2;  /* ゼロ除算エラー */
    }

    *result = a / b;
    return 0;  /* 成功 */
}
```

4. 大きなデータ構造の効率的な処理

構造体が大きくなると、値渡しのコピーコストが無視できなくなります。

```
/* 大きな構造体の例 */
struct StudentRecord {
    char name[50];
    char id[20];
    int scores[10];
    double gpa;
    char address[100];
};

/* 構造体の値を渡す */
void print_student_by_value(struct StudentRecord student)
{
    printf("Name: %s\n", student.name);
    printf("GPA: %.2f\n", student.gpa);
}

/* const修飾した構造体へのポインター値を渡す */
void print_student_by_reference(const struct StudentRecord *student)
{
    if (student == NULL) return;

    printf("Name: %s\n", student->name);
    printf("GPA: %.2f\n", student->gpa);
}

/* 構造体を変更する関数 */
void update_gpa(struct StudentRecord *student, double new_gpa)
{
    if (student == NULL) return;

    student->gpa = new_gpa;
    printf("Updated %s GPA to %.2f\n",
           student->name, new_gpa);
}
```

5. 配列操作の実践例

配列式から変換されたポインターだけでは、参照できる要素数が分かりません。配列を処理する関数には、要素数や処理範囲も渡します。

```
/* 配列の要素をすべて特定の値に設定 */
void fill_array(int arr[], int size, int value)
{
    int i;

    if (arr == NULL || size <= 0) return;

    for (i = 0; i < size; i++) {
        arr[i] = value;
    }
```

```
}

/* 2つの配列を比較 */
int compare_arrays(const int arr1[], const int arr2[], int size)
{
    int i;

    if (arr1 == NULL || arr2 == NULL || size <= 0) {
        return -1;  /* エラー */
    }

    for (i = 0; i < size; i++) {
        if (arr1[i] != arr2[i]) {
            return 0;  /* 異なる */
        }
    }

    return 1;  /* 同じ */
}

/* 配列の一部を別の配列にコピー */
int copy_array_range(int dest[], int dest_size,
                     const int src[], int src_size,
                     int start, int count)
{
    int i;

    if (dest == NULL || src == NULL || start < 0 || count < 0) return -1;
    if (start > src_size || count > src_size - start) return -1;
    if (count > dest_size) return -1;

    for (i = 0; i < count; i++) {
        dest[i] = src[start + i];
    }
    return 0;
}
```

このコピー関数は、コピー元とコピー先の範囲が重ならないことを前提とします。

6. 戻り値と出力引数の組み合わせ

戻り値で処理結果の状態を返し、ポインターで指定された出力先へ値を書き込む設計があります。

```
#include <errno.h>
#include <limits.h>
#include <stdlib.h>

/* 文字列を整数に変換（エラーチェック付き） */
int string_to_int(const char *str, int *result)
{
    char *end;
    long value;

    if (str == NULL || result == NULL) {
        return -1;  /* 引数エラー */
    }

    errno = 0;
    value = strtol(str, &end, 10);
    if (end == str || *end != '\0' || errno == ERANGE ||
        value < INT_MIN || value > INT_MAX) {
        return -2;
    }

    *result = (int)value;
    return 0;  /* 成功 */
}

/* 使用例 */
int main(void)
{
    int num;
    int status;

    status = string_to_int("123", &num);
    if (status == 0) {
        printf("Converted: %d\n", num);
    } else {
```

```
        printf("Conversion failed: %d\n", status);
    }

    return 0;
}
```

7. 引数の型を選ぶ基準

オブジェクトの値を直接渡す例:

- 基本データ型(int, char, float, double)
- コピーのコストを許容できる構造体
- 関数から呼び出し側のオブジェクトを変更する必要がない場合

オブジェクトへのポインターを渡す例:

- 大きな構造体や配列
- 複数の値を返したい場合
- 元の値を変更する必要がある場合
- 構造体のコピーを避けたい場合

`const`修飾した型へのポインターを渡す例:

- 大きなデータを読むだけの場合
- 誤った変更を防ぎたい場合
- APIの意図を明確にしたい場合

const修飾子による保護

ポインターを介してオブジェクトを変更させない場合は、ポインターが指す型を`const`で修飾します。

```
/* 配列の内容を変更しない関数 */
int array_sum(const int arr[], int size)
{
    int i, sum = 0;

    /* arr[i] = 0;  コンパイルエラー (constで保護)  */

    for (i = 0; i < size; i++) {
        sum += arr[i];  /* 読み取りはOK */
    }

    return sum;
}
```

引数渡しのベストプラクティス

1. **基本データ型(int, char, floatなど)**
   - 通常は値渡しを使用
   - 呼び出し側の値を変更する場合は、そのオブジェクトへのポインターを渡す
2. **配列**
   - 配列式は先頭要素へのポインターに変換される
   - 変更しない場合はconstを付ける
3. **構造体**
   - コピーのコストを許容できる構造体:構造体の値を直接渡す
   - コピーを避ける構造体:構造体へのポインターを渡す

- 変更しない場合：const修飾した型へのポインターを渡す

4 **文字列**

- char配列またはchar*で渡す
- 変更しない場合はconstを付ける

様々な関数の種類

関数は、戻り値や仮引数の有無によって宣言が変わります。用途に応じて関数型を選びます。

戻り値のない関数（void関数）

処理だけ行って、結果を返さない関数です。

```
void print_header(void)
{
    printf("====================\n");
    printf(" Program start\n");
    printf("====================\n");
}

void greet_user(char *name)
{
    printf("Hello, %s\n", name);
}
```

配列を扱う関数

```
/* 配列の要素数を計算できないため、サイズを別途渡す必要がある */
int array_sum(int arr[], int size)
{
    int i, sum = 0;

    for (i = 0; i < size; i++)
    {
        sum += arr[i];
    }

    return sum;
}

/* 配列を初期化する関数 */
void initialize_array(int arr[], int size, int value)
{
    int i;

    for (i = 0; i < size; i++)
    {
        arr[i] = value;
    }
}
```

再帰関数

再帰関数は自分自身を呼び出す関数です。

再帰関数の例

再帰関数には、再帰を終了する基底条件と、問題を小さくする再帰呼び出しが必要です。次のfactorialはn - 1を渡して問題を縮小し、n <= 1で呼び出しを止めます。

```
/* 階乗を計算する再帰関数 */
int factorial(int n)
{
    if (n <= 1) {
        return 1;  /* 基底条件 */
    }
    return n * factorial(n - 1);  /* 再帰呼び出し */
}

/* フィボナッチ数列を計算する再帰関数 */
int fibonacci(int n)
```

```
{
    if (n <= 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

再帰関数の構成要素

1. **基底条件** - 再帰を終了する条件
2. **再帰呼び出し** - 自分自身を呼び出す部分
3. **問題の分割** - 元の問題をより小さな問題に分割

関数のスコープと生存期間

変数を利用できるソースコード上の範囲を**スコープ**、オブジェクトが保持される期間を**記憶域期間**と呼びます。両者は別の概念です。

ローカル変数とグローバル変数

関数内やブロック内で宣言した識別子の多くはブロックスコープを持ちます。関数の外で宣言した識別子はファイルスコープを持ちます。

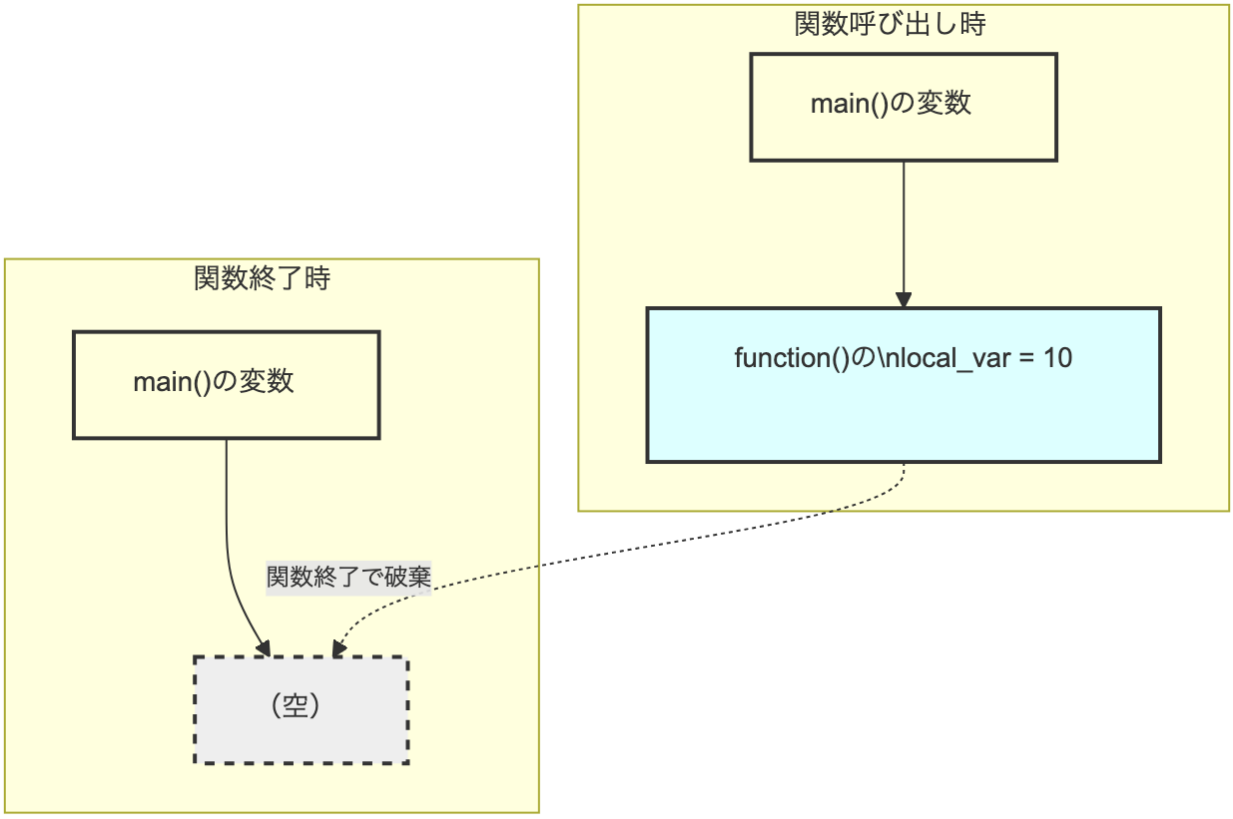
ローカル変数の詳細

関数内で宣言したローカル変数は、宣言位置を含むブロック内で参照できます。通常のローカル変数は自動記憶域期間を持ち、ブロックへ入るたびにオブジェクトの生存期間が始まります。

ローカル変数の特徴

```
void function_example(void)
{
    int local_var = 10;  /* この関数内でのみ有効 */

    {  /* ブロックスコープ */
        int block_var = 20;  /* このブロック内でのみ有効 */
        printf("block_var = %d\n", block_var);
    }  /* block_varはここで破棄される */

    /* printf("%d\n", block_var); エラー:block_varは見えない */

}  /* local_varはここで破棄される */
```

ローカル変数のメモリ配置



同名のローカル変数

```
void func1(void)
{
    int count = 0;  /* func1のcount */
    count++;
    printf("func1: count = %d\n", count);  /* 1 */
}

void func2(void)
{
    int count = 100;  /* func2のcount (func1とは別物)  */
    count++;
    printf("func2: count = %d\n", count);  /* 101 */
}

int main(void)
{
    int count = 50;  /* mainのcount */

    func1();  /* func1のcountを使用 */
    func2();  /* func2のcountを使用 */

    printf("main: count = %d\n", count);  /* 50 (変更されない)  */
    return 0;
}
```

グローバル変数の詳細

関数の外で宣言した変数はファイルスコープを持ちます。外部リンケージを持つ変数は、別の翻訳単位からextern宣言を介して参照できます。staticを付けた変数は内部リンケージを持ち、その翻訳単位だけで名前を参照できます。

グローバル変数の宣言と使用

```
/* グローバル変数の宣言 */
int global_count = 0;      /* 初期化あり */
double global_rate;        /* 初期化なし（0.0になる） */
char global_message[100];  /* 初期化なし（全て0になる） */

void increment_counter(void)
{
    global_count++;  /* どの関数からでもアクセス可能 */
}

void print_counter(void)
{
    printf("Counter: %d\n", global_count);
}

int main(void)
{
    print_counter();     /* カウンター: 0 */
    increment_counter();
    increment_counter();
    print_counter();     /* カウンター: 2 */

    global_count = 10;   /* main関数からも変更可能 */
    print_counter();     /* カウンター: 10 */

    return 0;
}
```

グローバル変数の初期化

```
/* グローバル変数は自動的に0で初期化される */
int g_int;          /* 0 */
double g_double;    /* 0.0 */
char g_char;        /* '\0' */
int g_array[10];    /* すべて0 */

/* 明示的な初期化も可能 */
int g_initialized = 100;
char g_string[] = "Hello, Global!";

/* 関数呼び出しによる初期化はできない */
/* int g_value = get_value(); 定数式ではないため初期化できない */
```

変数の可視性（スコープ）と名前の隠蔽

同じ名前の変数が複数ある場合、最も内側のスコープの変数が優先されます。

```
int value = 100;  /* グローバル変数 */

void test_scope(void)
{
    printf("1: value = %d\n", value);  /* 100（グローバル） */

    {
        int value = 200;  /* グローバル変数を隠蔽 */
        printf("2: value = %d\n", value);  /* 200 */

        {
            int value = 300;  /* 外側のブロックの変数を隠蔽 */
            printf("3: value = %d\n", value);  /* 300 */
        }

        printf("4: value = %d\n", value);  /* 200 */
    }
}

int main(void)
{
    test_scope();
    printf("5: value = %d\n", value);  /* 100（グローバル） */
    return 0;
}
```

グローバル変数の問題点と対策

1. 予期しない変更

```
int total_score = 0;  /* グローバル変数 */

void add_score(int points)
{
    total_score += points;
}

void reset_score(void)
{
    total_score = 0;  /* 他の処理が共有している状態も変更する */
}

/* より良い設計：引数と戻り値を使用 */
int add_score_safe(int current_score, int points)
{
    return current_score + points;
}
```

2. デバッグの困難さ

```
/* 悪い例：グローバル変数を多用 */
int g_width, g_height, g_area;

void calculate_area(void)
{
    g_area = g_width * g_height;
}

/* 良い例：引数と戻り値を使用 */
int calculate_area_safe(int width, int height)
{
    return width * height;
}
```

3. 名前の衝突

```
/* file1.c */
int counter = 0;  /* グローバル変数 */

/* file2.c */
int counter = 0;  /* 同じ外部名の定義が重複し、リンクに失敗する */

/* 対策：staticを使用してファイルスコープに限定 */
static int counter = 0;  /* このファイル内でのみ有効 */
```

ローカル変数とグローバル変数の使い分け

ローカル変数を使うべき場合（推奨）

```
/* 関数内で完結する処理 */
double calculate_average(int arr[], int size)
{
    int i;
    int sum = 0;  /* ローカル変数 */

    for (i = 0; i < size; i++) {
        sum += arr[i];
    }

    return (double)sum / size;
}

/* 一時的な計算 */
void print_multiplication_table(int n)
{
    int i, j;  /* ループカウンタはローカル変数 */

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }
}
```

グローバル変数が適切な場合(限定的)

```
/* 設定値やフラグ */
int debug_mode = 0;  /* デバッグモードのフラグ */
const double PI = 3.14159265359;  /* 定数 */

/* プログラム全体の状態 */
char program_name[256] = "MyApplication";
int error_count = 0;

void enable_debug(void)
{
    debug_mode = 1;
}

void log_message(const char *msg)
{
    if (debug_mode) {
        printf("[DEBUG] %s\n", msg);
    }
}
```

グローバル変数の安全な使用方法

1. constを使用して読み取り専用にする

```
const int MAX_USERS = 100;
const double TAX_RATE = 0.08;
const char VERSION[] = "1.0.0";
```

2. staticでファイルスコープに限定

```
/* このファイル内でのみ使用可能 */
static int file_local_counter = 0;

static void increment_counter(void)
{
    file_local_counter++;
}
```

3. アクセス関数を提供する

```
/* グローバル変数を直接公開しない */
static int score = 0;  /* ファイルスコープ */

/* アクセス関数を通じて操作 */
int get_score(void)
{
    return score;
}

void set_score(int new_score)
{
    if (new_score >= 0) {  /* 検証も可能 */
        score = new_score;
    }
}

void add_score(int points)
{
    if (points > 0) {
        score += points;
    }
}
```

変数のスコープを限定する指針

1. **可能な限りローカル変数を使用する**
   - バグが少なく、理解しやすいコードになる
2. **グローバル変数は最小限に**
   - 本当に必要な場合のみ使用
   - constやstaticで保護する

3 **明確な命名規則**
   - グローバル変数にはプレフィックスを付ける(例:g_count)
   - 意味のある名前を使用する
4 **初期化を忘れない**
   - ローカル変数は必ず初期化する
   - グローバル変数の初期値も明示する

static変数

```
void counter_function(void)
{
    static int count = 0;  /* 初回のみ初期化 */
    count++;
    printf("Call count: %d\n", count);
}
```

関数ポインターの基礎

関数ポインターは、指定した戻り値の型と仮引数の型を持つ関数を指します。同じ関数ポインター変数へ互換性のある別の関数を代入すると、呼び出す処理を実行時に選べます。

基本的な関数ポインター

```
/* 関数の定義 */
int add(int a, int b)
{
    return a + b;
}

int multiply(int a, int b)
{
    return a * b;
}

int main(void)
{
    /* 関数ポインターの宣言 */
    int (*operation)(int, int);

    /* 関数ポインターに関数を代入 */
    operation = add;
    printf("Add: %d\n", operation(5, 3));

    operation = multiply;
    printf("Multiply: %d\n", operation(5, 3));

    return 0;
}
```

実例コード

完全な実装例は以下のファイルを参照してください。

基本的な関数の使い方

- function_basics.c - C90準拠版
- function_basics_c99.c - C99準拠版

高度な関数の使い方

- advanced_functions.c - C90準拠版
- advanced_functions_c99.c - C99準拠版

コンパイルと実行

```
# 基本的な関数の例をコンパイル

gcc -Wall -Wextra -pedantic -std=c90 examples/function_basics.c -o function_basics

# 実行

./function_basics

# C99版をコンパイル

gcc -Wall -Wextra -pedantic -std=c99 examples/function_basics_c99.c -o function_basics_c99

# 数学関数を使う場合は-lmを追加

gcc -Wall -Wextra -pedantic examples/advanced_functions.c -lm -o advanced_functions
```

注意事項

関数を定義し、呼び出すときは次の点を確認します。

1. **関数プロトタイプ**:呼び出しより後で関数を定義する場合は、先にプロトタイプを宣言する

```
/* 不適切: 呼び出しより前に宣言がない */
int main(void) {
    add(1, 2);
    return 0;
}
int add(int a, int b) { return a + b; }
```

2. **配列の扱い**:配列式から変換されたポインターと、処理する要素数を一緒に渡す

```
/* NG: サイズが分からない */
void print_array(int arr[]) { /* 要素数不明 */ }

/* OK: サイズも渡す */
void print_array(int arr[], int size) { /* OK */ }
```

3. **再帰の深さ**:再帰が深くなると、処理系が利用できる呼び出し用の記憶領域を使い切る可能性がある
4. **ファイルスコープ変数**:共有する範囲を限定し、必要ならstaticで内部リンケージにする
5. static**変数**:ブロックを抜けても値が保持されるため、呼び出し間で状態を共有する意図を明確にする

次のステップ

関数の基本を理解したら、以下のトピックに進みましょう。

1. より複雑な関数の設計パターン
2. 関数ポインターと高階関数
3. 可変長引数関数(stdarg.h)
4. インライン関数(C99以降)
5. ライブラリ関数の作成

学習フローとコンパイル方法

推奨学習順序

1. **理論学習**: README.mdで基本概念を理解
2. **サンプルコード**: examples/の基本例を確認
3. **演習課題**: exercises/README.mdで課題を確認
4. **実装練習**: solutions/の解答例を参考に自分で実装

Makefileを使用したコンパイル

```
# 全てのプログラムをコンパイル

make all

# 特定のプログラムをコンパイル

make function_basics

# C99版をコンパイル

make function_basics_c99

# プログラムを実行

make run-all

# クリーンアップ

make clean
```

参考資料

- C90規格書
- C99規格書

演習問題

演習問題では、プロトタイプ、値渡し、ポインター引数、再帰、スコープを確認できます。

- 基礎問題：基本的な文法や概念の確認
- 応用問題：より実践的なプログラムの作成
- チャレンジ問題：高度な理解と実装力が必要な問題

第12章:ビット操作

対応C規格

- **主要対象:** C90
- **学習内容:** ビット演算、ビットマスク、ビットフィールド、実践的なビット操作技術

学習目標

この章を完了すると、以下のことができるようになります。

- 符号なし整数にビット演算子を適用できる
- ビットマスクを使ったフラグ管理ができる
- ビットフィールドを理解して活用できる
- シフト演算の定義域を説明できる
- ビットフィールドの移植性上の制約を説明できる

概要と詳細

ビット操作とは

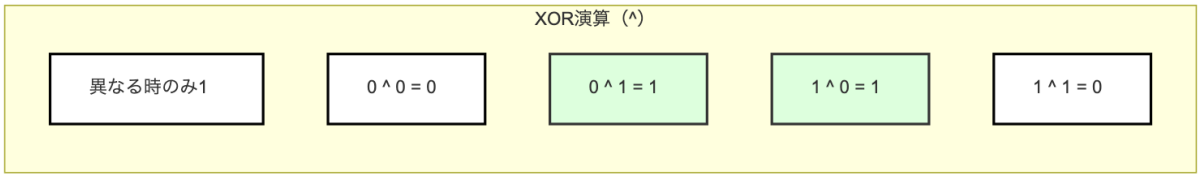
ビット操作は、整数型の各ビットに論理演算やシフト演算を適用する処理です。複数の真偽値をフラグとしてまとめる場合や、外部形式のビット位置を読み書きする場合に使います。演算結果を予測しやすくするため、通常は符号なし整数型を使います。

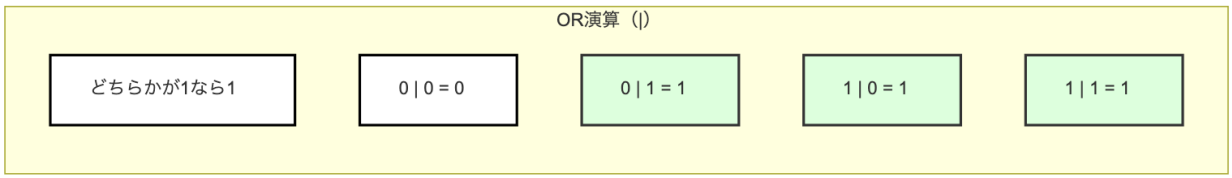
ビット演算子の詳細

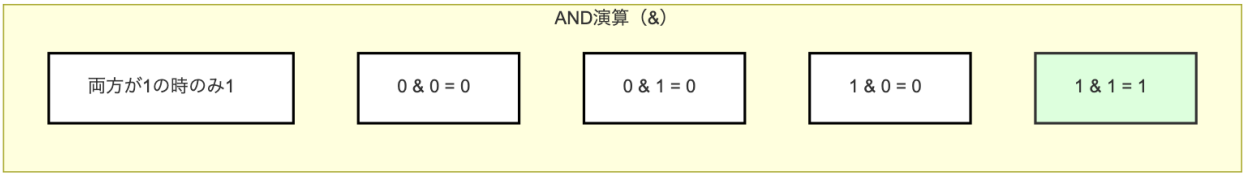
ビット演算子一覧

演算子	名称	動作	使用例
&	AND	両方が1の時1	`a & b`
|	OR	どちらかが1なら1	`a | b`
^	XOR	異なる時1	`a ^ b`
~	NOT	ビット反転	`~a`
<<	左シフト	ビットを左へ移動	`a << 2`
>>	右シフト	ビットを右へ移動	`a >> 2`

ビット演算の真理値表







ビットマスクの活用

ビットマスクとは

ビットマスクは、特定のビットを操作するためのパターンです。

```
/* 基本的なビットマスク */
#define BIT0 0x01  /* 00000001 */
#define BIT1 0x02  /* 00000010 */
#define BIT2 0x04  /* 00000100 */
#define BIT3 0x08  /* 00001000 */
#define BIT4 0x10  /* 00010000 */
#define BIT5 0x20  /* 00100000 */
#define BIT6 0x40  /* 01000000 */
#define BIT7 0x80  /* 10000000 */
```

ビット操作の基本テクニック

1. 特定ビットのセット(1にする)

```
unsigned char flags = 0x00;
flags |= BIT3;  /* ビット3を1にセット */
/* flags = 00001000 */
```

2. 特定ビットのクリア(0にする)

```
flags &= ~BIT3;  /* ビット3を0にクリア */
/* flags = 00000000 */
```

3. 特定ビットの反転(トグル)

```
flags ^= BIT3;  /* ビット3を反転 */
/* 0なら1に、1なら0に */
```

4. 特定ビットのテスト

```
if (flags & BIT3) {
    /* ビット3が1の場合 */
}
```

フラグ管理システム

実用的なフラグシステムの例

```
/* ファイルアクセス権限 */
#define PERM_READ    0x04  /* 100 */
#define PERM_WRITE   0x02  /* 010 */
#define PERM_EXECUTE 0x01  /* 001 */

/* ゲームの状態フラグ */
#define STATE_RUNNING   0x01
#define STATE_PAUSED    0x02
#define STATE_GAMEOVER  0x04
#define STATE_DEBUG     0x08

/* 使用例 */
unsigned char permissions = 0;
permissions |= PERM_READ | PERM_WRITE;  /* 読み書き権限を付与 */

if (permissions & PERM_WRITE) {
    /* 書き込み権限がある場合の処理 */
}
```

効率的なビット操作アルゴリズム

1. ビットカウント(1の個数を数える)

```
/* Brian Kernighanのアルゴリズム */
int count_bits(unsigned int n) {
    int count = 0;
    while (n) {
        n &= (n - 1);  /* 最下位の1を消す */
        count++;
    }
    return count;
}
```

2. 2のべき乗判定

```
int is_power_of_two(unsigned int n) {
    return n && !(n & (n - 1));
}
```

3. ビットスワップ

```
/* XORを使った値の交換 */
void swap_without_temp(int *a, int *b) {
    if (a == b) {
        return;
    }
    *a ^= *b;
    *b ^= *a;
    *a ^= *b;
}
```

同じオブジェクトへのポインターを二つ渡すとXOR交換は値を0にするため、事前にa == bを検査しています。通常は一時変数を使う交換の方が、別名参照の条件が少なく意図も明確です。

ビットフィールド

ビットフィールドとは

構造体のメンバーに対してビット単位でサイズを指定できる機能です。

基本的な宣言

```
struct PackedData {
    unsigned int flag1 : 1;   /* 1ビット */
    unsigned int flag2 : 1;   /* 1ビット */
    unsigned int value : 6;   /* 6ビット */
    unsigned int type  : 4;   /* 4ビット */
    unsigned int reserved : 4; /* 4ビット (予約) */
};
```

ビットフィールドの使用例

```
#include <stdio.h>

/* 日付を圧縮して格納 */
struct Date {
    unsigned int day   : 5;  /* 1-31 (5ビット) */
    unsigned int month : 4;  /* 1-12 (4ビット) */
    unsigned int year  : 7;  /* 0-127 (7ビット) 2000年からのオフセット */
};

int main(void) {
    struct Date today = {15, 12, 24};  /* 2024年12月15日 */

    printf("Date: %u-%u-%u\n",
           2000 + today.year, today.month, today.day);
    printf("Size: %lu bytes\n", (unsigned long)sizeof(struct Date));

    return 0;
}
```

ハードウェアレジスタを表す場合の制約

```
/* マイコンのレジスタ定義例 */
struct ControlRegister {
    unsigned int enable     : 1;   /* ビット0: 有効/無効 */
    unsigned int interrupt  : 1;   /* ビット1: 割り込み許可 */
    unsigned int mode       : 2;   /* ビット2-3: 動作モード */
    unsigned int speed      : 3;   /* ビット4-6: 速度設定 */
    unsigned int reserved   : 1;   /* ビット7: 予約 */
};
```

ビットフィールドの割り当て順序や境界は処理系定義です。特定のビット配置を要求するハードウェアレジスタへこの構造体をそのまま重ねられるかは、対象処理系のABIとコンパイラ仕様を確認する必要があります。移植性が必要なコードでは、符号なし整数値とビットマスクを使って各フィールドを読み書きします。

ビットフィールドの注意点

1. 移植性の問題

- ビットフィールドのレイアウトは処理系依存
- エンディアンの影響を受ける
- パディングの挿入位置が異なる場合がある

2. アドレスが取得できない

```
struct BitField {
    unsigned int flag : 1;
};

struct BitField bf;
/* &bf.flag は制約違反：ビットフィールドのアドレスは取得できない */
```

3. 配列にできない

```
struct Invalid {
    unsigned int flags[8] : 1;  /* 制約違反：配列をビットフィールドにはできない */
};
```

実践的な応用例

RGB565形式の色操作

```
/* 16ビットRGB (5-6-5形式) */
typedef struct {
    unsigned int blue  : 5;
    unsigned int green : 6;
    unsigned int red   : 5;
} RGB565;

/* または単一の16ビット値として */
#define RGB565_RED_MASK   0xF800  /* 11111000 00000000 */
#define RGB565_GREEN_MASK 0x07E0  /* 00000111 11100000 */
#define RGB565_BLUE_MASK  0x001F  /* 00000000 00011111 */

unsigned short make_rgb565(int r, int g, int b) {
    return ((r & 0x1F) << 11) | ((g & 0x3F) << 5) | (b & 0x1F);
}
```

ネットワークプロトコルのヘッダ

```
/* 論理上のフィールドを表す例。送受信バイト列へ直接重ねない */
struct PacketHeader {
    unsigned int version  : 4;   /* プロトコルバージョン */
    unsigned int type     : 4;   /* パケットタイプ */
    unsigned int flags    : 8;   /* 各種フラグ */
    unsigned int length   : 16;  /* データ長 */
};
```

ネットワーク上のバイト列では、ビット順序、バイト順序、パディングをプロトコルが指定します。ビットフィールドのメモリ表現には依存せず、マスクとシフトで明示的に直列化します。

ビット操作のベストプラクティス

1. マクロの活用

```
#define SET_BIT(value, bit)    ((value) |= (1U << (bit)))
#define CLEAR_BIT(value, bit)  ((value) &= ~(1U << (bit)))
#define TOGGLE_BIT(value, bit) ((value) ^= (1U << (bit)))
#define TEST_BIT(value, bit)   ((value) & (1U << (bit)))
```

`bit`は0以上かつ`unsigned int`の幅未満でなければなりません。この範囲を外れるシフトは未定義動作です。

2. 型の明確化

```
/* 符号なし型を使用してシフトの動作を明確に */
unsigned int flags = 0;
unsigned char byte_data = 0xFF;
```

3. 定数の使用

```
/* マジックナンバーを避ける */
#define MAX_FLAGS 32
#define BYTE_MASK 0xFF
#define NIBBLE_MASK 0x0F
```

よくある間違いとデバッグ

1. 符号付き整数の右シフト

```
int negative = -16;
int result = negative >> 2;
/* 負の符号付き整数の右シフト結果は処理系定義 */
```

2. シフト量の範囲外

```
unsigned int value = 1;
value <<= 32;  /* unsigned intが32ビットなら未定義動作 */
```

3. 演算子の優先順位

```
/* 間違い */
if (flags & BIT0 == 0)  /* ==が先に評価される */

/* 正しい */
if ((flags & BIT0) == 0)
```

実例コード

完全な実装例は以下のファイルを参照してください。

ビット操作の基本

- bit_basics.c - C90準拠版
- bit_basics_c99.c - C99準拠版

ビットフィールドの活用

- bitfield_examples.c - C90準拠版
- bitfield_examples_c99.c - C99準拠版

実践的な応用

- bit_applications.c - C90準拠版
- bit_applications_c99.c - C99準拠版

学習のポイント

1. **符号なし型**：ビット演算では、値の表現とシフト結果を予測しやすい符号なし整数型を使います
2. **シフト量**：右オペランドが負、または左オペランドの型の幅以上なら未定義動作です
3. **ビットマスク**：操作するビットだけを1にしたマスクで、セット、クリア、反転、検査を行います
4. **ビットフィールド**：メモリ配置を外部形式として利用する場合は、処理系の仕様を確認します

参考資料

- ビット演算
- ビットフィールド

演習問題

演習問題では、マスク、シフト量、符号なし整数、ビットフィールドの制約を確認できます。

- 基礎問題：基本的な文法や概念の確認
- 応用問題：より実践的なプログラムの作成
- チャレンジ問題：高度な理解と実装力が必要な問題

第13章：構造体

対応C規格

- **主要対象：** C90/C11
- **学習内容：** 構造体の基本、構造体とポインター、構造体配列、複雑なデータ構造の構築

学習目標

この章を完了すると、以下のことができるようになります。

- 構造体の定義と使用ができる
- 構造体ポインターを適切に操作できる
- 構造体配列を操作できる
- 構造体の値渡しとポインター値の受け渡しを区別できる
- パディングを考慮して構造体のサイズを確認できる

概要と詳細

構造体とは

構造体は、型の異なる複数のメンバを一つのオブジェクトにまとめる集成体型です。座標のxとyのように、同時に扱う値へ一つの型名を与えられます。

構造体で関連する値をまとめる

構造体がないと、関連するデータをバラバラに管理することになります。

```
/* 構造体を使わない場合は、値の対応関係を添字で管理する */
char names[10][50];
int ids[10];
int ages[10];

/* 構造体を使う場合は、学生ごとの値を一つの要素にまとめる */
struct student students[10];
```

構造体の基本概念

構造体の定義

```
struct 構造体名 {
    データ型 メンバ名1;
    データ型 メンバ名2;
    /* ... */
};
```

基本的な構造体の例

```
/* 学生の情報を表す構造体 */
struct student {
    char name[50];
    int id;
    int age;
    float gpa;
};

/* 2次元座標を表す構造体 */
struct point {
    int x;
    int y;
};

/* 日付を表す構造体 */
struct date {
    int year;
    int month;
    int day;
};
```

構造体変数の宣言と初期化

構造体変数の宣言

```
struct student s1;           /* 単一の構造体変数 */
struct student s2, s3;       /* 複数の構造体変数 */
struct student class[30];    /* 構造体配列 */
```

構造体の初期化

```
/* 宣言と同時に初期化 */
struct point p1 = {10, 20};

/* メンバ指定初期化（C99以降） */
struct point p2 = {.x = 15, .y = 25};

/* 後から初期化 */
struct student s;
strcpy(s.name, "山田太郎");
s.id = 12345;
s.age = 20;
s.gpa = 3.5;
```

構造体メンバのアクセス

ドット演算子(.)

構造体変数のメンバにアクセスします。

```
struct point p = {10, 20};

printf("x: %d\n", p.x);
printf("y: %d\n", p.y);

p.x = 30;  /* メンバの値を変更 */
p.y = 40;
```

アロー演算子(->)

構造体ポインターのメンバにアクセスします。

```
struct point p = {10, 20};
struct point *ptr = &p;

printf("x: %d\n", ptr->x);
printf("y: %d\n", ptr->y);

ptr->x = 50;  /* ポインター経由でメンバを変更 */
ptr->y = 60;
```

構造体とポインター

構造体ポインターの基本

```
struct student s = {"田中花子", 67890, 19, 3.8};
struct student *ptr = &s;

/* 以下は同じ結果 */
printf("Name: %s\n", s.name);
printf("Name: %s\n", ptr->name);
printf("Name: %s\n", (*ptr).name);
```

構造体ポインターの利点

1. **コピーの回避**:構造体の値ではなく、構造体へのポインター値を渡せます
2. **呼び出し側の変更**:間接参照を通じて、呼び出し側の構造体を変更できます
3. **読み取り専用の指定**:`const struct point *`のように、変更しない契約を型で示せます

構造体と関数

構造体を値渡しする関数

```
void print_point(struct point p)
{
    printf("Point: (%d, %d)\n", p.x, p.y);
}

int main(void)
{
    struct point p = {10, 20};
    print_point(p);  /* 構造体のコピーが作られる */
    return 0;
}
```

構造体ポインターを渡す関数

```
void print_point_ptr(const struct point *p)
{
    printf("Point: (%d, %d)\n", p->x, p->y);
}

void move_point(struct point *p, int dx, int dy)
{
    p->x += dx;
    p->y += dy;
}

int main(void)
{
    struct point p = {10, 20};
    print_point_ptr(&p);
    move_point(&p, 5, 3);
    print_point_ptr(&p);  /* (15, 23) */
    return 0;
}
```

構造体を返す関数

```
struct point add_points(struct point p1, struct point p2)
{
    struct point result;
    result.x = p1.x + p2.x;
    result.y = p1.y + p2.y;
    return result;
}

struct point create_point(int x, int y)
{
    struct point p = {x, y};
    return p;
}
```

構造体配列

構造体配列の宣言と初期化

```
/* 学生配列の宣言 */
struct student class[3];

/* 初期化 */
struct student class[3] = {
    {"山田太郎", 12345, 20, 3.5},
    {"佐藤花子", 12346, 19, 3.8},
    {"田中次郎", 12347, 21, 3.2}
};
```

構造体配列の操作

```
int i;

/* 全学生の情報を表示 */
for (i = 0; i < 3; i++) {
    printf("Student %d: %s (ID: %d)\n",
           i+1, class[i].name, class[i].id);
}

/* 特定の学生を検索 */
for (i = 0; i < 3; i++) {
    if (class[i].id == 12346) {
        printf("Found: %s\n", class[i].name);
        break;
    }
}
```

ネストした構造体

構造体の中に構造体

```
struct date {
    int year;
    int month;
    int day;
};

struct person {
    char name[50];
    struct date birthday;  /* 構造体をメンバとして含む */
    int age;
};

/* 使用例 */
struct person p = {
    "鈴木一郎",
    {1990, 5, 15},  /* birthday */
    33
};

printf("Birthday: %04d-%02d-%02d\n",
       p.birthday.year, p.birthday.month, p.birthday.day);
```

構造体の動的メモリ割り当て

malloc()を使った構造体の動的確保

```
#include <stdlib.h>

/* 単一の構造体を動的確保 */
struct student *s = malloc(sizeof *s);

if (s == NULL) {
    fprintf(stderr, "Allocation failed\n");
    return 1;
}

/* 構造体の使用 */
strcpy(s->name, "動的太郎");
s->id = 99999;
s->age = 22;

/* メモリ解放 */
free(s);
s = NULL;

/* 構造体配列を動的確保 */
int size = 100;
struct student *students = malloc((size_t)size * sizeof *students);

if (students != NULL) {
    /* 構造体配列を使用 */
    free(students);
}
```

Cではvoid *からオブジェクトへのポインターへ暗黙に変換できるため、mallocの戻り値をキャストする必要はありません。sizeof *sの形式なら、ポインターの型を変更したときも確保サイズが追従します。

typedefを使った構造体

typedefの利点

```
/* typedef なし */
struct point {
    int x;
    int y;
};
struct point p1, p2;  /* 毎回 struct が必要 */

/* typedef あり */
typedef struct {
    int x;
    int y;
} Point;

Point p1, p2;
```

typedefの使用例

```
typedef struct {
    char name[50];
    int id;
    float gpa;
} Student;

typedef struct {
    int year;
    int month;
    int day;
} Date;

/* 使用が簡潔に */
Student s1, s2;
Date today = {2024, 12, 25};
```

共用体(union)

共用体の基本概念

共用体は、同じメモリ領域を複数のメンバで共有します。

```
union data {
    int i;
    float f;
    char c;
};

union data d;
d.i = 10;
printf("int: %d\n", d.i);

d.f = 3.14f;
printf("float: %.2f\n", d.f);
/* 最後に格納したメンバはd.f */
```

最後に値を格納したメンバとは異なるメンバを読むと、格納した値のオブジェクト表現を別の型として解釈します。結果は型や処理系に依存し、トラップ表現を生成する可能性もあるため、移植可能な型変換の代わりには使えません。

共用体の用途

- 同時には一つしか使わない複数の表現で記憶域を共有する
- 別のタグメンバと組み合わせて、現在有効なメンバを管理する

構造体の高度な使用法

自己参照構造体(リンクリスト)

```
typedef struct node {
    int data;
    struct node *next;  /* 自分自身の型へのポインター */
} Node;

/* リンクリストの簡単な例 */
Node *head = NULL;
Node *new_node = malloc(sizeof *new_node);
if (new_node != NULL) {
    new_node->data = 42;
    new_node->next = head;
    head = new_node;
}
```

関数ポインターを含む構造体

```
typedef struct {
    char name[20];
    int (*calculate)(int, int);
} Operation;

int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }

Operation ops[] = {
    {"加算", add},
    {"乗算", multiply}
};

/* 使用例 */
int result = ops[0].calculate(5, 3);  /* add(5, 3) */
```

よくある間違いとデバッグ

1. 構造体の比較

問題: 構造体を直接比較しようとする

```
struct point p1 = {10, 20};
struct point p2 = {10, 20};
if (p1 == p2) { /* 制約違反:構造体には==を適用できない */ }
```

対策：

```
int points_equal(struct point p1, struct point p2)
{
    return (p1.x == p2.x && p1.y == p2.y);
}

if (points_equal(p1, p2)) { /* 正しい */ }
```

2. 未初期化構造体の使用

問題： 構造体メンバが未初期化

```
struct student s;
printf("Name: %s\n", s.name);  /* 未初期化データ */
```

対策：

```
struct student s = {0};  /* ゼロ初期化 */
/* または */
memset(&s, 0, sizeof(s));
```

3. 文字列の代入ミス

問題： 文字列を直接代入しようとする

```
struct student s;
s.name = "山田太郎";  /* 制約違反：配列には代入できない */
```

対策：

```
struct student s;
strcpy(s.name, "山田太郎");  /* strcpy を使用 */
```

4. 構造体ポインターの初期化忘れ

問題： NULLポインターの使用

```
struct student *s;
strcpy(s->name, "太郎");  /* 未初期化ポインターの間接参照は未定義動作 */
```

対策：

```
struct student *s = malloc(sizeof *s);
if (s != NULL) {
    strcpy(s->name, "太郎");
}
```

パフォーマンスとメモリ効率

構造体のパディング

```
struct example {
    char c;
    int i;
    char c2;
};
```

処理系は、各メンバの間や構造体の末尾にパディングを挿入できます。たとえば`sizeof(int) == 4`で`int`を4バイト境界に配置する処理系では、この構造体が12バイトになる場合があります。実際のサイズは`sizeof(struct example)`で確認します。

メモリ効率の改善

```
/* パディングが増える場合がある配置 */
struct bad {
    char c1;
    int i;
    char c2;
};

/* パディングが減る場合がある配置 */
struct good {
    int i;
    char c1;
    char c2;
};
```

メンバ順の変更がサイズを減らすかどうかは、各型のサイズとアラインメントに依存します。外部形式やABIとの互換性が必要な構造体では、サイズだけを理由にメンバ順を変更してはいけません。

実例コード

完全な実装例は以下のファイルを参照してください。

構造体の基本操作

- struct_basics.c - C90準拠版
- struct_basics_c99.c - C99準拠版

構造体とポインター

- struct_pointers.c - C90準拠版
- struct_pointers_c99.c - C99準拠版

複雑なデータ構造

- complex_structures.c - C90準拠版
- complex_structures_c99.c - C99準拠版

コンパイル方法

基本的なコンパイル(C90準拠)

```
gcc -std=c90 -Wall -Wextra -pedantic struct_basics.c -o struct_basics
```

Makefileを使用した場合

```
# 全てのプログラムをコンパイル

make all

# 特定のプログラムをコンパイル

make struct_basics

# C99版をコンパイル

make struct_basics_c99

# プログラムを実行

make run-all

# クリーンアップ

make clean
```

学習フローとコンパイル方法

推奨学習順序

1. **理論学習:** README.mdで基本概念を理解
2. **サンプルコード:** examples/の基本例を確認
3. **演習課題:** exercises/README.mdで課題を確認
4. **実装練習:** solutions/の解答例を参考に自分で実装

学習のコツ

1. **実際のデータでモデリング:** 身近なものを構造体で表現
2. **段階的構築:** 単純な構造体から複雑なものへ
3. **メモリ図の描画:** ポインターと構造体の関係を視覚化
4. **実用例の実装:** 学生管理システムなどの小さなプロジェクト

C90とC11の違い

C90での制限

- メンバ指定初期化なし
- ブロック内の宣言は、そのブロックの文より前に置く
- コメントは `/* */` のみ

C11での拡張

- 匿名構造体・共用体
- `_Static_assert` での構造体サイズチェック

メンバ指定初期化はC99で追加された機能であり、C11固有ではありません。

参考資料

- examples/ - 実装例(C90、C99両対応)
- exercises/ - 演習問題
- solutions/ - 解答例
- C標準ライブラリリファレンス
- 構造体設計ベストプラクティス

演習問題

演習問題では、構造体の値渡し、ポインターによる操作、配列、パディングを確認できます。

- 基礎問題:基本的な文法や概念の確認
- 応用問題:より実践的なプログラムの作成
- チャレンジ問題:高度な理解と実装力が必要な問題

第14章：関数ポインター

対応C規格

- **主要対象：** C90
- **学習内容：** 関数ポインターの基本、関数ポインター配列、コールバック関数、高階関数の実装

学習目標

この章を完了すると、以下のことができるようになります。

- 関数ポインターの概念を理解し、適切に宣言・使用できる
- 関数ポインターを引数として渡す関数を作成できる
- 関数ポインター配列を使った表駆動の分岐を書ける
- コールバック関数の仕組みを理解し、実装できる
- 互換性のある関数型を見分けられる

概要と詳細

関数ポインターとは

関数ポインターは、特定の関数型を持つ関数を指すポインターです。指している関数は、通常の関数呼び出しと同じ形式で呼び出せます。

関数ポインターを使う場面

1. **実行時の選択：**互換性のある複数の関数から、呼び出す関数を選びます
2. **表駆動の分岐：**番号や演算子と処理を表の要素として対応させます
3. **コールバック：**処理の一部を関数引数として受け取ります

関数ポインターの基本概念

関数ポインターの宣言

```
戻り値の型 (*ポインター名)(引数リスト);
```

基本的な例

```
/* 関数の定義 */
int add(int a, int b)
{
    return a + b;
}

int multiply(int a, int b)
{
    return a * b;
}

int main(void)
{
    /* 関数ポインターの宣言 */
    int (*operation)(int, int);

    /* 関数ポインターに関数を代入 */
    operation = add;
    printf("5 + 3 = %d\n", operation(5, 3));

    operation = multiply;
    printf("5 * 3 = %d\n", operation(5, 3));

    return 0;
}
```

関数ポインターの宣言形式

引数なし・戻り値なしの関数ポインター

```
void (*func_ptr)(void);

void hello(void)
{
    printf("Hello, World!\n");
}

func_ptr = hello;
func_ptr();  /* hello()と同じ */
```

複雑な引数を持つ関数ポインター

```
/* 文字列を引数に取る関数ポインター */
void (*string_func)(const char *);

void print_upper(const char *str)
{
    /* 文字列を大文字で出力 */
    /* 実装は省略 */
}

string_func = print_upper;
string_func("hello");
```

関数ポインターを返す関数

typedefで関数ポインター型に名前を付けると、戻り値の型を読みやすくできます。

```
typedef int (*operation_t)(int, int);

operation_t get_operation(int choice)
{
    if (choice == 1) {
        return add;
    } else {
        return multiply;
    }
}

/* 使用例 */
operation_t op = get_operation(1);
int result = op(10, 20);
```

関数ポインター配列

基本的な関数ポインター配列

```
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return a / b; }

/* 関数ポインター配列 */
int (*operations[4])(int, int) = {add, subtract, multiply, divide};

/* 使用例 */
int a = 10, b = 5;
int i;

for (i = 0; i < 4; i++) {
    printf("Result: %d\n", operations[i](a, b));
}
```

この例ではbが0でないことを前提とします。除算を含む表では、呼び出し前に除数を検査するか、関数側からエラーを返す設計が必要です。

関数ポインター配列を使った計算機

```
#include <stddef.h>

typedef struct {
    char symbol;
    int (*func)(int, int);
    const char *name;
} Calculator;

Calculator calc[] = {
    {'+', add, "add"},
    {'-', subtract, "subtract"},
    {'*', multiply, "multiply"},
    {'/', divide, "divide"}
};

/* 操作の検索と実行 */
int execute_operation(char op, int a, int b, int *result)
{
    size_t i;
    size_t size = sizeof(calc) / sizeof(calc[0]);

    if (result == NULL) {
        return 0;
    }

    for (i = 0; i < size; i++) {
        if (calc[i].symbol == op) {
            if (op == '/' && b == 0) {
                return 0;
            }
            *result = calc[i].func(a, b);
            return 1;
        }
    }
    return 0;
}
```

コールバック関数

コールバック関数の概念

コールバック関数は、別の関数へ関数ポインターとして渡し、その関数の処理中に呼び出してもらう関数です。

基本的なコールバック関数

```
/* コールバック関数の型定義 */
typedef void (*callback_t)(int);

/* コールバック関数の例 */
void print_result(int result)
{
    printf("Result: %d\n", result);
}

void log_result(int result)
{
    printf("Log result: %d\n", result);
}

/* コールバック関数を受け取る関数 */
void process_data(int data, callback_t callback)
{
    int result = data * 2;  /* 何らかの処理 */
    callback(result);       /* コールバック関数を呼び出し */
}

/* 使用例 */
process_data(10, print_result);  /* 結果: 20 */
process_data(15, log_result);    /* ログ: 計算結果は 30 です */
```

配列処理でのコールバック関数

```
/* 配列の各要素に対して処理を行う関数 */
void array_foreach(int *arr, int size, void (*callback)(int*, int))
{
    int i;
    for (i = 0; i < size; i++) {
        callback(&arr[i], i);
    }
}

/* コールバック関数の例 */
void double_value(int *value, int index)
{
    *value *= 2;
    printf("Element %d: %d\n", index, *value);
}

void square_value(int *value, int index)
{
    *value *= *value;
    printf("Element %d: %d\n", index, *value);
}

/* 使用例 */
int numbers[] = {1, 2, 3, 4, 5};
array_foreach(numbers, 5, double_value);   /* 各要素を2倍 */
array_foreach(numbers, 5, square_value);   /* 各要素を2乗 */
```

より高度な関数ポインター

関数ポインターの構造体

```
typedef struct {
    const char *name;
    int (*init)(void);
    void (*process)(int);
    void (*cleanup)(void);
} Module;

/* モジュールの実装例 */
int module_a_init(void) { printf("Module A init\n"); return 0; }
void module_a_process(int data) { printf("Module A data: %d\n", data); }
void module_a_cleanup(void) { printf("Module A cleanup\n"); }

Module modules[] = {
    {"ModuleA", module_a_init, module_a_process, module_a_cleanup},
    /* 他のモジュールも追加可能 */
};

/* モジュールの実行 */
void run_module(Module *mod, int data)
{
    if (mod->init() == 0) {
        mod->process(data);
        mod->cleanup();
    }
}
```

関数ポインターチェーン

```
typedef struct filter {
    int (*process)(int);
    struct filter *next;
} Filter;

/* フィルター関数の例 */
int double_filter(int x) { return x * 2; }
int add_ten_filter(int x) { return x + 10; }
int square_filter(int x) { return x * x; }

/* フィルターチェーンの実行 */
int apply_filters(int input, const Filter *chain)
{
    int result = input;
    const Filter *current = chain;

    while (current != NULL) {
        result = current->process(result);
        current = current->next;
    }

    return result;
}

/* フィルターチェーンの構築 */
Filter filter3 = {square_filter, NULL};
Filter filter2 = {add_ten_filter, &filter3};
Filter filter1 = {double_filter, &filter2};

/* 使用例: 5 -> 10 -> 20 -> 400 */
int result = apply_filters(5, &filter1);
```

qsort()関数との組み合わせ

標準ライブラリのqsort()

```
#include <stdlib.h>

/* 比較関数の例 */
int compare_int(const void *a, const void *b)
{
    int ia = *(const int*)a;
    int ib = *(const int*)b;
    return (ia > ib) - (ia < ib);
}

int compare_int_desc(const void *a, const void *b)
{
    return compare_int(b, a);  /* 降順 */
}

/* 使用例 */
int numbers[] = {5, 2, 8, 1, 9, 3};
int size = sizeof(numbers) / sizeof(numbers[0]);

qsort(numbers, size, sizeof(int), compare_int);      /* 昇順 */
qsort(numbers, size, sizeof(int), compare_int_desc); /* 降順 */
```

文字列の比較関数

```
int compare_string(const void *a, const void *b)
{
    const char *const *sa = a;
    const char *const *sb = b;
    return strcmp(*sa, *sb);
}

/* 文字列配列のソート */
const char *names[] = {"Alice", "Bob", "Charlie", "David"};
size_t name_count = sizeof(names) / sizeof(names[0]);

qsort(names, name_count, sizeof names[0], compare_string);
```

実用的な応用例

イベントシステム

```
#define MAX_EVENTS 10

typedef struct {
    int event_type;
    void (*handler)(int);
} EventHandler;

EventHandler event_handlers[MAX_EVENTS];
int handler_count = 0;

/* イベントハンドラの登録 */
void register_event_handler(int event_type, void (*handler)(int))
{
    if (handler_count < MAX_EVENTS) {
        event_handlers[handler_count].event_type = event_type;
        event_handlers[handler_count].handler = handler;
        handler_count++;
    }
}

/* イベントの発生 */
void trigger_event(int event_type, int data)
{
    int i;
    for (i = 0; i < handler_count; i++) {
        if (event_handlers[i].event_type == event_type) {
            event_handlers[i].handler(data);
        }
    }
}
```

プラグインシステム

```
typedef struct {
    const char *name;
    const char *version;
    int (*init)(void);
    void (*execute)(const char*);
    void (*shutdown)(void);
} Plugin;

/* プラグインの管理 */
Plugin *plugins[MAX_PLUGINS];
int plugin_count = 0;

int load_plugin(Plugin *plugin)
{
    if (plugin == NULL || plugin_count >= MAX_PLUGINS) {
        return -1;
    }
    if (plugin->init() == 0) {
        plugins[plugin_count++] = plugin;
        return 0;
    }
    return -1;
}

void execute_all_plugins(const char *command)
{
    int i;
    for (i = 0; i < plugin_count; i++) {
        plugins[i]->execute(command);
    }
}
```

よくある間違いとデバッグ

1. 関数ポインターの間違った宣言

問題: 括弧の位置が間違っている

```
int *func(int, int);     /* 関数（ポインターを返す） */
int (*func)(int, int);   /* 関数ポインター（正しい） */
```

2. NULLポインターの呼び出し

問題: 未初期化の関数ポインターを呼び出す

```
int (*func)(int, int);
int result = func(5, 3);  /* 未初期化の値を使うため未定義動作 */
```

対策:

```
int (*func)(int, int) = NULL;
if (func != NULL) {
    int result = func(5, 3);
}
```

3. 関数の型の不一致

問題: 関数ポインターの型と実際の関数の型が異なる

```
int add(int a, int b) { return a + b; }
void (*func)(int, int) = add;  /* 戻り値の型が違う */
```

4. 関数ポインター配列の初期化ミス

問題: 配列のサイズと関数の数が一致しない

```
int (*ops[3])(int, int) = {add, subtract, multiply, divide};  /* 4つの関数 */
```

性能を評価する場合

間接呼び出しを直接呼び出しと同じように最適化できるかは、コンパイラが呼び出し先を特定できるかによって変わります。性能上の問題が疑われる場合は、最適化を有効にした実際のプログラムを計測して判断します。

実例コード

完全な実装例は以下のファイルを参照してください。

基本的な関数ポインター

- function_pointer_basics.c - C90準拠版
- function_pointer_basics_c99.c - C99準拠版

関数ポインター配列と計算機

- calculator.c - C90準拠版
- calculator_c99.c - C99準拠版

コールバック関数とイベントシステム

- callback_system.c - C90準拠版
- callback_system_c99.c - C99準拠版

コンパイル方法

基本的なコンパイル(C90準拠)

```
gcc -std=c90 -Wall -Wextra -pedantic function_pointer_basics.c -o function_pointer_basics
```

Makefileを使用した場合

```
# 全てのプログラムをコンパイル

make all

# 特定のプログラムをコンパイル

make calculator

# C99版をコンパイル

make calculator_c99

# プログラムを実行

make run-all

# クリーンアップ

make clean
```

学習フローとコンパイル方法

推奨学習順序

1. **理論学習:** README.mdで基本概念を理解
2. **サンプルコード:** examples/の基本例を確認
3. **演習課題:** exercises/README.mdで課題を確認
4. **実装練習:** solutions/の解答例を参考に自分で実装

学習のコツ

1. **段階的理解:** 基本→配列→コールバック→高度な応用の順で学習
2. **実用例の実装:** 計算機、イベントシステムなどを実際に作成
3. **デバッグ技術:** 代入した関数名と、呼び出しに使う添字や条件を確認

4 **パターンの習得:** よくある設計パターンを覚える

実践的な応用分野

システムプログラミング

- デバイスドライバのハンドラ
- 割り込み処理関数
- システムコールハンドラ

GUIプログラミング

- イベントハンドラ
- ボタンのクリック処理
- メニュー項目の処理

ゲーム開発

- ステートマシン
- アニメーション制御
- 入力処理

組み込みシステム

- タイマー割り込み処理
- センサーデータ処理
- 通信プロトコル処理

参考資料

- examples/ - 実装例(C90、C99両対応)
- exercises/ - 演習問題
- solutions/ - 解答例
- C標準ライブラリリファレンス
- 関数ポインター設計パターン

演習問題

演習問題では、宣言の読み方、互換性のある関数型、コールバック、表駆動の分岐を確認できます。

- 基礎問題:基本的な文法や概念の確認
- 応用問題:より実践的なプログラムの作成
- チャレンジ問題:高度な理解と実装力が必要な問題

第15章：分割コンパイルと発展技術

対応C規格

- **主要対象：** C90/C99/C11/C17
- **学習内容：** 分割コンパイル、extern宣言、プリプロセッサ、モジュール設計、ライブラリ作成、高度な演算子テクニック

学習目標

この章を完了すると、以下のことができるようになります。

- 複数ファイルに分割したプログラムを作成できる
- extern宣言を正しく使用できる
- プリプロセッサ機能を効果的に活用できる
- 再利用可能なモジュールを設計できる
- 静的ライブラリを作成・使用できる
- 大規模プロジェクトの構成を理解する
- 短絡評価の評価順序を利用して、事前条件を検査できる
- 共用体（union）を理解し活用できる

概要と詳細

複数ファイルへ分割する目的

ソースファイルを機能ごとに分けると、公開する宣言と内部の実装を分離できます。変更時に再コンパイルする範囲も翻訳単位ごとに限定できます。

分割によって得られる性質

1. **大規模プログラムの開発**
   - 機能ごとに宣言と定義を配置できる
   - 担当範囲と変更範囲を分けられる
2. **コードの再利用**
   - 一度書いたコードを他のプロジェクトでも使える
   - ライブラリとして配布可能
3. **保守性の向上**
   - 機能ごとにファイルを分けて管理
   - バグの特定と修正が容易

分割コンパイルの基本

大きなプログラムを複数のソースファイルに分割することで、保守性と再利用性が向上します。

基本的な分割例

math_utils.h(ヘッダーファイル)

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b);
int multiply(int a, int b);
double circle_area(double radius);

#endif
```

math_utils.c(実装ファイル)

```
#include "math_utils.h"
#define PI 3.14159265359

int add(int a, int b)
{
    return a + b;
}

int multiply(int a, int b)
{
    return a * b;
}

double circle_area(double radius)
{
    return PI * radius * radius;
}
```

main.c(メインプログラム)

```
#include <stdio.h>
#include "math_utils.h"

int main(void)
{
    int sum = add(10, 20);
    int product = multiply(5, 6);
    double area = circle_area(3.0);

    printf("Sum: %d\n", sum);
    printf("Product: %d\n", product);
    printf("Circle area: %.2f\n", area);

    return 0;
}
```

extern宣言とグローバル変数

extern宣言の基本

```
/* config.h */
#ifndef CONFIG_H
#define CONFIG_H

extern int debug_level;
extern char app_name[];

#endif

/* config.c */
#include "config.h"

int debug_level = 1;
char app_name[] = "MyApplication";
```

外部リンケージを持つ変数の宣言

```
/* logger.h */
#ifndef LOGGER_H
#define LOGGER_H

typedef enum {
    LOG_ERROR,
    LOG_WARNING,
    LOG_INFO,
    LOG_DEBUG
} LogLevel;

extern LogLevel current_log_level;

void log_message(LogLevel level, const char *message);

#endif
```

プリプロセッサの活用

基本的なマクロ定義

```
#define MAX_SIZE 100
#define PI 3.14159265359
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

条件付きコンパイル

可変個引数マクロはC99で追加されました。次の形式は、DEBUG_PRINT("value=%d\n", value)のように書式文字列を含む引数を渡します。

```
#ifdef DEBUG
    #define DEBUG_PRINT(...) fprintf(stderr, __VA_ARGS__)
#else
    #define DEBUG_PRINT(...) ((void)0)
#endif

#if defined(_WIN32)
    #include <windows.h>
#elif defined(__linux__)
    #include <unistd.h>
#endif
```

_WIN32や__linux__は処理系が定義するマクロであり、C規格が定めるマクロではありません。利用するコンパイラの定義を確認します。

マクロと関数の選択

```
/* 引数を一度ずつ評価し、状態を戻り値で返す */
int safe_divide(int a, int b, int *result)
{
    if (b == 0 || result == NULL) {
        return 0;
    }

    *result = a / b;
    return 1;
}

/* 配列サイズ取得マクロ */
#define ARRAY_SIZE(arr) (sizeof(arr) / sizeof((arr)[0]))
```

関数形式なら、引数に副作用のある式を渡しても各引数を一度だけ評価します。ARRAY_SIZEは配列オブジェクトにだけ使用でき、ポインターへ適用すると要素数を求められません。

ヘッダーファイルの設計

インクルードガードの重要性

```
#ifndef MODULE_NAME_H
#define MODULE_NAME_H

/* ヘッダーファイルの内容 */

#endif /* MODULE_NAME_H */
```

共通ヘッダー

```
/* common.h - よく使われるヘッダーをまとめる */
#ifndef COMMON_H
#define COMMON_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stddef.h>

/* 共通の型定義 */
typedef unsigned char u8;
typedef unsigned short u16;
typedef unsigned int u32;

/* 共通のマクロ */
#define TRUE 1
#define FALSE 0

#endif
```

u8、u16、u32が正確に8、16、32ビットを表す保証はありません。C99以降で幅を固定する必要がある場合は、処理系が提供する`<stdint.h>`の`uint8_t`、`uint16_t`、`uint32_t`を使用します。

モジュール設計の原則

単一責任の原則

各モジュールは1つの明確な責任を持つべきです。

```
/* string_utils.h - 文字列処理のみ */
#ifndef STRING_UTILS_H
#define STRING_UTILS_H

char *trim_whitespace(char *str);
int string_compare_ignore_case(const char *s1, const char *s2);
char *string_duplicate(const char *src);

#endif

/* file_utils.h - ファイル操作のみ */
#ifndef FILE_UTILS_H
#define FILE_UTILS_H

int file_exists(const char *filename);
long file_size(const char *filename);
char *read_entire_file(const char *filename);

#endif
```

依存関係の最小化

```
/* good_module.h - 最小限の依存関係 */
#ifndef GOOD_MODULE_H
#define GOOD_MODULE_H

/* 標準ライブラリのみに依存 */
#include <stddef.h>

typedef struct point {
    double x;
    double y;
} Point;

Point point_create(double x, double y);
double point_distance(const Point *p1, const Point *p2);

#endif
```

静的ライブラリの作成

ライブラリの構築手順

```
# オブジェクトファイルの作成

gcc -c math_utils.c -o math_utils.o
gcc -c string_utils.c -o string_utils.o

# 静的ライブラリの作成

ar rcs libmyutils.a math_utils.o string_utils.o

# ライブラリの使用

gcc main.c -L. -lmyutils -o main
```

Makefileでのライブラリ管理

```
CC = gcc
CFLAGS = -Wall -Wextra -pedantic -std=c90
AR = ar
ARFLAGS = rcs

# ライブラリのソースファイル

LIB_SOURCES = math_utils.c string_utils.c file_utils.c
LIB_OBJECTS = $(LIB_SOURCES:.c=.o)
LIB_TARGET = libmyutils.a

# メインプログラム

MAIN_SOURCE = main.c
MAIN_TARGET = main

all: $(LIB_TARGET) $(MAIN_TARGET)

$(LIB_TARGET): $(LIB_OBJECTS)
    $(AR) $(ARFLAGS) $@ $^

$(MAIN_TARGET): $(MAIN_SOURCE) $(LIB_TARGET)
    $(CC) $(CFLAGS) $< -L. -lmyutils -o $@

clean:
    rm -f *.o $(LIB_TARGET) $(MAIN_TARGET)
```

大規模プロジェクトの構成

推奨ディレクトリ構造

```
project/
├── src/           # ソースファイル
│   ├── main.c
│   └── modules/
│       ├── math/
│       ├── string/
│       └── file/
├── include/       # ヘッダーファイル
│   └── myproject/
├── lib/           # ライブラリファイル
├── tests/         # テストファイル
├── docs/          # ドキュメント
├── Makefile
└── README.md
```

モジュール間のインターフェース設計

```
/* 良いインターフェース設計の例 */

/* logger.h */
typedef enum {
    LOG_LEVEL_ERROR,
    LOG_LEVEL_WARNING,
    LOG_LEVEL_INFO,
    LOG_LEVEL_DEBUG
} LogLevel;

/* 初期化と終了 */
int logger_init(const char *filename);
void logger_cleanup(void);

/* ログ出力 */
void log_error(const char *format, ...);
void log_warning(const char *format, ...);
void log_info(const char *format, ...);
void log_debug(const char *format, ...);

/* 設定 */
void logger_set_level(LogLevel level);
LogLevel logger_get_level(void);
```

C11で追加された機能

_Static_assert

```
#include <assert.h>

/* コンパイル時アサーション */
_Static_assert(sizeof(int) >= 4, "int must be at least 4 bytes");

typedef struct {
    char data[64];
} Buffer;

_Static_assert(sizeof(Buffer) == 64, "Buffer size must be exactly 64 bytes");
```

_Generic(型汎用選択)

```
#include <math.h>
#include <stdlib.h>

#define abs_generic(x) _Generic((x), \
    int: abs, \
    long: labs, \
    float: fabsf, \
    double: fabs, \
    long double: fabsl \
    )(x)

/* 使用例 */
int i = -5;
double d = -3.14;
printf("%d\n", abs_generic(i));    /* abs(i) */
printf("%.2f\n", abs_generic(d));  /* fabs(d) */
```

実践的なモジュール例

エラーハンドリングモジュール

```
/* error.h */
#ifndef ERROR_H
#define ERROR_H

typedef enum {
    ERROR_SUCCESS = 0,
    ERROR_NULL_POINTER,
    ERROR_OUT_OF_MEMORY,
    ERROR_FILE_NOT_FOUND,
    ERROR_INVALID_ARGUMENT
} ErrorCode;

const char *error_get_message(ErrorCode code);
void error_set_last_error(ErrorCode code);
ErrorCode error_get_last_error(void);

#endif
```

設定管理モジュール

```
/* config.h */
#ifndef CONFIG_H
#define CONFIG_H

typedef struct {
    char *key;
    char *value;
} ConfigItem;

int config_load(const char *filename);
const char *config_get_string(const char *key, const char *default_value);
int config_get_int(const char *key, int default_value);
double config_get_double(const char *key, double default_value);
void config_cleanup(void);

#endif
```

デバッグとテスト支援

デバッグマクロの活用

次の__func__と可変個引数マクロはC99以降で使用できます。

```
/* debug.h */
#ifndef DEBUG_H
#define DEBUG_H

#ifdef DEBUG_MODE
    #define DBG_ENTER() printf("ENTER: %s\n", __func__)
    #define DBG_EXIT() printf("EXIT: %s\n", __func__)
    #define DBG_PRINT(...) fprintf(stderr, __VA_ARGS__)
#else
    #define DBG_ENTER() ((void)0)
```

```
    #define DBG_EXIT() ((void)0)
    #define DBG_PRINT(...) ((void)0)
#endif

#endif
```

単体テストフレームワーク

```
/* test_framework.h */
#ifndef TEST_FRAMEWORK_H
#define TEST_FRAMEWORK_H

#include <stdio.h>

extern int test_count;
extern int test_passed;

#define TEST_ASSERT(condition) \
    do { \
        test_count++; \
        if (condition) { \
            test_passed++; \
            printf("PASS: %s\n", #condition); \
        } else { \
            printf("FAIL: %s (line %d)\n", #condition, __LINE__); \
        } \
    } while (0)

#define TEST_SUMMARY() \
    printf("\nTest Results: %d/%d passed\n", test_passed, test_count)

#endif
```

パフォーマンス最適化

インライン関数（C99以降）

```
/* C99のinline関数 */
static inline int max_inline(int a, int b)
{
    return (a > b) ? a : b;
}

/* ヘッダーファイルでは内部リンケージにすると定義を完結させやすい */
static inline double square_inline(double x)
{
    return x * x;
}
```

コンパイラ最適化の活用

```
/* 最適化ヒントの提供 */
#ifdef __GNUC__
    #define LIKELY(x)   __builtin_expect(!!(x), 1)
    #define UNLIKELY(x) __builtin_expect(!!(x), 0)
#else
    #define LIKELY(x)   (x)
    #define UNLIKELY(x) (x)
#endif

/* 使用例 */
if (LIKELY(ptr != NULL)) {
    /* 通常の処理 */
} else {
    /* エラー処理（稀） */
}
```

実例コード

完全な実装例は以下のファイルを参照してください。

基本的な分割コンパイル

- multi_file_basic/ - 基本的な分割例

- library_example/ - ライブラリ作成例

高度なプリプロセッサ活用

- preprocessor_demo.c - マクロとプリプロセッサ
- conditional_compile.c - 条件付きコンパイル

実用的なモジュール

- utils_library/ - 実用的なユーティリティライブラリ
- project_template/ - プロジェクトテンプレート

コンパイル方法

基本的な分割コンパイル

```
# 個別にコンパイル

gcc -c math_utils.c -o math_utils.o
gcc -c main.c -o main.o
gcc math_utils.o main.o -o main

# 一括コンパイル

gcc math_utils.c main.c -o main
```

静的ライブラリの作成と使用

```
# ライブラリ作成

gcc -c *.c
ar rcs libmyutils.a *.o

# ライブラリ使用

gcc main.c -L. -lmyutils -o main
```

Makefileを使用した場合

```
# 全てのターゲットをビルド

make all

# 特定のターゲットをビルド

make library

# クリーンアップ

make clean

# インストール

make install
```

学習フローとコンパイル方法

推奨学習順序

1. **理論学習**: README.mdで基本概念を理解
2. **サンプルコード**: examples/の基本例を確認
3. **演習課題**: exercises/README.mdで課題を確認
4. **実装練習**: solutions/の解答例を参考に自分で実装

実践的な学習方法

1. **小さなプロジェクトから始める:** 2-3ファイルの分割から
2. **段階的に複雑化:** 機能追加ごとにモジュール分割
3. **実際のライブラリを参考:** オープンソースプロジェクトの構成を学習
4. **チーム開発の模擬:** 複数人での分担開発を体験

C標準の違いと対応

C90での制限と対策

- inline関数なし → マクロまたは最適化に頼る
- 可変長引数マクロなし → 固定引数マクロを使用
- _Bool型なし → int型でbooleanを表現

C99以降の拡張機能活用

- inline関数の使用
- 可変長引数マクロ
- 指定初期化子
- 複合リテラル

C11の機能とC17

- _Static_assert
- _Generic
- _Alignas/_Alignof
- _Thread_local

C17はC11に対する不具合修正が中心で、新しい言語機能は追加していません。

短絡評価と共用体

論理AND演算子&&は左オペランドが0なら右オペランドを評価しません。論理OR演算子||は左オペランドが0以外なら右オペランドを評価しません。この評価順序を使うと、ポインターや添字の事前条件を検査してから対象の式を評価できます。

短絡評価による安全なプログラミング

短絡評価を使う場合は、左から右へ事前条件が成立する順に式を並べます。

配列とポインターの安全な操作

C言語では配列の境界チェックが自動的に行われないため、プログラマーが明示的にチェックする必要があります。

```
/* 配列の境界チェック */
int array[10];
int index = 15;
if (index >= 0 && index < 10 && array[index] > 0) {
    /* index が範囲外なので、array[index] へのアクセスは行われない */
    printf("Valid value: %d\n", array[index]);
}

/* 多次元配列の安全なアクセス */
int matrix[5][5];
int row = 3, col = 7;
if (row >= 0 && row < 5 && col >= 0 && col < 5 && matrix[row][col] != 0) {
    /* 行と列の両方が範囲内の場合のみアクセス */
    process_element(matrix[row][col]);
}
```

```
/* 動的配列（ポインター）の操作 */
int *data = NULL;
if (size > 0 && (size_t)size <= ((size_t)-1) / sizeof *data) {
    data = malloc((size_t)size * sizeof *data);
}
if (data != NULL && initialize_array(data, size)) {
    /* メモリ確保成功、かつ初期化成功の場合のみ使用 */
    use_array(data, size);
}
free(data);  /* free()はNULLに対して安全 */
```

文字列処理の安全性確保

```
/* 文字列の安全なチェック */
char *str = get_string();  /* NULL を返す可能性がある */
if (str && strlen(str) > 0 && str[0] == 'A') {
    /* str が NULL の場合、strlen や str[0] は評価されない */
    printf("String starts with 'A'\n");
}

/* 文字列の詳細な検証 */
char *input = get_user_input();
if (input && *input && strlen(input) < MAX_LENGTH && is_valid_format(input)) {
    /* NULL でない、空でない、長さ制限内、フォーマット正しい */
    process_input(input);
}
```

ファイル操作とリソース管理

```
/* ファイル操作の連鎖的エラーチェック */
FILE *fp = fopen("data.txt", "r");
if (fp != NULL && read_header(fp) && validate_data(fp)) {
    /* 各段階でエラーがあれば、後続の処理はスキップされる */
    process_file(fp);
}
if (fp != NULL) {
    fclose(fp);
}

/* 複数リソースの管理 */
void process_data_file(const char *filename)
{
    FILE *fp = NULL;
    char *buffer = NULL;
    int *data = NULL;

    /* リソースの段階的確保 */
    if ((fp = fopen(filename, "rb")) != NULL &&
        (buffer = malloc(BUFFER_SIZE)) != NULL &&
        (data = malloc(sizeof *data * MAX_ITEMS)) != NULL &&
        read_file_to_buffer(fp, buffer, BUFFER_SIZE) &&
        parse_buffer_to_data(buffer, data, MAX_ITEMS)) {

        /* すべてのリソースが正常に確保され、処理が成功 */
        analyze_data(data, MAX_ITEMS);

    } else {
        /* どこかでエラーが発生した */
        fprintf(stderr, "Data processing failed\n");
    }

    /* クリーンアップ（NULL チェック不要） */
    free(data);
    free(buffer);
    if (fp != NULL) fclose(fp);
}
```

ビット操作とビットフィールド

ビット演算の詳細な解説、ビットマスクを使った高度なテクニック、ビットフィールドによるメモリ最適化については、専用の章を設けています。

第12章：ビット操作とビットフィールドでは、次の内容を説明しています。

- ビット演算子の詳細と活用法

- フラグ管理システムの実装
- ビットカウントなどの高度なアルゴリズム
- ビットフィールドによるデータ圧縮
- RGB色操作などの実践的な応用例

共用体(union)

共用体は、複数のメンバが同じ記憶域を共有する型です。同時には一つの表現だけを保持するデータを、タグと組み合わせて表す場合に使います。

基本的な共用体の使い方

```
/* 基本的な共用体の定義 */
union Data {
    int i;
    float f;
    char str[20];
};

int main(void) {
    union Data data;

    /* 整数として使用 */
    data.i = 42;
    printf("Integer: %d\n", data.i);

    /* 浮動小数点として使用（前の値は上書きされる） */
    data.f = 3.14f;
    printf("Floating: %f\n", data.f);

    /* 文字列として使用 */
    strcpy(data.str, "Hello");
    printf("String: %s\n", data.str);

    printf("Union size: %lu bytes\n", (unsigned long)sizeof(union Data));
    return 0;
}
```

オブジェクト表現の取得

```
/* floatのオブジェクト表現を1バイトずつ表示する */
#include <stddef.h>
#include <stdio.h>
#include <string.h>

void print_float_representation(float value)
{
    unsigned char bytes[sizeof value];
    size_t i;

    memcpy(bytes, &value, sizeof value);
    for (i = 0; i < sizeof bytes; i++) {
        printf("%02X ", (unsigned int)bytes[i]);
    }
    putchar('\n');
}
```

`unsigned char`の配列へ`memcpy`すると、別の共用体メンバを読む方法に依存せずオブジェクト表現を取得できます。ただし、`float`の形式やバイト順序は処理系に依存するため、表示結果をIEEE 754の各フィールドとみなせるとは限りません。

32ビット値からオクテットを取り出す

```
#include <stdint.h>

void print_ipv4_value(uint32_t ip)
{
    printf("%u.%u.%u.%u\n",
           (unsigned int)((ip >> 24) & UINT32_C(0xff)),
           (unsigned int)((ip >> 16) & UINT32_C(0xff)),
           (unsigned int)((ip >> 8) & UINT32_C(0xff)),
           (unsigned int)(ip & UINT32_C(0xff)));
}
```

整数と文字配列を共用体で重ねる方法は、処理系のバイト順序と`unsigned int`の幅に依存します。外部形式の順序が決まっている場合は、幅が明確な整数型とシフト演算で各部分を取り出します。

タグ付き共用体(判別共用体)

```
/* データ型を識別するタグ付き共用体 */
typedef enum {
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_STRING
} DataType;

typedef struct {
    DataType type;
    union {
        int i;
        float f;
        char *s;
    } value;
} Variant;

void print_variant(const Variant *var) {
    switch (var->type) {
        case TYPE_INT:
            printf("Integer: %d\n", var->value.i);
            break;
        case TYPE_FLOAT:
            printf("Floating: %f\n", var->value.f);
            break;
        case TYPE_STRING:
            printf("String: %s\n", var->value.s);
            break;
    }
}
```

共用体の注意点

1. **現在の表現を管理する**
   - 最後に値を格納したメンバをタグで記録する
   - 別のメンバを読む処理は型や処理系に依存するため、移植可能な型変換には使わない
2. **メモリサイズ**
   - 最大のメンバを格納できる大きさ以上になる
   - パディングの影響を受ける
3. **初期化**
   - C90では最初のメンバーでのみ初期化可能
   - C99では指定初期化子が使用可能

条件の評価順序を使った処理

```
/* 条件付き処理の最適化 */
typedef struct {
    int is_cached;
    int cache_value;
} Cache;

int get_expensive_value(Cache *cache, int param)
{
    int result;

    /* キャッシュ済みなら、値が0でもその値を返す */
    if (cache != NULL && cache->is_cached) {
        return cache->cache_value;
    }

    /* 高コストな計算 */
    result = expensive_calculation(param);

    /* キャッシュに保存 */
    if (cache != NULL) {
        cache->is_cached = 1;
        cache->cache_value = result;
    }

    return result;
}

/* 権限チェックの最適化 */
int can_access_resource(User *user, Resource *resource)
{
    /* 管理者の場合は、右辺の詳細な権限検査を評価しない */
    return (user != NULL && user->is_admin) ||
           /* 一般ユーザーは詳細な権限チェック */
           (user != NULL &&
            resource != NULL &&
            user->level >= resource->required_level &&
            has_permission(user, resource->type) &&
            !is_blocked(user, resource));
}
```

参考資料

- examples/ - 実装例(複数C標準対応)
- exercises/ - 演習問題
- solutions/ - 解答例
- C標準ライブラリリファレンス
- GNU Make Manual

演習問題

演習問題では、分割コンパイル、リンケージ、マクロ、短絡評価、共用体を確認できます。

- 基礎問題:基本的な文法や概念の確認
- 応用問題:より実践的なプログラムの作成
- チャレンジ問題:高度な理解と実装力が必要な問題

第16章：C23の機能

対応C規格

- **主要対象：** C23
- **学習内容：** bool、typeof、nullptr、2進整数定数、桁区切り、型推論、constexpr

学習目標

この章を完了すると、次のことができるようになります。

- C23の規格バージョンを機能テストマクロで判定できる
- bool、true、falseをキーワードとして使用できる
- typeofとtypeof_unqualの違いを説明できる
- nullptrと整数定数0の違いを説明できる
- 2進整数定数と桁区切りを正しい構文で記述できる
- 利用する処理系が各機能を実装しているか確認できる

C23の位置づけ

C23はC言語規格の通称です。国際規格はISO/IEC 9899:2024として2024年に発行されました。規格に準拠する処理系では、__STDC_VERSION__が202311Lになります。

```
#if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 202311L
/* C23 mode */
#else
#error "C23 or later is required"
#endif
```

規格モードを指定しても、処理系がすべての機能を実装済みとは限りません。コンパイル時の診断と処理系のリリースノートを併せて確認します。

bool、true、false

C99では_Boolが言語の型として追加され、<stdbool.h>がbool、true、falseをマクロとして提供しました。C23ではbool、true、falseが言語機能になったため、これらを使うためだけに<stdbool.h>をインクルードする必要はありません。

```
bool is_ready = false;

if (!is_ready) {
    is_ready = true;
}
```

boolへ変換した値は0ならfalse、0以外ならtrueになります。boolは状態を表す型ですが、入力値の妥当性や処理の成功を自動的に保証するものではありません。

以前の規格との比較

```
/* C90 */
int is_ready = 0;

/* C99以降 */
#include <stdbool.h>
bool is_ready = false;

/* C23 */
bool is_ready = false;
```

typeofとtypeof_unqual

typeofは、式または型名から型を得る演算子です。GNU Cなどが拡張として提供してきた綴りに相当する機能が、C23で標準化されました。

```
int count = 10;
typeof(count) next_count = count + 1;

double price = 19.99;
typeof(price) tax = 1.10;
```

typeofは、対象の型に含まれる修飾子や原子型の性質を保持します。typeof_unqualは、結果の型からconst、volatile、restrict、_Atomicに相当する修飾を除きます。

```
const int limit = 100;
typeof(limit) same_limit_type = 50;          /* const int */
typeof_unqual(limit) mutable_limit = 50;     /* int */
```

配列を対象にしたtypeofは、通常の式で起こる先頭要素へのポインター変換を行わず、配列型を得ます。

```
int values[4] = {1, 2, 3, 4};
typeof(values) copy = {5, 6, 7, 8};  /* int[4] */
```

nullptr

nullptrは、C23で追加された定義済み定数です。型はnullptr_tであり、オブジェクトへのポインター型、関数ポインター型、boolへ変換できます。

```
#include <stddef.h>

int *data = nullptr;
nullptr_t no_address = nullptr;

if (data == nullptr) {
    /* data does not point to an object */
}
```

従来のnullポインター定数である整数定数0やNULLも引き続き使用できます。nullptrは整数型の値としては扱えないため、ポインターを意図した値であることを型で区別できます。

```
int *pointer_value = nullptr;  /* valid */
bool flag = nullptr;           /* false */
/* int integer_value = nullptr; */  /* constraint violation */
```

Cには関数オーバーロードがないため、C++のようなオーバーロード解決をnullptrの利点として説明することはできません。

2進整数定数

C23では、0bまたは0Bで始まる2進整数定数を記述できます。ビット位置がそのまま表記に現れるため、フラグやマスクの確認に使えます。

```
#define PERMISSION_READ    0b100U
#define PERMISSION_WRITE   0b010U
#define PERMISSION_EXECUTE 0b001U

unsigned int permissions = PERMISSION_READ | PERMISSION_WRITE;
```

2進表記を使っても、整数型の幅や変換規則は変わりません。値が格納先の型で表現できるかを確認します。

数値リテラルの桁区切り

C23では、整数定数と浮動小数点定数の数字の間へ単一引用符'を置けます。区切り文字は値の計算では無視されます。

```
unsigned int byte_mask = 0b1111'1111U;
unsigned long population = 1'000'000UL;
unsigned long color = 0xFF'80'20UL;
```

アンダースコアはC23の桁区切りではありません。接頭辞の直後や数字列の末尾にも単一引用符は置けません。

autoによる型推論

C23では、初期化子からオブジェクトの型を推論する宣言にautoを使えます。

```
auto count = 42;       /* int */
auto ratio = 3.14;     /* double */
auto pointer = &count; /* int * */
```

初期化子は必須です。型を明示した方がインターフェースや変換の意図を伝えやすい場合は、従来どおり型名を書きます。

constexprオブジェクト

constexprを指定したオブジェクトは、翻訳時に評価できる定数として定義します。

```
constexpr int element_count = 100;
int values[element_count];
```

初期化子は定数式の要件を満たす必要があります。単に実行中の変更を防ぐだけならconst、整数定数式など翻訳時の定数として使う必要があるならconstexprという違いがあります。

条件付きインクルージョンの追加構文

#elifdefと#elifndefは、#elif defined(...)を短く書く構文です。

```
#if defined(DEBUG)
#define LOG_LEVEL 2
#elifdef RELEASE
#define LOG_LEVEL 0
#elifndef LOG_LEVEL
#define LOG_LEVEL 1
#endif
```

コンパイルと機能確認

処理系が受け付ける規格モードを確認し、C23モードでコンパイルします。新しい処理系では-std=c23、移行期の処理系では-std=c2xというオプションを使う場合があります。

```
gcc -std=c23 -Wall -Wextra -pedantic source.c -o output
```

規格モードの名称と各機能の実装状況は、コンパイラとバージョンによって異なります。特定のバージョン番号だけで一括判定せず、必要な機能を含む小さなコードを警告付きでコンパイルして確認します。

実例コード

実装例は次のファイルにあります。

- bool_basics.c：boolの基本
- typeof_demo.c：typeofの使用例
- nullptr_example.c：nullptrの使用例
- binary_literals.c：2進整数定数の使用例

以前の規格から移行するときの確認事項

1. **処理系の対応**：使用するすべてのコンパイラが、採用するC23機能を実装しているか確認します
2. **規格モード**：ビルドシステム、静的解析、エディタの設定を同じ規格モードにそろえます
3. **公開ヘッダー**：利用側が古い規格を使う場合、公開APIにC23専用の構文を出さない設計も検討します

4 **機械的な置換を避ける**：NULLをすべてnullptrへ置換する必要はなく、対応する処理系とコード規約に合わせます

参考資料

- WG14公式サイト
- C23最終公開ドラフトN3096
- WG14のプロジェクト状況

演習問題

演習問題では、__STDC_VERSION__、bool、typeof、nullptr、2進整数定数、桁区切りを確認できます。

- 基礎問題：構文と型の確認
- 応用問題：以前の規格との条件付きコンパイル
- チャレンジ問題：複数のC23機能を組み合わせた実装

補章1：C言語規格の歴史

対応C規格

- **主要対象：** C90、C99、C11、C17、C23
- **学習内容：** 各規格の位置づけ、追加機能、規格モードの選び方

学習目標

この章を完了すると、次のことができるようになります。

- C89とC90の関係を説明できる
- C99、C11、C17、C23で追加または変更された主な機能を区別できる
- `__STDC_VERSION__`で言語規格の版を確認できる
- 対象処理系と依存ライブラリに合わせて規格モードを選べる

標準化までの経緯

Cは1970年代初頭にベル研究所で開発されました。処理系ごとの差異を抑え、同じソースコードを異なる環境へ移植しやすくするため、1980年代に標準化が進められました。

- **1978年**：Brian KernighanとDennis Ritchieによる初版の書籍が出版されました
- **1989年**：ANSI X3.159-1989が発行されました。通称はC89です
- **1990年**：ISO/IEC 9899:1990が発行されました。通称はC90です

C89とC90は技術的な内容がほぼ同じで、標準化団体と発行年が異なります。

規格の一覧

通称	国際規格	`__STDC_VERSION__`	主な変更
C90	ISO/IEC 9899:1990	定義なし	初の国際規格
C95	ISO/IEC 9899:1990/Amd 1:1995	`199409L`	ワイド文字関連などの追補
C99	ISO/IEC 9899:1999	`199901L`	複合リテラル、指定初期化子、`inline`、`_Bool`など
C11	ISO/IEC 9899:2011	`201112L`	`_Generic`、原子型、スレッド、静的アサーションなど
C17	ISO/IEC 9899:2018	`201710L`	C11に対する不具合修正と明確化
C23	ISO/IEC 9899:2024	`202311L`	`bool`、`typeof`、`nullptr`、2進整数定数など

C23という通称の数字と、国際規格の発行年は一致しません。C23は2024年にISO/IEC 9899:2024として発行されました。

C90

C90は、関数プロトタイプ、`void`、`const`、`volatile`や標準ライブラリを含む、国際標準としての基礎を定めました。現在も古い処理系や長期保守される環境との互換性要件で使われることがあります。

C90では、ブロック内の宣言をそのブロックの文より前に置きます。宣言場所は関数の先頭だけに限定されず、内側の複合文を作ればその先頭でも宣言できます。

```
int sum_values(void)
{
    int i;
    int sum = 0;

    for (i = 0; i < 10; i++) {
        sum += i;
    }

    return sum;
}
```

C99

C99では、宣言と文の混在、for初期化部での宣言、//コメント、long long、_Bool、inline、複合リテラル、指定初期化子などが追加されました。<stdint.h>の規定幅整数型もC99で追加されています。

```
#include <stdbool.h>
#include <stdio.h>

int main(void)
{
    bool enabled = true;
    int values[6] = {[0] = 10, [5] = 20};

    for (int i = 0; i < 6; i++) {
        printf("%d\n", values[i]);
    }

    return enabled ? 0 : 1;
}
```

可変長配列もC99で追加されました。C11以降では自動記憶域期間を持つ可変長配列を提供しない処理系があるため、利用時は規格版だけでなく処理系の対応を確認します。

C11

C11では、型に応じた式を選択する_Generic、_Static_assert、アラインメント指定、原子型、スレッドライブラリ、匿名構造体・共用体などが追加されました。

```
#include <limits.h>

_Static_assert(sizeof(int) * CHAR_BIT >= 16,
               "int must occupy at least 16 bits");

#define TYPE_NAME(x) _Generic((x), \
    int: "int", \
    float: "float", \
    double: "double", \
    default: "other")
```

スレッドや原子操作など、一部の機能には処理系が対応しないことを示す機能テストマクロがあります。たとえば、__STDC_NO_THREADS__が定義されている処理系は<threads.h>を提供しません。

C17

C17はC11に対する不具合修正と明確化をまとめた規格です。新しい言語機能は追加していません。C11の機能を使うコードでも、利用可能ならC17モードを選ぶことで修正版の規格を基準にできます。

C23

C23では、bool、true、false、static_assertなどがキーワードになりました。ほかにtypeof、typeof_unqual、nullptr、_BitInt、2進整数定数、桁区切り、#embed、autoによる型推論、constexprオブジェクトなどが追加されています。

```
bool ready = true;
auto count = 42;
typeof(count) next_count = count + 1;
int *pointer = nullptr;
```

```
unsigned int mask = 0b1111'0000U;
```

C23の詳細はC23の新機能で扱います。

規格版をプログラムから確認する

C95以降の規格に準拠する処理系は、`__STDC_VERSION__`を定義します。C90モードではこのマクロが定義されないため、先に`defined`で確認します。

```
#if !defined(__STDC_VERSION__)
/* C90 or a non-conforming mode */
#elif __STDC_VERSION__ >= 202311L
/* C23 or later */
#elif __STDC_VERSION__ >= 201710L
/* C17 */
#elif __STDC_VERSION__ >= 201112L
/* C11 */
#elif __STDC_VERSION__ >= 199901L
/* C99 */
#endif
```

このマクロは規格版の判定には使えますが、任意機能や処理系拡張の有無をすべて示すものではありません。必要なヘッダー、機能テストマクロ、コンパイル結果も確認します。

規格モードを選ぶ基準

規格モードは、学習順序の新旧ではなく、プロジェクトの制約から選びます。

1. **対象処理系**：対応するすべてのコンパイラが受け付ける規格を確認します
2. **依存関係**：利用するライブラリやSDKが要求する規格を確認します
3. **公開インターフェース**：ヘッダーを利用する側が受け付けられる構文を選びます
4. **必要な機能**：`_Generic`や`nullptr`など、実際に必要な機能を特定します
5. **検証方法**：CIと警告オプションで、選んだ規格への準拠を継続して確認します

古い規格ほど移植性が高い、新しい規格ほど安全である、と一律には判断できません。実装の品質、利用する機能、対象環境の組み合わせで決まります。

コンパイラで規格モードを指定する

GCCやClang系の処理系では、次のようなオプションを使用します。

```
gcc -std=c90 -Wall -Wextra -pedantic program.c
gcc -std=c99 -Wall -Wextra -pedantic program.c
gcc -std=c11 -Wall -Wextra -pedantic program.c
gcc -std=c17 -Wall -Wextra -pedantic program.c
gcc -std=c23 -Wall -Wextra -pedantic program.c
```

C23への移行期の処理系では、`-std=c2x`という名称を使う場合があります。利用できるオプションは、対象コンパイラのヘルプで確認します。

参考資料

- WG14公式サイト
- WG14のプロジェクト状況
- C23最終公開ドラフトN3096

演習問題

演習問題では、規格ごとの構文差、`__STDC_VERSION__`、コンパイルオプションを確認できます。

補章2：開発環境

対応C規格

- **主要対象：** 全規格
- **学習内容：** 必要なツール、翻訳工程、警告、規格モード、デバッグ

学習目標

この章を完了すると、次のことができるようになります。

- エディタ、コンパイラ、リンカ、デバッガの役割を説明できる
- プリプロセス、コンパイル、アセンブル、リンクの各段階を確認できる
- 規格モードと警告を指定してプログラムをビルドできる
- コンパイルエラーとリンクエラーを区別できる
- デバッグ情報を付けてデバッガを起動できる

最小構成

Cプログラムの作成に必要なのは、ソースファイルを編集するエディタと、ソースコードを実行ファイルへ変換する処理系です。複数ファイルの管理やデバッグが必要になったら、ビルドツールとデバッガを追加します。

1. **エディタ：**プレーンテキストのソースファイルを編集します
2. **C処理系：**コンパイラ、アセンブラ、リンカ、標準ライブラリを提供します
3. **デバッガ：**停止位置の設定、変数の確認、1ステップ実行を行います
4. **ビルドツール：**複数の翻訳単位と依存関係をまとめてビルドします

特定のエディタやビルドツールは必須ではありません。プロジェクトですでに使われている処理系とビルド手順がある場合は、その構成を優先します。

処理系を確認する

GCC系またはClang系の処理系では、次のコマンドで実行ファイルとバージョンを確認できます。

```
cc --version
cc -v
```

`cc`が利用できない場合は、環境が提供する`gcc`、`clang`などのコマンド名を確認します。インストール方法やパッケージ名はOSと配布元によって変わるため、利用する環境の管理手順に従います。

翻訳工程

Cのソースコードは、概念上、前処理、翻訳、アセンブル、リンクを経て実行ファイルになります。処理系は複数の段階を一つのコマンドで実行できますが、中間結果を出力すると問題の発生箇所を確認できます。

前処理

```
cc -E source.c -o source.i
```

前処理では、`#include`、マクロ置換、条件付きインクルージョンなどを処理します。生成された`source.i`を調べると、展開後の宣言やマクロを確認できます。

アセンブリコードの生成

```
cc -S source.c -o source.s
```

-Sはリンクを行わず、処理系が生成したアセンブリコードを出力します。最適化による変化を調べる場合は、最適化オプションを明示します。

オブジェクトファイルの生成

```
cc -c source.c -o source.o
```

-cはリンクを行わず、翻訳単位ごとのオブジェクトファイルを生成します。

リンク

```
cc main.o utility.o -o program
```

リンクでは、複数のオブジェクトファイルと必要なライブラリから実行ファイルを作ります。宣言だけがあり定義が見つからない場合や、同じ外部定義が複数ある場合は、この段階で診断されます。

警告と規格モード

GCCやClangでは、次のように規格モードと警告を指定できます。

```
cc -std=c17 -Wall -Wextra -Wpedantic source.c -o program
```

- -std=c17：C17を基準に翻訳します
- -Wall：よく使われる警告群を有効にします。「すべての警告」という意味ではありません
- -Wextra：追加の警告群を有効にします
- -Wpedantic：選択した規格から外れる構文や機能の診断を増やします

警告を無効にする前に、警告が示すコード上の問題を確認します。-Werrorは警告をエラーとして扱うため、継続的インテグレーションでは有用ですが、処理系の更新で新しい警告が増える影響も考慮します。

規格モードの例

```
cc -std=c90 -Wall -Wextra -Wpedantic source.c -o program
cc -std=c99 -Wall -Wextra -Wpedantic source.c -o program
cc -std=c11 -Wall -Wextra -Wpedantic source.c -o program
cc -std=c17 -Wall -Wextra -Wpedantic source.c -o program
cc -std=c23 -Wall -Wextra -Wpedantic source.c -o program
```

受け付けるオプション名は処理系によって異なります。C23への移行期の処理系では、-std=c2xを使う場合があります。

最適化とデバッグ情報

開発中は、デバッグ情報と、デバッグを妨げにくい最適化を指定できます。

```
cc -std=c17 -Wall -Wextra -Wpedantic -g -Og source.c -o program
```

- -g：デバッガが利用する情報を出力します
- -O0：最適化を抑えます
- -Og：デバッグ可能性を考慮した最適化を行います
- -O2：一般的な最適化を有効にします
- -O3：より積極的な最適化を試みます
- -Os：コードサイズを抑える最適化を試みます

最適化レベルが高いと、変数が除去されたり、ソースコードの行と実行順序が対応しにくくなったりします。性能を比較する場合は、同じ入力とビルド条件で計測します。

実装の性質を確認する

型のサイズや範囲は処理系によって異なります。次のプログラムは、sizeofの結果と整数型の範囲を表示します。

```
#include <limits.h>
#include <stdio.h>

int main(void)
{
    printf("CHAR_BIT: %d\n", CHAR_BIT);
    printf("sizeof(char): %lu\n", (unsigned long)sizeof(char));
    printf("sizeof(short): %lu\n", (unsigned long)sizeof(short));
    printf("sizeof(int): %lu\n", (unsigned long)sizeof(int));
    printf("sizeof(long): %lu\n", (unsigned long)sizeof(long));
    printf("sizeof(void *): %lu\n", (unsigned long)sizeof(void *));
    printf("INT_MIN: %d\n", INT_MIN);
    printf("INT_MAX: %d\n", INT_MAX);
    printf("LONG_MIN: %ld\n", LONG_MIN);
    printf("LONG_MAX: %ld\n", LONG_MAX);
    return 0;
}
```

sizeof(char)は常に1ですが、1バイトが8ビットとは限りません。1バイトのビット数はCHAR_BITで確認します。

規格版と処理系を確認する

処理系固有のマクロは、対応する処理系でだけ使用します。ClangはGCC互換マクロも定義するため、Clangの判定を先に置きます。

```
#include <stdio.h>

int main(void)
{
#if defined(__clang__)
    printf("Clang: %d.%d.%d\n",
           __clang_major__, __clang_minor__, __clang_patchlevel__);
#elif defined(__GNUC__)
    printf("GCC: %d.%d.%d\n",
           __GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__);
#elif defined(_MSC_VER)
    printf("MSVC: %d\n", _MSC_VER);
#else
    printf("Compiler: unknown\n");
#endif

#if defined(__STDC_VERSION__)
    printf("__STDC_VERSION__: %ld\n", __STDC_VERSION__);
#else
    printf("__STDC_VERSION__: not defined\n");
#endif

    return 0;
}
```

処理系固有マクロは、言語規格ではなく処理系の機能を判定するために使います。

エラーを段階で分類する

コンパイル時の診断

構文、型、宣言に問題がある場合は、オブジェクトファイルを生成する前に診断されます。最初の診断が後続の診断を引き起こすことがあるため、先頭のエラーから確認します。

リンク時の診断

undefined referenceなどの診断は、必要な定義を含むオブジェクトファイルやライブラリがリンク対象にない場合に発生します。関数の宣言を追加するだけでは解決せず、その関数の定義をリンクする必要があります。

実行時の問題

境界外アクセス、解放済み領域の使用、未初期化値の使用などは、翻訳に成功しても実行時に問題になります。デバッガや、処理系が提供するサニタイザを利用して原因を絞り込みます。

```
cc -std=c17 -Wall -Wextra -Wpedantic -g \
  -fsanitize=address,undefined source.c -o program
```

サニタイザはC規格の機能ではなく、処理系の拡張です。利用可能な種類と対応環境は、使用する処理系で確認します。

デバッガの基本操作

GDBを使う場合は、デバッグ情報を付けてビルドし、実行ファイルを指定して起動します。

```
cc -g -Og source.c -o program
gdb ./program
```

代表的な操作は次のとおりです。

```
break main
run
next
step
print variable
continue
backtrace
quit
```

LLDBなど別のデバッガではコマンドが異なります。

Makefileの最小例

複数の翻訳単位を扱う場合は、依存関係をビルドツールへ記述します。Makefileのレシピ行は、先頭をタブ文字にします。

```
CC = cc
CFLAGS = -std=c17 -Wall -Wextra -Wpedantic -g

program: main.o utility.o
    $(CC) $(CFLAGS) -o $@ $^

main.o: main.c utility.h
    $(CC) $(CFLAGS) -c main.c

utility.o: utility.c utility.h
    $(CC) $(CFLAGS) -c utility.c

clean:
    rm -f main.o utility.o program
```

ビルドコマンドをシェルのエイリアスへ埋め込むより、Makefileなどプロジェクト内の設定へ記録する方が、他の環境と同じ条件を再現しやすくなります。

演習問題

演習問題では、翻訳工程、警告、リンクエラー、デバッグ情報を確認できます。

演習問題

第1章：C言語入門の演習問題

演習の目的

この演習を通して、C言語の開発環境が正しく動作するか確認し、簡単なプログラムを作成・実行できるようになります。

基礎問題

問題1-1：Hello Worldの実行

`examples/hello_world.c` をコンパイルして実行してください。

```
# コンパイル

gcc examples/hello_world.c -o hello_world

# 実行

./hello_world
```

期待される出力

```
Hello, World!
```

問題1-2：自分でHello Worldを書く

新しいファイル `solutions/ex1_2_my_hello.c` を作成し、Hello Worldプログラムを自分で書いてください。

ヒント

- `#include <stdio.h>` を忘れずに
- `main` 関数の中に `printf` を書く
- 最後に `return 0;` を書く

問題1-3：日本語メッセージの表示

`solutions/ex1_3_japanese.c` を作成し、以下のメッセージを表示するプログラムを書いてください。

表示するメッセージ

```
こんにちは。
C言語の世界へようこそ。
プログラミングを始めます。
```

ヒント： `printf` を3回使うか、`\n` で改行します。

応用問題

問題1-4：簡単な自己紹介プログラム

`solutions/ex1_4_introduction.c` を作成し、自己紹介を表示するプログラムを書いてください。

要件

- 名前を表示
- 好きなものを1つ表示
- C言語を学ぶ理由を表示

実行例

```
=== 自己紹介 ===
名前: 山田太郎
好きなもの: プログラミング
C言語を学ぶ理由: ゲームを作りたいから。
```

問題1-5:数字を使った挨拶

`solutions/ex1_5_greeting_with_number.c` を作成し、名前と年齢を入力して挨拶するプログラムを書いてください。

実行例

```
お名前は□: 花子
年齢は□: 20
こんにちは、花子さん（20歳）。
```

ヒント

```
char name[50];
int age;
scanf("%s", name);
scanf("%d", &age);  /* 数字の入力には & が必要 */
```

チャレンジ問題

問題1-6:簡単な計算機

`solutions/ex1_6_calculator.c` を作成し、2つの数を足し算するプログラムを書いてください。

実行例

```
簡単な計算機
1つ目の数: 10
2つ目の数: 25
10 + 25 = 35
```

問題1-7:ASCIIアート

`solutions/ex1_7_ascii_art.c` を作成し、簡単なASCIIアートを表示してください。

例

```
  /\_/\
 ( o.o )
  > ^ <
```

または自分で好きな絵を作ってもOKです。

提出形式

各問題の回答は以下のファイルに記述してください:

- **問題1-2**:`solutions/ex1_2_my_hello.c`
- **問題1-3**:`solutions/ex1_3_japanese.c`
- **問題1-4**:`solutions/ex1_4_introduction.c`
- **問題1-5**:`solutions/ex1_5_greeting_with_number.c`
- **問題1-6**:`solutions/ex1_6_calculator.c`
- **問題1-7**:`solutions/ex1_7_ascii_art.c`

ヒント集

printfの使い方

```
printf("Display text\n");            /* 文字だけ */
printf("Name: %s\n", name);          /* 文字列を埋め込む */
printf("Number: %d\n", num);         /* 数字を埋め込む */
```

scanfの使い方

```
scanf("%s", name);    /* 文字列の入力（&不要） */
scanf("%d", &age);    /* 数字の入力（&必要） */
```

第2章：基本構文の演習問題

基礎問題

問題2-1：基本的な出力

プログラムを作成して以下を出力してください。

```
こんにちは、C言語の世界へ。
私の名前は［あなたの名前］です。
今日からプログラミングを始めます。
```

要求事項

- printf関数を使用すること
- 適切なエスケープシーケンスを使用すること
- ［あなたの名前］の部分は実際の名前に置き換えること

期待される出力例

```
こんにちは、C言語の世界へ。
私の名前は 山田太郎 です。
今日からプログラミングを始めます。
```

ファイル名： `ex2_1_hello_intro.c`

問題2-2：変数と基本データ型

以下の変数を定義し、値を代入して出力するプログラムを作成してください。

変数一覧

- 整数型：年齢（例：25）
- 浮動小数点型：身長（例：170.5）
- 文字型：血液型（例：'A'）
- 整数型：好きな数字（例：7）

出力例

```
=== 自己紹介データ ===
年齢：25歳
身長：170.5cm
血液型：A型
好きな数字：7
================
```

要求事項

- 適切なデータ型を選択すること
- printfの書式指定子を正しく使用すること
- コメントで各変数の説明を記述すること

期待される出力例

```
=== 自己紹介データ ===
年齢：25歳
身長：170.5cm
血液型：A型
好きな数字：7
================
```

ファイル名： `ex2_2_personal_data.c`

問題2-3：入力と出力

ユーザーから名前と年齢を入力してもらい、それを使って計算結果を表示するプログラムを作成してください。

機能

1 名前を入力してもらう

2 年齢を入力してもらう

3 10年後の年齢を計算する

4 結果を表示する

出力例

```
お名前を入力してください：田中太郎
年齢を入力してください：25

こんにちは、田中太郎さん。
現在の年齢：25歳
10年後の年齢：35歳
```

要求事項

- scanf関数を使用すること
- 適切なバッファーサイズを設定すること
- 計算結果を変数に保存すること

期待される出力例

```
お名前を入力してください：田中太郎
年齢を入力してください：25

こんにちは、田中太郎さん。
現在の年齢：25歳
10年後の年齢：35歳
```

ファイル名： `ex2_3_age_calculator.c`

応用問題

問題2-4：書式指定子の練習

様々な書式指定子を使って、数値を異なる形式で表示するプログラムを作成してください。

表示する数値： 123, 3.14159, 255

出力例

```
=== 書式指定子のデモ ===
整数 123 の表示：
  10進数：123
  16進数：7b
  8進数：173
  フィールド幅5：|  123|
  ゼロ埋め：|00123|

実数 3.14159 の表示：
  デフォルト：3.14159
  小数点以下2桁：3.14
  指数表記：3.14159e+00
  フィールド幅10.2：|      3.14|

文字コード 255：
  文字として：ÿ
  16進数：ff
  10進数：255
====================
```

要求事項

- 各種書式指定子を使用すること(%d, %x, %o, %f, %e, %cなど)
- フィールド幅とゼロ埋めを実演すること
- 適切なコメントを記述すること

期待される出力例

```
=== 書式指定子のデモ ===
整数 123 の表示:
  10進数: 123
  16進数: 7b
  8進数: 173
  フィールド幅5: |  123|
  ゼロ埋め: |00123|

実数 3.14159 の表示:
  デフォルト: 3.141590
  小数点以下2桁: 3.14
  指数表記: 3.141590e+00
  フィールド幅10.2: |      3.14|

文字コード 65:
  文字として: A
  16進数: 41
  10進数: 65
====================
```

ファイル名: `ex2_4_format_demo.c`

問題2-5:簡単な計算機

四則演算を実行する簡単な計算機プログラムを作成してください。

機能

1. 2つの数値を入力してもらう
2. 四則演算(+, -, *, /)の結果を表示する
3. 割り算では整数除算と実数除算の両方を表示する

出力例

```
簡単な計算機プログラム
===================
第1の数値を入力してください: 7
第2の数値を入力してください: 2

計算結果:
  7 + 2 = 9
  7 - 2 = 5
  7 * 2 = 14
  7 / 2 = 3 (整数除算)
  7 / 2 = 3.50 (実数除算)
```

要求事項

- 整数除算と実数除算を区別すること
- ゼロ除算のチェックは不要(基礎課題のため)
- 適切な変数名を使用すること

期待される出力例

```
簡単な計算機プログラム
==================
第1の数値を入力してください: 10
第2の数値を入力してください: 3

計算結果:
  10 + 3 = 13
  10 - 3 = 7
  10 * 3 = 30
  10 / 3 = 3 (整数除算)
  10 / 3 = 3.33 (実数除算)
```

ファイル名: `ex2_5_simple_calculator.c`

チャレンジ問題

問題2-6:文字とASCIIコード

文字とASCIIコードの関係を学ぶプログラムを作成してください。

機能

1. ユーザーから文字を入力してもらう
2. その文字のASCIIコードを表示する
3. ASCIIコード表の一部を表示する

出力例

```
文字を1つ入力してください: A
入力された文字: A
ASCIIコード: 65

ASCII表 (32-126) の一部:
 32:    33: !  34: "  35: #  36: $  37: %  38: &  39: '
 40: (  41: )  42: *  43: +  44: ,  45: -  46: .  47: /
 48: 0  49: 1  50: 2  51: 3  52: 4  53: 5  54: 6  55: 7
 56: 8  57: 9  58: :  59: ;  60: <  61: =  62: >  63: ?
 64: @  65: A  66: B  67: C  68: D  69: E  70: F  71: G
```

要求事項

- `getchar()`または`scanf(" %c", &ch)`を使用すること
- ループを使ってASCII表を表示すること
- 適切な書式で表を整列させること

期待される出力例

```
文字を1つ入力してください: A
入力された文字: A
ASCIIコード: 65

ASCII表 (32-126) の一部:
 32:    33: !  34: "  35: #  36: $  37: %  38: &  39: '
 40: (  41: )  42: *  43: +  44: ,  45: -  46: .  47: /
 48: 0  49: 1  50: 2  51: 3  52: 4  53: 5  54: 6  55: 7
 56: 8  57: 9  58: :  59: ;  60: <  61: =  62: >  63: ?
 64: @  65: A  66: B  67: C  68: D  69: E  70: F  71: G
```

ファイル名: `ex2_6_ascii_explorer.c`

問題2-7:文字列の基本操作

文字配列を使って基本的な文字列操作を学ぶプログラムを作成してください。

機能

1. ユーザーから短い文字列(単語)を入力してもらう

2　文字列の各文字を1文字ずつ表示

3　文字列を逆順に表示

4　各文字のASCIIコードを表示

出力例

```
単語を入力してください: Hello

入力された文字列: Hello

1文字ずつの表示:
H - e - l - l - o

逆順表示: olleH

各文字のASCIIコード:
H: 72
e: 101
l: 108
l: 108
o: 111
```

要求事項

- 文字配列を使用すること
- ループを使って文字を1つずつ処理すること
- 文字列の長さを数える処理を含めること

期待される出力例

```
単語を入力してください: Hello

入力された文字列: Hello

1文字ずつの表示:
H - e - l - l - o

逆順表示: olleH

各文字のASCIIコード:
H: 72
e: 101
l: 108
l: 108
o: 111
```

ファイル名: `ex2_7_string_basics.c`

問題2-8:プログラマー情報カード

プログラマーの情報を整形して表示するプログラムを作成してください。

機能

1　複数の情報を入力(名前、年齢、経験年数、好きな言語)

2　情報をカード形式で綺麗に表示

3　経験レベルを判定(初心者/中級者/上級者)

出力例

```
=== プログラマー情報入力 ===
名前: 山田太郎
年齢: 25
プログラミング経験年数: 3
好きなプログラミング言語: C

+-------------------------------------+
|      プログラマー情報カード            |
+-------------------------------------+
| 名前: 山田太郎                        |
| 年齢: 25歳                           |
| 経験: 3年                            |
| レベル: 中級者                        |
| 好きな言語: C                         |
+-------------------------------------+
```

経験レベルの判定基準

- 0-1年: 初心者
- 2-4年: 中級者
- 5年以上: 上級者

要求事項

- 適切な変数型を使用すること
- 条件分岐で経験レベルを判定すること
- 整形された出力を作成すること

期待される出力例

```
=== プログラマー情報入力 ===
名前: 山田太郎
年齢: 25
プログラミング経験年数: 3
好きなプログラミング言語: C

+-------------------------------------+
|      プログラマー情報カード            |
+-------------------------------------+
| 名前: 山田太郎                        |
| 年齢: 25歳                           |
| 経験: 3年                            |
| レベル: 中級者                        |
| 好きな言語: C                         |
+-------------------------------------+
```

ファイル名: `ex2_8_programmer_card.c`

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- 警告を有効にしてコンパイルし、実行結果を確認してください。

コンパイル例

```
# コンパイル例

gcc -o hello_intro ex2_1_hello_intro.c
gcc -o personal_data ex2_2_personal_data.c
gcc -o age_calculator ex2_3_age_calculator.c

# 実行例

./hello_intro
./personal_data
./age_calculator
```

評価基準

- **基本文法**：C言語の構文に従っていること
- **可読性**：インデント、変数名、コメントが処理内容を表していること
- **機能**：要求された入出力と計算を実装していること
- **書式指定**：データ型に対応する書式指定子を使用していること

学習項目

- C言語の基本的な入出力方法
- 変数の宣言と初期化
- 基本データ型の理解
- 書式指定子の使い方
- コメントの書き方
- プログラムの基本構造

第3章：データ型と変数の演習問題

基礎問題

演習3-1：変数宣言と初期化

さまざまなデータ型の変数を宣言し、初期化して値を表示するプログラムを作成してください。

要件

- 各基本データ型（char, short, int, long, float, double）の変数を宣言
- 適切な値で初期化
- printf関数で型に応じた書式指定子を使用して出力
- 符号付き・符号なしの両方を含める

期待される出力例

```
char型: 文字 = 'A', 値 = 65
unsigned char型: 値 = 255
short型: 値 = -1000
unsigned short型: 値 = 65535
int型: 値 = -123456
unsigned int型: 値 = 4294967295
float型: 値 = 3.14159
double型: 値 = 3.141592653589793
```

演習3-2：四則演算計算機

2つの数値を変数に格納し、四則演算（加算、減算、乗算、除算）の結果を表示するプログラムを作成してください。

要件

- 整数型と浮動小数点型の両方で実装
- 除算では整数除算と実数除算の違いを表示
- 各演算結果を見やすく表示

期待される出力例

```
=== 整数演算 ===
a = 15, b = 4
加算: 15 + 4 = 19
減算: 15 - 4 = 11
乗算: 15 * 4 = 60
除算: 15 / 4 = 3 (整数除算)

=== 実数演算 ===
x = 15.0, y = 4.0
加算: 15.00 + 4.00 = 19.00
減算: 15.00 - 4.00 = 11.00
乗算: 15.00 * 4.00 = 60.00
除算: 15.00 / 4.00 = 3.75
```

演習3-3：データ型サイズの確認

sizeof演算子を使って、各データ型のサイズを表示するプログラムを作成してください。

要件

- 基本データ型すべてのサイズを表示
- 配列のサイズも確認
- ポインターのサイズも確認
- サイズをバイト単位で表示

期待される出力例

```
=== データ型のサイズ ===
char: 1 バイト
short: 2 バイト
int: 4 バイト
long: 8 バイト
float: 4 バイト
double: 8 バイト

=== 配列のサイズ ===
int array[10]: 40 バイト
要素数: 10

=== ポインターのサイズ ===
char*: 8 バイト
int*: 8 バイト
double*: 8 バイト
```

応用問題

演習3-4：型変換の理解

整数除算と実数除算の違い、および暗黙的・明示的型変換を確認するプログラムを作成してください。

要件

- 同じ数値で整数除算と実数除算を実行し、結果を比較
- 暗黙的型変換が発生する例を実装
- 明示的型変換（キャスト）を使用した例を実装
- 精度の損失が発生する例を示す

期待される出力例

```
=== 整数除算と実数除算 ===
int a = 10, b = 3
整数除算: 10 / 3 = 3
実数除算（キャスト使用）: 10 / 3 = 3.333333

=== 暗黙的型変換 ===
int i = 10
double d = 3.14
i + d = 13.140000 (intがdoubleに変換)

=== 明示的型変換（キャスト） ===
double x = 3.75
(int)x = 3 (小数部分が切り捨て)

=== 精度の損失 ===
float f = 1234567890.123456
表示: 1234567936.000000（精度が失われる）
```

演習3-5：スコープの実験

グローバル変数、ローカル変数、静的変数を使い分けるプログラムを作成してください。

要件

- 同じ名前の変数を異なるスコープで宣言
- スコープの隠蔽（シャドウイング）を実演
- 静的変数を使ったカウンター関数を実装
- 各変数の値の変化を追跡して表示

期待される出力例

```
=== スコープの実験 ===
グローバル変数 x = 100

main関数内:
ローカル変数 x = 10 (グローバル変数を隠蔽)

ブロック内:
ブロック内の x = 5 (main関数の変数を隠蔽)
ブロックを出た後の x = 10

=== 静的変数カウンター ===
counter()の呼び出し 1回目: 1
counter()の呼び出し 2回目: 2
counter()の呼び出し 3回目: 3

グローバル変数 x = 100 (変更されていない)
```

演習3-6:定数の活用

constと#defineを使って定数を定義し、円の面積と円周を計算するプログラムを作成してください。

要件

- 円周率をconstと#defineの両方で定義
- 半径を入力として受け取る
- 面積と円周を計算して表示
- 定数を変更しようとした場合のエラーを確認(コメントで説明)

期待される出力例

```
円の計算プログラム
================
半径を入力してください: 5.0

計算結果:
半径: 5.00
円周: 31.42 (#define版の円周率使用)
面積: 78.54 (const版の円周率使用)

両方の円周率の値:
#define PI = 3.141593
const pi = 3.141593
```

チャレンジ問題

演習3-7:温度変換プログラム

摂氏・華氏・ケルビンの温度単位を相互変換するプログラムを作成してください。

要件

- 変換式で使用する定数を適切に定義
- 浮動小数点演算の精度に注意
- ユーザーフレンドリーな入出力
- 変換公式。
  - 華氏 = 摂氏 × 9/5 + 32
  - ケルビン = 摂氏 + 273.15

期待される出力例

```
=== 温度変換プログラム ===
温度を入力してください（摂氏）: 25.0

変換結果:
摂氏: 25.00 ℃
華氏: 77.00 ℉
ケルビン: 298.15 K

別の例（氷点）:
摂氏: 0.00 ℃
華氏: 32.00 ℉
ケルビン: 273.15 K

別の例（沸点）:
摂氏: 100.00 ℃
華氏: 212.00 ℉
ケルビン: 373.15 K
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と扱うデータ型をコメントで記載してください。
- C90規格と警告オプションを指定してコンパイルしてください。

コンパイル例

```
gcc -std=c90 -Wall -Wextra -pedantic ex_variables.c -o ex_variables
./ex_variables
```

ヒント

- printf関数の書式指定子を正しく使用する
  - %d: int
  - %u: unsigned int
  - %ld: long
  - %f: float/double
  - %c: char
- 初期化されていない変数の使用に注意
- 型の範囲を超えないよう注意
- sizeof演算子の戻り値はsize_t型（%luで出力）

第4章：演算子の演習問題

基礎問題

問題4-1：四則演算計算機

2つの整数を入力として受け取り、すべての算術演算（加算、減算、乗算、除算、剰余）の結果を表示するプログラムを作成してください。

要件

- ユーザーから2つの整数を入力
- 各演算の結果を見やすく表示
- ゼロ除算のチェックを含める

期待される出力例

```
2つの整数を入力してください: 10 3
10 + 3 = 13
10 - 3 = 7
10 * 3 = 30
10 / 3 = 3
10 % 3 = 1
```

ファイル名： `ex4_1_calculator.c`

問題4-2：比較と論理演算

2つの整数を入力として受け取り、関係演算子と論理演算子を使って最大値と最小値を求めるプログラムを作成してください。

要件

- 2つの整数を入力
- if文を使わず、条件演算子（三項演算子）のみで実装
- 最大値と最小値を表示

期待される出力例

```
2つの整数を入力してください: 15 8
最大値: 15
最小値: 8

別の例:
2つの整数を入力してください: -5 3
最大値: 3
最小値: -5
```

ファイル名： `ex4_2_comparison.c`

問題4-3：インクリメント・デクリメント

前置と後置のインクリメント・デクリメント演算子の違いを確認するプログラムを作成してください。

要件

- 変数に対して前置・後置の両方を使用
- 各操作後の値を表示
- 式の中での動作も確認

期待される出力例

```
=== インクリメント・デクリメント演算子の動作確認 ===
初期値: x = 5

後置インクリメント (x++): 5
操作後の x: 6

前置インクリメント (++x): 7
操作後の x: 7

初期値: y = 10
式の中での動作:
a = y++ + 2 = 12 (y = 11)
b = ++y + 2 = 14 (y = 12)

デクリメントも同様:
初期値: z = 8
c = z-- = 8 (z = 7)
d = --z = 6 (z = 6)
```

ファイル名: `ex4_3_increment.c`

応用問題

問題4-4:ビット操作

整数を入力として受け取り、その数値の各ビットを表示し、特定のビット操作を実行するプログラムを作成してください。

要件

- 2進数表示
- 指定したビット位置のON/OFF確認
- 特定ビットの設定・クリア・トグル機能

期待される出力例

```
整数を入力してください: 10
10 の2進数表示: 00001010

ビット位置を入力してください (0-7) : 2
ビット位置 2 は OFF です

操作を選択してください:

1. ビットを設定 (ON)
2. ビットをクリア (OFF)
3. ビットをトグル (反転)

選択: 1

操作後: 14 (00001110)
ビット位置 2 を ON にしました
```

ファイル名: `ex4_4_bitwise.c`

問題4-5:条件演算子の活用

3つの数値を入力として受け取り、条件演算子(三項演算子)のみを使って昇順に並び替えるプログラムを作成してください。

要件

- if文、switch文を使用しない
- 条件演算子のネストを活用
- 並び替えた結果を表示

期待される出力例

```
3つの整数を入力してください: 9 3 6
昇順に並び替え: 3 6 9

別の例:
3つの整数を入力してください: -5 10 0
昇順に並び替え: -5 0 10
```

ファイル名: `ex4_5_ternary_sort.c`

問題4-6:演算子優先順位

複雑な式を含むプログラムを作成し、演算子の優先順位による計算結果の違いを確認してください。

要件

- 同じ数値で異なる式を評価
- 括弧の有無による結果の違いを表示
- 少なくとも3種類以上の演算子を使用

期待される出力例

```
=== 演算子優先順位の確認 ===
a = 2, b = 3, c = 4

式1: a + b * c = 14
説明: 乗算が先 → 2 + (3 * 4) = 2 + 12 = 14

式2: (a + b) * c = 20
説明: 括弧内が先 → (2 + 3) * 4 = 5 * 4 = 20

式3: a << b + c = 256
説明: 加算が先 → 2 << (3 + 4) = 2 << 7 = 256

式4: a & b | c = 6
説明: &が先、次に| → (2 & 3) | 4 = 2 | 4 = 6
```

ファイル名: `ex4_6_precedence.c`

チャレンジ問題

問題4-7:複合代入演算子の応用

複合代入演算子をすべて使い、数値を変換するプログラムを作成してください。対象は+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=です。

要件

- 初期値から各種複合代入演算子を使って変換
- 各ステップでの値の変化を表示
- ビット演算の複合代入も含める

期待される処理

1. 初期値を入力
2. 各種演算を順番に適用
3. 最終結果と各ステップを表示

ファイル名: `ex4_7_compound_operators.c`

問題4-8:ビット演算による最適化

ビット演算を使って、以下の処理を高速に実行するプログラムを作成してください。

実装する機能

1. 2の累乗かどうかの判定
2. 2の累乗による乗算・除算
3. 偶数・奇数の判定
4. 数値の符号反転(2の補数)
5. 最下位の1ビットを抽出

要件

- 通常の演算子を使わずビット演算のみで実装
- 各機能の動作原理をコメントで説明
- テストケースを含める

ファイル名: `ex4_8_bit_optimization.c`

問題4-9:演算子組み合わせパズル

与えられた数値と演算子を使って、目標の値を作るプログラムを作成してください。

仕様

1. 4つの1桁の数字を入力
2. 目標値を入力
3. +, -, *, / の演算子を使用
4. 括弧の使用も可能
5. すべての数字を1回ずつ使用
6. 解が存在する場合は式を表示

例

```
数字を4つ入力: 2 3 4 5
目標値: 24
解: (2 + 3) * 4 + 5 - 5 = 24
```

ファイル名: `ex4_9_operator_puzzle.c`

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と使用する演算子をコメントで記載してください。
- C90規格と警告オプションを指定してコンパイルしてください。

コンパイル例

```
gcc -std=c90 -Wall -Wextra -pedantic ex4_1_calculator.c -o ex4_1_calculator
./ex4_1_calculator
```

評価ポイント

- **正確性**:要求された機能が正しく動作するか
- **エラー処理**:ゼロ除算などの例外的な状況への対処
- **コード品質**:可読性、適切なコメント、変数名

- **効率性**：特にビット演算問題での最適化

ヒント

- 演算子の優先順位に注意
- ゼロ除算やオーバーフローに対する考慮
- 可読性を意識したコードを心がける
- 適切なコメントを追加する
- ビット演算では2進数表現を意識する

第5章：条件分岐の演習問題

演習の目的

- if文、switch文の理解を深める
- 条件演算子の適切な使い方を習得する
- 複雑な条件分岐の設計力を養う

基礎問題

問題5-1：年齢による料金計算

映画館の料金システムを実装してください。

要件

- 3歳未満：無料
- 3歳以上12歳以下：子供料金（800円）
- 13歳以上18歳以下：学生料金（1200円）
- 19歳以上65歳以下：大人料金（1800円）
- 66歳以上：シニア料金（1200円）

実装のヒント

- if-else if文を使用
- 境界値に注意

期待される出力例

```
映画館料金計算システム
年齢を入力してください：25
お客様の料金は 1800円 です（大人料金）

別の例：
年齢を入力してください：5
お客様の料金は 800円 です（子供料金）

年齢を入力してください：70
お客様の料金は 1200円 です（シニア料金）
```

ファイル名： `ex5_1_movie_price.c`

問題5-2：電卓プログラム

四則演算ができる簡単な電卓を作成してください。

要件

- ユーザーから2つの数値と演算子（+, -, *, /）を入力
- switch文を使って演算を実行
- ゼロ除算のエラー処理を含める

実装のヒント

- 演算子はchar型で受け取る
- 除算の場合は分母が0でないかチェック

期待される出力例

```
簡単な電卓プログラム
第1の数値を入力: 15
演算子を入力 (+, -, *, /): *
第2の数値を入力: 3
計算結果: 15.00 * 3.00 = 45.00

別の例（ゼロ除算）:
第1の数値を入力: 10
演算子を入力 (+, -, *, /): /
第2の数値を入力: 0
エラー: ゼロで除算することはできません
```

ファイル名: `ex5_2_calculator.c`

問題5-3:BMI判定プログラム

身長と体重からBMIを計算し、判定するプログラムを作成してください。

要件

- BMI = 体重(kg) / (身長(m) × 身長(m))
- BMI判定基準:
  - 18.5未満: 低体重
  - 18.5以上25未満: 標準体重
  - 25以上30未満: 肥満度1
  - 30以上: 肥満度2

実装のヒント

- 身長はセンチメートルで入力してメートルに変換
- 浮動小数点数を使用

期待される出力例

```
BMI判定プログラム
身長(cm)を入力してください: 170
体重(kg)を入力してください: 65

BMI値: 22.49
判定: 標準体重です

別の例:
身長(cm)を入力してください: 165
体重(kg)を入力してください: 80

BMI値: 29.38
判定: 肥満度1です
```

ファイル名: `ex5_3_bmi_check.c`

応用問題

問題5-4:うるう年判定

西暦年を入力してうるう年かどうか判定するプログラムを作成してください。

うるう年の条件

1. 4で割り切れる年はうるう年
2. ただし、100で割り切れる年は平年
3. ただし、400で割り切れる年はうるう年

実装のヒント

- 複数の条件を論理演算子で組み合わせる
- 条件の順序に注意

期待される出力例

```
うるう年判定プログラム
西暦年を入力してください: 2024
2024年はうるう年です

別の例:
西暦年を入力してください: 1900
1900年は平年です

西暦年を入力してください: 2000
2000年はうるう年です
```

ファイル名: `ex5_4_leap_year.c`

問題5-5:成績評価システム

複数の科目の点数から総合評価を出すプログラムを作成してください。

要件

- 3科目(国語、数学、英語)の点数を入力
- 平均点を計算
- 平均点による評価(A〜F)
- 全科目60点以上の場合のみ「合格」、それ以外は「不合格」
- 条件演算子を使って簡潔に表示

評価基準

- 90点以上: A
- 80点以上90点未満: B
- 70点以上80点未満: C
- 60点以上70点未満: D
- 50点以上60点未満: E
- 50点未満: F

実装のヒント

- 平均点の計算は整数演算に注意
- 複数の条件を組み合わせる

期待される出力例

```
成績評価システム
国語の点数: 85
数学の点数: 92
英語の点数: 78

平均点: 85.00
評価: B
判定: 合格

別の例（不合格）:
国語の点数: 75
数学の点数: 55
英語の点数: 80

平均点: 70.00
評価: C
判定: 不合格（数学が60点未満）
```

ファイル名: `ex5_5_grade_system.c`

問題5-6:曜日判定プログラム

年月日を入力して、その日の曜日を判定するプログラムを作成してください。

要件

- ツェラーの公式を使用して曜日を計算
- 入力値の妥当性チェック(月は1-12、日は1-31など)
- 曜日を日本語で表示

ツェラーの公式(簡易版)

- 1月、2月は前年の13月、14月として計算

期待される出力例

```
曜日判定プログラム
年を入力: 2024
月を入力: 3
日を入力: 15

2024年3月15日は金曜日です

別の例:
年を入力: 2023
月を入力: 12
日を入力: 25

2023年12月25日は月曜日です
```

ファイル名: `ex5_6_weekday.c`

チャレンジ問題

問題5-7:じゃんけんゲーム

コンピューターとじゃんけんをするプログラムを作成してください。

要件

- ユーザーの手を数値で入力(1:グー、2:チョキ、3:パー)
- コンピューターの手はランダムに決定
- 勝敗を判定して表示
- 不正な入力のチェック

追加要件

- 3回勝負で最終的な勝者を決定
- 各回の結果を記録して最後に表示
- 連勝ボーナスポイントシステム

期待される出力例

```
=== じゃんけんゲーム（3回勝負） ===

第1回戦
あなたの手を選んでください（1:グー、2:チョキ、3:パー）：1
あなた：グー
コンピューター：チョキ
結果：あなたの勝ち。

第2回戦
あなたの手を選んでください（1:グー、2:チョキ、3:パー）：2
あなた：チョキ
コンピューター：チョキ
結果：あいこ

第3回戦
あなたの手を選んでください（1:グー、2:チョキ、3:パー）：3
あなた：パー
コンピューター：グー
結果：あなたの勝ち。（連勝ボーナス +1）

=== 最終結果 ===
あなた：3ポイント（2勝0敗1分）
コンピューター：0ポイント
優勝：あなた。
```

ファイル名： `ex5_7_janken_game.c`

問題5-8：交通信号シミュレーター

信号機の動作をシミュレートするプログラムを作成してください。

要件

- 現在の信号の色と経過時間から次の動作を決定
- 歩行者用信号も含める
- 緊急車両通過モード（すべて赤信号）
- 時間帯による点滅信号への切り替え

信号のパターン

- 青（30秒）→ 黄（3秒）→ 赤（30秒）
- 歩行者：青（20秒）→ 点滅（10秒）→ 赤（33秒）

期待される出力例

```
交通信号シミュレーター
現在の信号状態を入力（1:青、2:黄、3:赤）：1
経過時間（秒）：28
現在時刻（0-23時）：14

=== 信号状態 ===
車両用信号：青（残り2秒）
歩行者信号：赤
次の状態：まもなく黄信号に変わります

別の例（深夜）：
現在時刻（0-23時）：2
深夜モード：黄色点滅信号

別の例（緊急車両）：
緊急車両通過モード（1:有効、0:無効）：1
すべての信号：赤（緊急車両通過中）
```

ファイル名： `ex5_8_traffic_light.c`

問題5-9：複雑な料金計算システム

タクシー料金計算システムを実装してください。

要件

- 初乗り料金：1.052kmまで420円
- 加算料金：以降237mごとに80円
- 時間距離併用：時速10km以下では90秒ごとに80円
- 深夜割増（22:00-5:00）：2割増
- 予約料金、高速料金の加算オプション

入力項目

- 走行距離、走行時間、時刻、オプション選択

期待される出力例

```
タクシー料金計算システム
走行距離(km): 5.5
走行時間(分): 25
現在時刻(0-23): 23
予約利用(1:あり、0:なし): 0
高速道路利用(1:あり、0:なし): 0

=== 料金明細 ===
初乗り料金: 420円
加算料金: 760円 (4.448km分)
深夜割増 (20%) : 236円
-----------------
合計料金: 1,416円

別の例 (渋滞時) :
走行距離(km): 2.0
走行時間(分): 30
現在時刻(0-23): 18

=== 料金明細 ===
初乗り料金: 420円
加算料金: 320円 (距離加算)
時間加算料金: 640円 (低速走行)
-----------------
合計料金: 1,380円
```

ファイル名： `ex5_9_taxi_fare.c`

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と判定条件をコメントで記載してください。
- 警告を有効にしてコンパイルし、境界値を含む実行結果を確認してください。

評価基準

- **正確性**：要求仕様を満たしているか
- **可読性**：適切なインデント、変数名、コメント
- **効率性**：不要な条件分岐がないか
- **エラー処理**：不正な入力への対処

学習のポイント

- 条件の順序と網羅性
- switch文とif文の使い分け
- 条件演算子による簡潔な記述
- 複雑な条件の論理的な整理

第6章：繰り返し処理の演習問題

演習の目的

- for文、while文、do-while文の使い分けを理解する
- break文とcontinue文を適切に使えるようになる
- ネストしたループを活用した問題解決能力を養う

基礎問題

問題6-1：フィボナッチ数列

最初のN個のフィボナッチ数を表示するプログラムを作成してください。

要件

- ユーザーから表示する個数Nを入力
- フィボナッチ数列：0, 1, 1, 2, 3, 5, 8, 13, …
- for文を使用して実装

実装のヒント

- 最初の2つの数は0と1
- 3番目以降は前の2つの数の和

期待される出力例

```
フィボナッチ数列表示プログラム
表示する個数を入力してください: 10

フィボナッチ数列（最初の10個）:
0 1 1 2 3 5 8 13 21 34

別の例:
表示する個数を入力してください: 15

フィボナッチ数列（最初の15個）:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

ファイル名： `ex6_1_fibonacci.c`

問題6-2：数当てゲーム

1から100までの数当てゲームを作成してください。

要件

- プログラムがランダムに1～100の数を決定
- ユーザーが予想を入力
- 「もっと大きい」「もっと小さい」のヒントを出す
- 10回以内に当てられなければゲームオーバー
- do-while文を使用

実装のヒント

- `rand() % 100 + 1` でランダムな数を生成
- 試行回数をカウント

期待される出力例

```
=== 数当てゲーム ===
1から100までの数を当ててください。
10回以内に当てられればクリアです。

あなたの予想: 50
もっと小さい数です

あなたの予想: 25
もっと大きい数です

あなたの予想: 37
もっと大きい数です

あなたの予想: 43
もっと小さい数です

あなたの予想: 40
正解です。
4回で当てました。

別の例（失敗）:
（10回予想しても当たらない場合）
残念。答えは 72 でした。
```

ファイル名: `ex6_2_number_game.c`

問題6-3:掛け算表

9×9の掛け算表を表示するプログラムを作成してください。

要件

- ネストしたfor文を使用
- 見やすい表形式で出力
- 列と行のヘッダーを含める

出力例

```
    1   2   3   4   5   6   7   8   9
1   1   2   3   4   5   6   7   8   9
2   2   4   6   8  10  12  14  16  18
3   3   6   9  12  15  18  21  24  27
...
```

実装のヒント

- printf()の書式指定子で桁揃えを行う
- %3dなどを使用して桁数を統一

ファイル名: `ex6_3_multiplication_table.c`

応用問題

問題6-4:素数リスト

指定された範囲内のすべての素数を表示するプログラムを作成してください。

要件

- ユーザーから範囲(開始値と終了値)を入力
- その範囲内の素数をすべて表示
- 素数の個数も表示
- ネストしたループとbreak文を活用

実装のヒント

- 2からsqrt(n)まで割り切れるか確認
- 効率化のため、偶数は2以外スキップ

期待される出力例

```
素数リスト表示プログラム
開始値を入力してください: 10
終了値を入力してください: 50

10から50までの素数:
11 13 17 19 23 29 31 37 41 43 47

素数の個数: 11個

別の例:
開始値を入力してください: 1
終了値を入力してください: 30

1から30までの素数:
2 3 5 7 11 13 17 19 23 29

素数の個数: 10個
```

ファイル名: `ex6_4_prime_list.c`

問題6-5:図形の描画

アスタリスク(*)を使って様々な図形を描画するプログラムを作成してください。

要件

以下の4つの図形を描画する機能を実装:

1. 正方形
2. 直角三角形
3. 逆直角三角形
4. ダイヤモンド

例(サイズ5の場合)

正方形:

```
*****
*****
*****
*****
*****
```

直角三角形:

```
*
**
***
****
*****
```

逆直角三角形:

```
*****
****
***
**
*
```

ダイヤモンド:

```
  *
 ***
```

```
*****
 ***
  *
```

実装のヒント

- ネストしたループを使用
- 空白文字とアスタリスクの配置に注意

ファイル名： `ex6_5_draw_shapes.c`

問題6-6：完全数の探索

完全数を探すプログラムを作成してください。

完全数とは

その数自身を除く約数の和が、その数自身と等しくなる自然数例：6 = 1 + 2 + 3, 28 = 1 + 2 + 4 + 7 + 14

要件

- ユーザーから上限値を入力
- 1からその値までの完全数をすべて表示
- 各完全数について、約数も表示

期待される出力例

```
完全数探索プログラム
上限値を入力してください：1000

1から1000までの完全数：

6は完全数です
約数：1 + 2 + 3 = 6

28は完全数です
約数：1 + 2 + 4 + 7 + 14 = 28

496は完全数です
約数：1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248 = 496

見つかった完全数の個数：3個
```

ファイル名： `ex6_6_perfect_numbers.c`

チャレンジ問題

問題6-7：パスカルの三角形

パスカルの三角形を表示するプログラムを作成してください。

要件

- ユーザーから行数を入力（最大15行）
- 適切な配置で三角形を表示
- 各数値は前の行の2つの数の和

出力例（5行の場合）

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

実装のヒント

- 2次元配列を使用
- 適切な空白で中央揃え

ファイル名: `ex6_7_pascal_triangle.c`

問題6-8:コンソールアニメーション

簡単なコンソールアニメーションを作成してください。

要件

- 横に移動するアスタリスクのアニメーション
- 画面幅を考慮して左右に往復
- ESCキーまたはCtrl+Cで終了
- system("clear")またはsystem("cls")を使用

動作例

```
*                    (右に移動)

 *
  *
   *

    *                (端に到達したら折り返し)

   *
  *
 *

*                    (左端に戻る)
```

期待される出力例

```
コンソールアニメーション
Ctrl+Cで終了します...

(画面がクリアされて以下のアニメーションが繰り返される)

フレーム1:
*

フレーム2:
 *

フレーム3:
  *

...(続く)...

フレーム40:
                                        *

(端に到達後、逆方向に移動)

フレーム41:
                                       *

...(左端まで戻って繰り返し)...
```

ファイル名: `ex6_8_console_animation.c`

問題6-9:数字ピラミッド

数字を使ったピラミッドパターンを生成するプログラムを作成してください。

要件

- ユーザーから高さを入力(1-9)

- 複数のパターンを選択可能
- continue文を活用した効率的な実装

パターン例(高さ5)

パターン1:

```
    1
   2 2
  3 3 3
 4 4 4 4
5 5 5 5 5
```

パターン2:

```
    1
   1 2 1
  1 2 3 2 1
 1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

パターン3:

```
    1
   2 3 2
  3 4 5 4 3
 4 5 6 7 6 5 4
5 6 7 8 9 8 7 6 5
```

ファイル名: `ex6_9_number_pyramid.c`

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と終了条件をコメントで記載してください。
- 警告を有効にしてコンパイルし、境界値を含む実行結果を確認してください。

評価基準

- **正確性**:要求仕様を満たしているか
- **効率性**:適切なループの選択と最適化
- **可読性**:適切なインデント、変数名、コメント
- **創造性**:チャレンジ問題での工夫

学習のポイント

- for、while、do-whileの使い分け
- breakとcontinueの適切な使用
- ネストループの理解と活用
- ループの効率化技術

第7章：配列の演習問題

演習の目的

- 1次元配列と多次元配列の理解を深める
- 配列を使った実践的な問題解決能力を養う
- 基本的な配列操作を身に付ける

基礎問題

問題7-1：配列の基本操作

整数配列に対する各種操作を実行するプログラムを作成してください。

要件

- 5個の整数を入力して配列に格納
- 以下の操作を実装：
  - 配列の全要素を表示
  - 最大値と最小値を見つける
  - 平均値を計算
  - 配列を逆順に並べ替える
  - 指定した値が配列内に存在するか検索

実装のヒント

- 配列のサイズは#defineで定義
- 逆順は別の配列にコピーまたはその場で交換

期待される出力例

```
配列操作プログラム
5個の整数を入力してください: 34 12 67 89 23

配列の内容: 34 12 67 89 23
最大値: 89
最小値: 12
平均値: 45.00

逆順配列: 23 89 67 12 34

検索する値を入力: 67
値 67 は位置 2 に存在します（0から数えて）

検索する値を入力: 50
値 50 は配列内に存在しません
```

ファイル名： `ex7_1_array_operations.c`

問題7-2：配列の要素シフト

配列の要素を左右にシフトするプログラムを作成してください。

要件

- 10個の整数を配列に格納
- 以下の操作を実装：
  - 全要素を右に1つシフト（最後の要素は先頭へ）

    - 全要素を左に1つシフト(最初の要素は最後へ)
    - 指定した数だけシフト
    - シフト前後の配列を表示

実装のヒント

- 一時変数を使って端の要素を保存
- ループの方向に注意

期待される出力例

```
配列シフトプログラム
初期配列: 1 2 3 4 5 6 7 8 9 10

右に1シフト: 10 1 2 3 4 5 6 7 8 9
左に1シフト: 2 3 4 5 6 7 8 9 10 1

シフト数を入力してください: 3
右に3シフト: 8 9 10 1 2 3 4 5 6 7

シフト数を入力してください: -2
左に2シフト: 3 4 5 6 7 8 9 10 1 2
```

ファイル名: `ex7_2_array_shift.c`

問題7-3:配列の統計処理

整数配列に対する基本的な統計処理を行うプログラムを作成してください。

要件

- 10個の整数データを配列に格納
- 以下の統計値を計算:
    - 合計値と平均値
    - 最大値と最小値およびその位置
    - 標準偏差(簡易計算でよい)

実装のヒント

- 標準偏差は各要素と平均の差の二乗の平均の平方根

期待される出力例

```
配列統計処理プログラム
10個の整数を入力してください: 78 65 92 83 71 96 54 88 67 79

=== 統計情報 ===
データ: 78 65 92 83 71 96 54 88 67 79
合計値: 773
平均値: 77.30
最大値: 96 (位置: 5)
最小値: 54 (位置: 6)
標準偏差: 12.85
```

ファイル名: `ex7_3_array_statistics.c`

応用問題

問題7-4:成績管理システム

複数の学生の複数科目の成績を管理するプログラムを作成してください。

要件

- 5人の学生、4科目(国語、数学、英語、理科)の成績を2次元配列で管理

- 各学生の合計点と平均点を計算
- 各科目の平均点を計算
- 最高得点の学生と科目を表示
- 成績表を見やすく表示

実装のヒント

- 2次元配列grades[][] を使用
- 行が学生、列が科目

期待される出力例

```
成績管理システム

成績表:
学生   国語   数学   英語   理科   合計   平均
-----------------------------------------
学生1   85     92     78     88    343  85.8
学生2   76     88     91     83    338  84.5
学生3   92     76     85     90    343  85.8
学生4   68     95     73     79    315  78.8
学生5   89     84     88     86    347  86.8
-----------------------------------------
科目平均 82.0  87.0  83.0  85.2

最高得点: 学生4の数学 95点
クラス全体の平均点: 84.3
```

ファイル名: `ex7_4_grade_management.c`

問題7-5:行列演算

2次元配列を使って行列演算を実行するプログラムを作成してください。

要件

- 3x3の行列を2つ入力
- 以下の演算を実装:
  - 行列の加算
  - 行列の減算
  - 行列の乗算
  - 転置行列の計算

実装のヒント

- 行列の乗算:`result[i][j] = Σ(a[i][k] * b[k][j])`
- 各演算結果を見やすく表示

期待される出力例

```
行列演算プログラム

行列A:
1  2  3
4  5  6
7  8  9

行列B:
9  8  7
6  5  4
3  2  1

加算結果 (A + B):
10  10  10
10  10  10
10  10  10

減算結果 (A - B):
-8  -6  -4
-2   0   2
 4   6   8

乗算結果 (A x B):
30  24  18
84  69  54
138 114  90

転置行列 (A^T):
1  4  7
2  5  8
3  6  9
```

ファイル名： `ex7_5_matrix_operations.c`

問題7-6：頻度分布表

テストの点数分布を分析するプログラムを作成してください。

要件

- 30人分のテスト点数（0-100点）を配列に格納
- 10点刻みで頻度分布表を作成
- ヒストグラムを*を使って表示
- 最頻値の範囲を表示

出力例

```
0- 9: ** (2人)
10-19: *** (3人)
20-29: * (1人)
...
90-99: ******** (8人)
```

ファイル名： `ex7_6_frequency_distribution.c`

チャレンジ問題

問題7-7：ソートアルゴリズムの実装

配列のソートアルゴリズムを複数実装して比較してください。

要件

- 以下のソートアルゴリズムを実装：
  - バブルソート
  - 選択ソート

    - 挿入ソート
- 各アルゴリズムの実行時間を測定
- ランダムな配列、ソート済み配列、逆順配列で性能比較

追加要件

- 比較回数と交換回数をカウント
- 結果をグラフィカルに表示(*を使った簡易グラフ)

期待される出力例

```
ソートアルゴリズムの比較

配列サイズ: 20
元の配列: 34 7 23 32 5 62 78 4 87 12 45 28 91 15 53 67 39 22 84 10

=== バブルソート ===
ソート後: 4 5 7 10 12 15 22 23 28 32 34 39 45 53 62 67 78 84 87 91
比較回数: 190
交換回数: 92
実行時間: 0.002ms

=== 選択ソート ===
ソート後: 4 5 7 10 12 15 22 23 28 32 34 39 45 53 62 67 78 84 87 91
比較回数: 190
交換回数: 19
実行時間: 0.001ms

=== 挿入ソート ===
ソート後: 4 5 7 10 12 15 22 23 28 32 34 39 45 53 62 67 78 84 87 91
比較回数: 92
交換回数: 92
実行時間: 0.001ms

効率性比較グラフ（比較回数）:
バブル: ******************** (190)
選択 : ******************** (190)
挿入 : ********** (92)
```

ファイル名: `ex7_7_sort_algorithms.c`

問題7-8:ライフゲーム

コンウェイのライフゲームの簡易版を実装してください。

要件

- 20x20の2次元配列でセルの状態を管理
- セルは生(1)または死(0)の状態
- 各世代で以下のルールを適用:
    - 生きているセルの周囲に2-3個の生きたセルがある→生存
    - 死んでいるセルの周囲にちょうど3個の生きたセルがある→誕生
    - それ以外→死亡
- 初期状態をランダムまたはパターンで設定
- 各世代の状態を表示

期待される出力例

```
コンウェイのライフゲーム（20x20）

世代 0:
....................
......***..........
......*..*.........
.......*...........
....................
（以下省略）

世代 1:
....................
.......*...........
......*..*.........
......*............
....................
（以下省略）

世代 2:
....................
......***..........
......*..*.........
.......*...........
....................
（パターンが繰り返される）

10世代後の生存セル数: 8
```

ファイル名： `ex7_8_life_game.c`

問題7-9：魔方陣の生成と検証

魔方陣を生成・検証するプログラムを作成してください。

要件

- 3x3の魔方陣を生成（1-9の数字を使用）
- 既存の配列が魔方陣かどうかを検証
- 検証条件：
  - 各行の合計が等しい
  - 各列の合計が等しい
  - 対角線の合計が等しい
  - 1-9の数字が重複なく使用されている
- 複数の魔方陣パターンを生成可能

期待される出力例

```
魔方陣生成・検証プログラム

生成された魔方陣:
2  7  6
9  5  1
4  3  8

検証結果:
行の合計: 15, 15, 15 (PASS)
列の合計: 15, 15, 15 (PASS)
対角線の合計: 15, 15 (PASS)
すべての数字が1-9で重複なし (PASS)

これは正しい魔方陣です。

別の魔方陣パターン:
8  1  6
3  5  7
4  9  2

検証結果: 正しい魔方陣です。
```

ファイル名: `ex7_9_magic_square.c`

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ファイルの冒頭に問題番号と実装内容をコメントで記載してください。
- 警告を有効にしてコンパイルし、実行結果を確認してください。

評価基準

- **正確性**:要求仕様を満たしているか
- **効率性**:配列操作の最適化
- **可読性**:適切なインデント、変数名、コメント
- **境界処理**:配列の境界チェック

学習のポイント

- 配列の初期化と要素アクセス
- 多次元配列の使い方
- 配列を使った効率的なデータ処理
- 配列の境界を超えないための注意点

第8章：ポインターの演習問題

基礎問題

演習8-1：ポインターの基本操作

2つの整数変数の値をポインターを使って交換するプログラムを作成してください。

要件

- 2つの整数変数を宣言・初期化
- ポインターを使った値の交換関数を実装
- 交換前後の値とアドレスを表示
- 直接的な値の交換とポインター経由の交換を比較

期待される出力例

```
交換前: a = 10 (アドレス: 0x7fffbffac), b = 20 (アドレス: 0x7fffbffa8)
ポインターを使った交換実行...
交換後: a = 20 (アドレス: 0x7fffbffac), b = 10 (アドレス: 0x7fffbffa8)
```

演習8-2：配列とポインター

ポインター演算を使って配列の要素を逆順に表示するプログラムを作成してください。

要件

- 整数配列を宣言・初期化
- ポインター演算で配列の最後から最初に向かってアクセス
- インデックス記法とポインター記法の両方で表示
- 配列のサイズを動的に計算

期待される出力例

```
配列の内容（順方向）：
インデックス記法: arr[0]=1, arr[1]=2, arr[2]=3, arr[3]=4, arr[4]=5
ポインター記法: *ptr=1, *(ptr+1)=2, *(ptr+2)=3, *(ptr+3)=4, *(ptr+4)=5

配列の内容（逆順）：
ポインター演算で逆順表示: 5 4 3 2 1

各要素のアドレス:
&arr[0] = 0x7fffc000
&arr[1] = 0x7fffc004
&arr[2] = 0x7fffc008
&arr[3] = 0x7fffc00c
&arr[4] = 0x7fffc010
```

演習8-3：文字列操作

ポインターを使って文字列の長さを計算し、文字列を逆順にするプログラムを作成してください。

要件

- 文字列の長さ計算関数（strlen相当）をポインターで実装
- 文字列を逆順にする関数をポインターで実装
- 元の文字列と逆順文字列を表示
- 文字列リテラルと文字配列の違いを考慮

期待される出力例

```
文字列操作プログラム
文字列を入力: Hello World

元の文字列: Hello World
文字列の長さ: 11文字

文字列を逆順に変換中...
逆順文字列: dlroW olleH

各文字とそのアドレス:
'H' at 0x7fffc100
'e' at 0x7fffc101
'l' at 0x7fffc102
... (以下省略)
```

応用問題

演習8-4:配列操作関数

ポインターを使って配列の最大値、最小値、平均値を計算する関数群を作成してください。

要件

- 最大値を見つけてそのポインターを返す関数
- 最小値を見つけてそのポインターを返す関数
- 平均値を計算する関数(戻り値はdouble)
- 各関数の結果を使ってレポートを作成

実装する関数

```
int* find_max(int *arr, int size);
int* find_min(int *arr, int size);
double calculate_average(int *arr, int size);
void print_statistics(int *arr, int size);
```

期待される出力例

```
配列統計分析プログラム
配列: 34 12 67 89 23 45 78 56 90 11

=== 統計情報 ===
最大値: 90 (アドレス: 0x7fffc120, インデックス: 8)
最小値: 11 (アドレス: 0x7fffc124, インデックス: 9)
平均値: 50.50

値の範囲: 11 □ 90
```

演習8-5:文字列処理

ポインターを使って文字列の検索、置換、分割を実行する関数群を作成してください。

要件

- 文字列内で特定の文字を検索する関数
- 文字列内の文字を置換する関数
- 文字列を特定の文字で分割する関数
- 大文字・小文字を無視した比較機能

期待される出力例

```
文字列処理プログラム
文字列: Hello,World,Programming

文字 'o' の検索:
見つかった位置: 4, 7, 18

文字 'o' を 'O' に置換:
置換後: HellO,WOrld,PrOgramming

カンマで分割:
1: Hello
2: World
3: Programming

大文字小文字を無視した検索 ('w'):
見つかった位置: 6 (W)
```

演習8-6:データ変換

ポインターを使って配列のデータ型変換を実行するプログラムを作成してください。

要件

- int配列をfloat配列に変換
- 文字列を数値配列に変換
- バイト配列を整数として解釈
- エラー処理の実装

期待される出力例

```
データ型変換プログラム

=== int配列からfloat配列への変換 ===
元のint配列: 10, 20, 30, 40, 50
変換後のfloat配列: 10.00, 20.00, 30.00, 40.00, 50.00

=== 文字列から数値配列への変換 ===
文字列: "12 34 56 78 90"
数値配列: 12, 34, 56, 78, 90

=== バイト配列を整数として解釈 ===
バイト配列: 0x01 0x02 0x03 0x04
リトルエンディアン解釈: 67305985 (0x04030201)
ビッグエンディアン解釈: 16909060 (0x01020304)
```

発展問題

演習8-7:メモリ操作

ポインターを使って任意のデータ型の配列をコピーする汎用関数を作成してください。

要件

- void *ポインターを使った汎用コピー関数
- バイト単位でのメモリコピー
- 型安全性の考慮
- コピー対象の境界チェック

関数プロトタイプ

```
void* generic_copy(void *dest, const void *src, size_t size);
int compare_memory(const void *ptr, const void *ptr, size_t size);
```

演習8-8：アルゴリズム実装

ポインターを使って各種ソートアルゴリズムを実装してください。

要件

- バブルソート（ポインター版）
- 選択ソート（ポインター版）
- 挿入ソート（ポインター版）
- 汎用的な比較関数の使用

演習8-9：データ構造

ポインターを使って簡単なリンクリスト構造を実装してください。

要件

- ノード構造体の定義
- リストへの要素追加・削除
- リストの走査・検索
- メモリリークの防止

構造体例

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

チャレンジ問題

演習8-10：動的配列シミュレータ

malloc/freeを使わずに、静的配列とポインターを使って動的配列のような動作を実現してください。

要件

- 固定サイズの大きな配列をメモリプールとして使用
- 要素の追加・削除・挿入機能
- メモリの断片化管理
- ガベージコレクション機能

提出物

- 問題ごとに独立したCファイルを作成し、処理内容が分かるファイル名を付けてください。
- ファイルの冒頭に問題番号とポインターの用途をコメントで記載してください。
- C90規格と警告オプションを指定してコンパイルしてください。

コンパイル例

```
gcc -std=c90 -Wall -Wextra -pedantic ex_pointer_swap.c -o ex_pointer_swap
./ex_pointer_swap
```

ヒント

ポインターの基本

- 宣言: `int *ptr;`
- アドレス取得: `ptr = &variable;`
- 値の取得: `value = *ptr;`
- 演算: `ptr + 1` は次の要素を指す

安全なプログラミング

- NULLポインターのチェック: `if (ptr != NULL)`
- 配列の境界チェック
- 初期化されていないポインターの使用を避ける

デバッグのコツ

- アドレスと値を分けて表示
- ポインター演算の結果を段階的に確認
- コンパイラーの警告に注意を払う

よくある間違い

- `*ptr++` と `(*ptr)++` の違い
- ローカル変数のアドレスを返すこと
- 配列の範囲外アクセス
- NULLポインターの参照

第9章：配列とポインターの演習問題

演習の目的

- 配列とポインターの関係を深く理解する
- ポインター演算を使って配列を操作できるようになる
- 関数への配列の渡し方を習得する
- より効率的な配列処理技術を身につける

基礎問題

問題9-1：ポインターによる配列走査

ポインターを使って配列を操作するプログラムを作成してください。

要件

- 10個の整数を配列に格納
- 以下の操作をポインターを使って実装：
  - 配列の全要素を表示（配列記法を使わない）
  - 最大値と最小値を見つける（ポインター演算使用）
  - 配列を逆順に並べ替える（ポインターで実装）
  - 特定の値を検索し、そのアドレスを返す

実装のヒント

- `arr[i]`の代わりに`*(arr + i)`を使用
- ポインターのインクリメント/デクリメントを活用

期待される出力例

```
=== ポインターによる配列操作 ===
配列の内容: 34 67 12 89 23 45 78 56 90 11

ポインター記法での表示:
*(ptr+0) = 34
*(ptr+1) = 67
*(ptr+2) = 12
... (続く)

最大値: 90 (アドレス: 0x7fffc120)
最小値: 11 (アドレス: 0x7fffc124)

配列を逆順に並べ替え中...
逆順配列: 11 90 56 78 45 23 89 12 67 34

値 45 を検索...
値 45 は アドレス 0x7fffc114 に見つかりました（インデックス: 5）
```

問題9-2：関数への配列渡し

配列を関数に渡して操作するプログラムを作成してください。

要件

- 配列操作を行う以下の関数を実装：
  - 配列の要素をすべてn倍する関数
  - 2つの配列を要素ごとに加算する関数

    - 配列内の最大値へのポインターを返す関数
    - 配列を指定位置で分割する関数

実装のヒント

- 関数の引数は`int *arr`または`int arr[]`
- サイズ情報は別引数で渡す

期待される出力例

```
=== 関数への配列渡し ===
元の配列: 1 2 3 4 5

配列の要素を3倍にする:
結果: 3 6 9 12 15

配列A: 10 20 30 40 50
配列B: 5 10 15 20 25
要素ごとの加算結果: 15 30 45 60 75

配列内の最大値: 75 (アドレス: 0x7fffc1a4)

配列を位置3で分割:
前半: 15 30 45
後半: 60 75
```

応用問題

問題9-3:文字列のポインター操作

文字列をポインターで操作するプログラムを作成してください。

要件

- ポインターを使った以下の文字列関数を実装:
    - 文字列の長さを計算(strlen相当)
    - 文字列をコピー(strcpy相当)
    - 文字列を連結(strcat相当)
    - 文字列を反転

実装のヒント

- `char *`を使った操作
- ヌル終端文字の処理に注意

期待される出力例

```
=== 文字列のポインター操作 ===
文字列1: Hello
文字列2: World

my_strlen("Hello") = 5

文字列コピー実行...
コピー結果: Hello

文字列連結実行...
連結結果: HelloWorld

文字列反転実行...
反転結果: dlroWolleH
```

問題9-4:多次元配列とポインター

2次元配列をポインターで操作するプログラムを作成してください。

要件

- 3×3の行列に対する以下の操作：
  - ポインターを使った要素アクセス
  - 行列の転置（ポインター使用）
  - 対角要素の合計
  - 行ごと、列ごとの合計

実装のヒント

- `*(*(arr + i) + j)`でアクセス
- `int (*ptr)[3]`の使い方を理解

期待される出力例

```
=== 多次元配列とポインター ===
元の行列:
1 2 3
4 5 6
7 8 9

ポインターを使った要素アクセス:
*(*(matrix+1)+1) = 5

転置行列:
1 4 7
2 5 8
3 6 9

対角要素の合計: 15 (1 + 5 + 9)

行ごとの合計: 6, 15, 24
列ごとの合計: 12, 15, 18
```

問題9-5：動的配列の模擬

ポインターを使った配列の動的な操作を行うプログラムを作成してください。

要件

- 固定サイズ配列を使いながら動的配列のような操作：
  - 使用中の要素数を管理
  - 要素の追加（容量チェック付き）
  - 要素の削除（詰める処理）
  - 配列の拡張（別の大きな配列へコピー）

実装のヒント

- 実際の要素数と配列容量を別管理
- ポインターを使った効率的なコピー

期待される出力例

```
=== 動的配列の模擬 ===
初期容量: 10, 使用中: 0

要素を追加: 10, 20, 30
現在の配列: [10, 20, 30]
容量: 10, 使用中: 3

要素 20 を削除...
現在の配列: [10, 30]
容量: 10, 使用中: 2

要素を追加して容量オーバー時の処理:
新しい容量: 20
配列の内容をコピー中...
拡張完了。
```

チャレンジ問題

問題9-6：高速文字列検索

ポインターを使った効率的な文字列検索アルゴリズムを実装してください。

要件

- Boyer-Moore法の簡易版を実装：
  - 文字列内から部分文字列を検索
  - 見つかった位置のポインターを返す
  - 複数箇所で見つかる場合はすべて表示

追加要件

- 検索回数をカウントして効率性を評価
- 通常の逐次検索と比較

期待される出力例

```
=== 高速文字列検索 ===
テキスト: "The quick brown fox jumps over the lazy dog"
検索パターン: "the"

Boyer-Moore法（簡易版）:
見つかった位置: 31 (0x7fffc200)
検索回数: 12回

通常の逐次検索:
見つかった位置: 31
検索回数: 32回

効率性向上: 62.5%
```

問題9-7：メモリプールの実装

大きな配列を使ったメモリプールを実装してください。

要件

- 大きなchar配列をメモリプールとして使用
- 以下の機能を実装：
  - 指定サイズのメモリ割り当て
  - メモリの解放
  - 使用状況の表示
  - フラグメンテーションの管理

実装のヒント

- ポインター演算でメモリブロックを管理
- 各ブロックの使用状況をヘッダで管理

期待される出力例

```
=== メモリプール管理 ===
プールサイズ: 1024 バイト

メモリ割り当て:
ブロック1: 100バイト確保 (アドレス: 0x7fffc000)
ブロック2: 200バイト確保 (アドレス: 0x7fffc064)
ブロック3: 150バイト確保 (アドレス: 0x7fffc12c)

使用状況:
使用中: 450バイト (43.9%)
空き: 574バイト (56.1%)

ブロック2を解放...
フラグメンテーション発生: 3個の断片

デフラグ実行...
連続した空き領域: 774バイト
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- ポインターを使う理由と、境界を保証する条件をコメントで記載してください。
- 配列境界とNULLポインターを検査してください。

学習のポイント

- 配列記法とポインター記法の変換を意識
- ポインター演算の正確な理解
- 関数への配列渡しの仕組みを理解
- メモリレイアウトを常に意識して実装

第10章：文字列の演習問題

演習の目的

- C言語の文字列（文字配列）の仕組みを深く理解する
- 標準ライブラリの文字列関数を使いこなす
- 安全な文字列操作の実装方法を習得する
- 文字列処理アルゴリズムを実装する

基礎問題

問題10-1：基本的な文字列操作

標準ライブラリの関数を使わずに、基本的な文字列操作関数を実装してください。

要件

- 以下の関数を実装：
  - `my_strlen()` - 文字列の長さを計算
  - `my_strcpy()` - 文字列をコピー
  - `my_strcmp()` - 文字列を比較
  - `my_strcat()` - 文字列を連結

実装のヒント

- ヌル終端文字（'\0'）の扱いに注意
- バッファーオーバーフローを防ぐ

期待される出力例

```
=== 基本的な文字列操作 ===
文字列1: "Hello"
文字列2: "World"

my_strlen("Hello") = 5
my_strlen("World") = 5

my_strcpy実行後: "Hello"
my_strcmp("Hello", "World") = -15 (Hello < World)
my_strcmp("Hello", "Hello") = 0 (等しい)

my_strcat実行後: "HelloWorld"
```

問題10-2：文字列の検索と置換

文字列内の文字や部分文字列を検索・置換する関数を実装してください。

要件

- 以下の関数を実装：
  - `my_strchr()` - 文字を検索
  - `my_strrchr()` - 文字を後方検索
  - 特定文字の出現回数をカウント
  - 文字列内の特定文字を別の文字に置換

実装のヒント

- ポインターを使った効率的な実装
- NULLチェックを忘れずに

期待される出力例

```
=== 文字列の検索と置換 ===
文字列: "Hello, World!"

文字 'o' の検索:
my_strchr: 最初の 'o' は位置 4
my_strrchr: 最後の 'o' は位置 7
出現回数: 2回

文字 'l' を 'L' に置換:
置換後: "HeLLo, WorLd!"
```

応用問題

問題10-3：文字列配列の管理

学生名を格納する文字列配列の管理システムを実装してください。

要件

- 最大20人の学生名を管理（各名前は最大50文字）
- 以下の機能を実装：
  - 学生名の追加
  - 学生名の削除
  - 学生名の検索
  - アルファベット順にソート
  - 全学生名の表示

実装のヒント

- 二次元配列を使用
- strcmp()でソートを実装

期待される出力例

```
=== 学生名管理システム ===

1. 学生追加
2. 学生削除
3. 学生検索
4. アルファベット順ソート
5. 全学生表示
6. 終了

選択: 1
学生名: Tanaka
追加しました。

選択: 1
学生名: Sato
追加しました。

選択: 5
--- 学生一覧 ---

1. Tanaka
2. Sato

選択: 4
ソート完了。

選択: 5
--- 学生一覧 ---

1. Sato
2. Tanaka
```

問題10-4：文字列の分割と結合

文字列を特定のデリミタで分割・結合する関数を実装してください。

要件

- カンマ区切りの文字列を分割して配列に格納
- 配列の文字列を指定のデリミタで結合
- 空白文字（スペース、タブ、改行）で分割
- トークンの前後の空白を除去（トリム機能）

実装のヒント

- strtok()の仕組みを理解
- 動的な配列管理を考慮

期待される出力例

```
=== 文字列の分割と結合 ===
元の文字列: "apple,banana,orange,grape"

カンマで分割:
1: "apple"
2: "banana"
3: "orange"
4: "grape"

スペースで結合: "apple banana orange grape"

空白文字で分割（トリム付き）:
元の文字列: "  hello   world  \t programming  \n"
1: "hello"
2: "world"
3: "programming"
```

問題10-5：文字列の変換と検証

文字列の変換および検証を行う関数を実装してください。

要件

- 文字列を大文字/小文字に変換
- 文字列が数値として有効か検証
- 文字列を整数/浮動小数点数に変換
- パスワードの強度チェック（英数字、記号を含む8文字以上）

実装のヒント

- ctype.hの関数を活用
- エラー処理を適切に

期待される出力例

```
=== 文字列の変換と検証 ===
文字列: "Hello World 123"
大文字変換: "HELLO WORLD 123"
小文字変換: "hello world 123"

数値検証:
"123" → 有効な整数
"12.34" → 有効な実数
"abc123" → 無効な数値

文字列を数値に変換:
"123" → 123 (整数)
"45.67" → 45.67 (実数)

パスワード強度チェック:
"password" → 弱い (英字のみ)
"Pass123!" → 強い (英数字と記号を含む8文字)
```

チャレンジ問題

問題10-6：高度な文字列検索

パターンマッチングアルゴリズムを実装してください。

要件

- ワイルドカード検索（*、？）の実装
- 大小文字を区別しない検索
- 単語境界での検索（単語単位の検索）
- 複数パターンの同時検索

追加要件

- 検索アルゴリズムの効率性を考慮
- KMP法やBoyer-Moore法の簡易版を実装

期待される出力例

```
=== 高度な文字列検索 ===
テキスト: "The quick brown fox jumps over the lazy dog"

ワイルドカード検索:
パターン "*fox*" □ マッチ (位置: 16)
パターン "qu?ck" □ マッチ (位置: 4)

大小文字を区別しない検索:
"THE" □ 見つかった位置: 0, 31

単語境界検索:
"the" □ 見つかった単語位置: 31 ("The"は単語境界でマッチしない)

複数パターン同時検索:
パターン: ["quick", "fox", "dog"]
結果:

- "quick": 位置 4
- "fox": 位置 16
- "dog": 位置 40
```

問題10-7:テキストエディターの基本機能

簡単なテキストエディターの基本機能を実装してください。

要件

- 複数行のテキストを管理(最大100行、各行最大80文字)
- 以下の機能を実装:
  - 行の挿入・削除
  - 文字列の検索・置換(全置換機能付き)
  - 行番号付きの表示
  - ファイルへの保存と読み込み

実装のヒント

- 二次元配列でテキストを管理
- ファイルI/Oの適切なエラー処理

期待される出力例

```
=== 簡易テキストエディター ===

1. 行の挿入
2. 行の削除
3. 検索
4. 置換
5. 表示
6. 保存
7. 読み込み
8. 終了

選択: 1
行番号: 1
内容: Hello, World!
挿入しました。

選択: 5
1: Hello, World!

選択: 3
検索文字列: World
見つかりました: 行1, 位置7

選択: 4
検索文字列: World
置換文字列: Programming
全置換しますか(y/n): y
1件置換しました。

選択: 5
1: Hello, Programming!
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- バッファーサイズと終端文字を検査し、入力エラー時の動作を実装してください。
- 警告を有効にしてコンパイルし、空文字列と上限長を含む実行結果を確認してください。

学習のポイント

- ヌル終端文字列の概念を確実に理解
- バッファーサイズと文字列長の違いを意識
- ポインターと配列の関係を文字列で実践
- セキュアな文字列操作の重要性を認識

第11章:関数の演習問題

基礎問題

問題11-1:基本的な関数作成

以下の関数を作成してください。

1. **数学関数**
   - 2つの整数の最大値を返す関数 `int max(int a, int b)`
   - 3つの整数の最小値を返す関数 `int min(int a, int b, int c)`
   - 数値が偶数かどうかを判定する関数 `int is_even(int n)`
   - 数値が素数かどうかを判定する関数 `int is_prime(int n)`
2. **文字・文字列関数**
   - 文字が英字かどうかを判定する関数 `int is_letter(char c)`
   - 文字列の長さを計算する関数 `int my_strlen(const char str[])`
   - 文字列内の特定文字を数える関数 `int count_char(const char str[], char ch)`
3. **表示関数**
   - 指定した長さの線を描く関数 `void draw_line(int length, char ch)`
   - 数値を指定した桁数で表示する関数 `void print_number_padded(int num, int width)`

期待される出力例

```
=== 数学関数のテスト ===
max(10, 20) = 20
min(30, 15, 25) = 15
is_even(24) = 1 (偶数)
is_even(17) = 0 (奇数)
is_prime(17) = 1 (素数)
is_prime(24) = 0 (素数ではない)

=== 文字・文字列関数のテスト ===
is_letter('A') = 1
is_letter('5') = 0
my_strlen("Hello") = 5
count_char("Hello World", 'l') = 3

=== 表示関数のテスト ===
draw_line(20, '-'): --------------------
print_number_padded(42, 5): 00042
print_number_padded(1234, 3): 1234
```

問題11-2:配列操作関数

以下の配列操作関数を作成してください。

1. **基本操作**
   - 配列の要素の合計を計算する関数 `int array_sum(int arr[], int size)`
   - 配列の平均値を計算する関数 `double array_average(int arr[], int size)`
   - 配列内の最大値のインデックスを返す関数 `int find_max_index(int arr[], int size)`
2. **検索・ソート**
   - 線形検索関数 `int linear_search(int arr[], int size, int target)`
   - 配列を昇順にソートする関数 `void sort_array(int arr[], int size)`

- 配列の要素を逆順にする関数 `void reverse_array(int arr[], int size)`

3 **配列統計**

- 配列の最大値と最小値を同時に求める関数 `void find_min_max(int arr[], int size, int *min, int *max)`
- 配列内の重複する要素の数を数える関数 `int count_duplicates(int arr[], int size)`

期待される出力例

```
=== 配列操作関数のテスト ===
配列: [34, 12, 67, 89, 23, 45, 78, 56, 90, 11]

基本操作:
array_sum() = 495
array_average() = 49.50
find_max_index() = 8 (値: 90)

検索・ソート:
linear_search(45) = 5
ソート前: [34, 12, 67, 89, 23, 45, 78, 56, 90, 11]
ソート後: [11, 12, 23, 34, 45, 56, 67, 78, 89, 90]
逆順: [90, 89, 78, 67, 56, 45, 34, 23, 12, 11]

配列統計:
最小値: 11, 最大値: 90
重複要素数: 0

配列: [1, 2, 3, 2, 4, 3, 5]
重複要素数: 2 (2と3が重複)
```

問題11-3:文字列処理関数

以下の文字列処理関数を作成してください。

1 **基本処理**

- 文字列をコピーする関数 `void my_strcpy(char dest[], const char src[])`
- 文字列を連結する関数 `void my_strcat(char dest[], const char src[])`
- 文字列を比較する関数 `int my_strcmp(const char str1[], const char str2[])`

2 **変換処理**

- 文字列を大文字に変換する関数 `void to_uppercase(char str[])`
- 文字列を小文字に変換する関数 `void to_lowercase(char str[])`
- 文字列を逆順にする関数 `void reverse_string(char str[])`

3 **解析処理**

- 文字列内の単語数を数える関数 `int count_words(const char str[])`
- 文字列が回文かどうかを判定する関数 `int is_palindrome(const char str[])`
- 文字列内の母音の数を数える関数 `int count_vowels(const char str[])`

期待される出力例

```
=== 文字列処理関数のテスト ===

基本処理:
元の文字列: "Hello"
my_strcpy後: "Hello"
my_strcat後: "HelloWorld"
my_strcmp("Hello", "World") = -15

変換処理:
元: "Hello World"
大文字: "HELLO WORLD"
小文字: "hello world"
逆順: "dlroW olleH"

解析処理:
"Hello World Programming" の単語数: 3
"racecar" は回文: Yes
"hello" は回文: No
"Hello World" の母音数: 3 (e, o, o)
```

応用問題

問題11-4:複数戻り値を持つ関数

以下の関数を作成してください。

1 **時間計算**

- 秒数を時分秒に変換する関数 `void seconds_to_hms(int total_seconds, int *hours, int *minutes, int *seconds)`
- 日数から年月日に変換する関数(365日/年として) `void days_to_ymd(int total_days, int *years, int *months, int *days)`

2 **座標計算**

- 2点間の距離と中点を計算する関数 `void calculate_line_info(double x1, double y1, double x2, double y2, double *distance, double *mid_x, double *mid_y)`
- 円の面積と周囲の長さを計算する関数 `void calculate_circle_info(double radius, double *area, double *circumference)`

3 **統計計算**

- 配列の統計情報を計算する関数 `void calculate_statistics(int arr[], int size, int *min, int *max, double *mean, double *median)`
- 成績から評価を計算する関数 `void calculate_grade_info(int scores[], int count, double *average, char *grade, int *pass_count)`

期待される出力例

```
=== 複数戻り値関数のテスト ===

時間計算:
3661秒 = 1時間 1分 1秒
400日 = 1年 1月 5日

座標計算:
点(0, 0)と点(3, 4)の間:
距離: 5.00, 中点: (1.50, 2.00)

半径5の円:
面積: 78.54, 周囲: 31.42

統計計算:
配列: [34, 12, 67, 89, 23, 45, 78, 56, 90, 11]
最小: 11, 最大: 90, 平均: 50.50, 中央値: 50.50

成績情報:
点数: [85, 92, 78, 65, 98, 55, 73, 88]
平均: 79.25, 評価: B, 合格者数: 6
```

問題11-5:エラー処理付き関数

安全性を考慮した以下の関数を作成してください。

1 **安全な基本操作**

- 安全な除算関数 `int safe_divide(double a, double b, double *result)`
- 安全な配列アクセス関数 `int safe_array_get(int arr[], int size, int index, int *value)`
- 安全な文字列コピー関数 `int safe_strcpy(char dest[], int dest_size, const char src[])`

2 **範囲チェック付き関数**

- 範囲指定付きランダム数生成関数 `int random_range(int min, int max, int *result)`
- 配列の範囲チェック付き設定関数 `int safe_array_set(int arr[], int size, int index, int value)`

3 **バリデーション関数**

- 文字列が数値として有効かチェックする関数 `int is_valid_number(const char str[])`
- メールアドレスの基本形式をチェックする関数 `int is_valid_email(const char email[])`

期待される出力例

```
=== エラー処理関数のテスト ===

安全な除算:
safe_divide(10, 3) = 3.33 (成功)
safe_divide(10, 0) = エラー (ゼロ除算)

安全な配列アクセス:
safe_array_get(index=2) = 67 (成功)
safe_array_get(index=15) = エラー (範囲外)

安全な文字列コピー:
safe_strcpy("Hello") = 成功
safe_strcpy("VeryLongStringThatExceedsBufferSize") = エラー (バッファー不足)

バリデーション:
is_valid_number("123.45") = 1 (有効)
is_valid_number("12.3.4") = 0 (無効)
is_valid_email("user@example.com") = 1 (有効)
is_valid_email("invalid.email") = 0 (無効)
```

問題11-6:構造体を使った関数

以下の構造体を使った関数を作成してください。

```
typedef struct {
```

```
    double x;
    double y;
} Point;

typedef struct {
    int id;
    char name[50];
    int age;
    double score;
} Student;

typedef struct {
    int year;
    int month;
    int day;
} Date;
```

1. **座標操作**
   - 2点間の距離を計算する関数 `double point_distance(Point p1, Point p2)`
   - 点を移動する関数 `Point move_point(Point p, double dx, double dy)`
   - 複数の点の重心を計算する関数 `Point calculate_centroid(Point points[], int count)`
2. **学生情報処理**
   - 学生情報を表示する関数 `void print_student(Student s)`
   - 学生の成績を更新する関数 `void update_score(Student *s, double new_score)`
   - 学生配列から最高得点者を見つける関数 `Student find_best_student(Student students[], int count)`
3. **日付操作**
   - 日付を表示する関数 `void print_date(Date d)`
   - 日付の妥当性をチェックする関数 `int is_valid_date(Date d)`
   - 2つの日付の差を計算する関数 `int date_difference(Date d1, Date d2)`

期待される出力例

```
=== 構造体関数のテスト ===

座標操作:
点A(0, 0)と点B(3, 4)の距離: 5.00
点(2, 3)を(1, -1)移動: (3, 2)
3点の重心: (2.00, 3.00)

学生情報処理:
学生: ID=1001, 名前=田中太郎, 年齢=20, 成績=85.5
成績更新後: 92.0
最高得点者: 山田花子 (成績: 95.0)

日付操作:
日付: 2024/3/15
is_valid_date(2024/2/30) = 0 (無効)
is_valid_date(2024/3/15) = 1 (有効)
2024/1/1 から 2024/3/15 までの日数: 74日
```

チャレンジ問題

問題11-7:再帰関数

以下の再帰関数を作成してください。

1. **数学的再帰**
   - 階乗を計算する再帰関数 `long factorial_recursive(int n)`
   - フィボナッチ数列を計算する再帰関数 `int fibonacci_recursive(int n)`

- ユークリッドの互除法による最大公約数を求める再帰関数 `int gcd_recursive(int a, int b)`

2 **文字列再帰**

- 文字列が回文かどうか再帰的に判定する関数 `int is_palindrome_recursive(const char str[], int start, int end)`
- 文字列を再帰的に逆順にする関数 `void reverse_string_recursive(char str[], int start, int end)`

3 **配列再帰**

- 配列の合計を再帰的に計算する関数 `int array_sum_recursive(int arr[], int size)`
- 配列の最大値を再帰的に見つける関数 `int find_max_recursive(int arr[], int size)`

期待される出力例

```
=== 再帰関数のテスト ===

数学的再帰:
5! = 120
fibonacci(10) = 55
gcd(48, 18) = 6

文字列再帰:
"racecar" は回文: Yes
"hello" は回文: No
"Hello" の逆順: "olleH"

配列再帰:
配列 [1, 2, 3, 4, 5] の合計: 15
配列 [34, 67, 12, 89, 23] の最大値: 89
```

問題11-8:高度な文字列処理

以下の高度な文字列処理関数を作成してください。

1 **文字列解析**

- 文字列をトークンに分割する関数 `int tokenize(char str[], char tokens[][50], char delimiter)`
- 文字列内の括弧の対応をチェックする関数 `int check_brackets(const char str[])`
- 文字列から数値を抽出する関数 `int extract_numbers(const char str[], int numbers[], int max_count)`

2 **パターンマッチング**

- 簡単なワイルドカード(*、?)パターンマッチング関数 `int wildcard_match(const char str[], const char pattern[])`
- 文字列の置換関数 `int string_replace(char str[], const char old_substr[], const char new_substr[])`

3 **文字列フォーマット**

- 文字列を指定幅で中央揃えする関数 `void center_string(char result[], const char str[], int width)`
- CSV形式の文字列を解析する関数 `int parse_csv_line(const char line[], char fields[][100], int max_fields)`

問題11-9:ソートアルゴリズム関数

以下のソートアルゴリズムを関数として実装してください。

1 **基本ソート**

- バブルソート `void bubble_sort(int arr[], int size)`
- 選択ソート `void selection_sort(int arr[], int size)`
- 挿入ソート `void insertion_sort(int arr[], int size)`

2 **高速ソート**

- マージソート `void merge_sort(int arr[], int left, int right)`
- クイックソート `void quick_sort(int arr[], int left, int right)`

3 **特殊ソート**

- 文字列配列のソート `void sort_strings(char strings[][100], int count)`
- 構造体配列のソート(複数キー対応)`void sort_students(Student students[], int count, int sort_by)`

挑戦課題

課題11-10:関数ポインター

以下の関数ポインターを使った課題に取り組んでください。

1 **計算機関数**

- 四則演算を関数ポインターで切り替える計算機 `double calculator(double a, double b, double (*operation)(double, double))`
- 配列に対する処理を関数ポインターで指定する関数 `void process_array(int arr[], int size, void (*processor)(int*))`

2 **ソート関数の汎用化**

- 比較関数を引数に取る汎用ソート関数 `void generic_sort(void *arr, int size, int elem_size, int (*compare)(const void*, const void*))`

3 **コールバック関数**

- イベント処理システム `void register_callback(int event_type, void (*callback)(int))`

課題11-11:メモリ効率を考慮した関数

メモリ使用量を最適化した関数を作成してください。

1 **動的メモリ管理**

- 動的配列を管理する関数群
- 文字列プールを管理する関数群
- メモリリークを検出する関数

2 **効率的なアルゴリズム**

- インプレース(元の配列内で処理)ソート関数
- メモリ使用量を抑えた文字列処理関数
- キャッシュ効率を考慮した配列処理関数

課題11-12:総合プロジェクト

複数の関数を組み合わせた総合的なプロジェクトを作成してください。

1 **学生管理システム**

- 学生情報の登録、検索、更新、削除機能
- 成績統計の計算機能
- データのファイル保存・読み込み機能

2 **テキスト解析ツール**
- ファイルの読み込みと解析
- 単語頻度の統計
- 文字列パターンの検索

3 **数値計算ライブラリ**
- 行列演算機能
- 統計計算機能
- 数値積分・微分機能

提出物

- 関数を実装するCファイル、必要な宣言を置くヘッダーファイル、テスト用のmain関数を提出してください。
- 各関数の事前条件、戻り値、エラー時の動作を記載してください。
- 正常値、境界値、エラー値を含むテストを用意してください。

評価ポイント

- **正確性**:仕様通りに動作するか
- **安全性**:エラー処理が適切か
- **効率性**:アルゴリズムの計算量は適切か
- **可読性**:コードが理解しやすいか
- **再利用性**:他のプログラムでも使えるか

難易度別の推奨学習順序

初学者向け(基本課題から開始)

1 問題11-1:基本的な関数作成

2 問題11-2:配列操作関数

3 問題11-3:文字列処理関数

中級者向け(中級課題に挑戦)

1 問題11-4:複数戻り値を持つ関数

2 問題11-5:エラー処理付き関数

3 問題11-6:構造体を使った関数

上級者向け(上級・挑戦課題)

1 問題11-7:再帰関数

2 問題11-8:高度な文字列処理

3 問題11-9:ソートアルゴリズム関数

4. 課題11-10:関数ポインター
5. 課題11-11:メモリ効率を考慮した関数
6. 課題11-12:総合プロジェクト

学習のヒント

1. **段階的実装**
   - 最初は基本機能のみ実装
   - 動作確認後に機能を追加
   - テストを頻繁に実行
2. **エラー処理**
   - NULLポインターのチェック
   - 配列の境界チェック
   - 無効な引数の処理
3. **関数設計の原則**
   - 単一責任の原則
   - 適切な関数名と引数名
   - const修飾子の活用
4. **テスト駆動開発**
   - 関数作成前にテストケースを考える
   - 境界値のテスト
   - エラーケースのテスト
5. **コードレビュー**
   - 他の人に読んでもらう
   - 改善点を見つける
   - ベストプラクティスを学ぶ

参考資料

- C言語関数リファレンス
- アルゴリズムとデータ構造の教科書
- セキュアプログラミングガイド
- 関数型プログラミングの考え方

実装例の構成

各課題の実装では以下の構成を推奨します。

```
// ヘッダーファイル (functions.h)
// - 関数プロトタイプ
// - 定数定義
// - 構造体定義

// 実装ファイル (functions.c)
```

```
// - 関数の実装
// - 静的関数（内部使用）
// - エラー処理

// テストファイル（test.c）
// - main関数
// - 各関数のテスト
// - 結果の検証
```

発展的な学習内容

1. **可変長引数**
   - va_list、va_start、va_endの使用
   - printfライクな関数の作成
2. **関数ポインター**
   - コールバック関数の実装
   - 関数テーブルの作成
3. **マクロ関数**
   - 関数ライクマクロの作成
   - 条件付きコンパイル
4. **インライン関数**
   - パフォーマンスの最適化
   - 適切な使用場面

第12章：ビット操作の演習問題

演習問題の目的

この章では、ビット演算子の使い方、ビットマスクによるフラグ管理、ビットフィールドの活用、そして実践的なビット操作技術を習得します。

基礎問題

問題12-1：基本的なビット操作

以下の処理を行う関数を実装してください：

1. 整数の特定ビットをセット/クリア/反転する関数
2. 整数の特定範囲のビットを抽出する関数
3. 整数のビットを左右に循環シフトする関数

```
/* 実装すべき関数のプロトタイプ */
void set_bit(unsigned int *value, int bit);
void clear_bit(unsigned int *value, int bit);
void toggle_bit(unsigned int *value, int bit);
int test_bit(unsigned int value, int bit);

unsigned int extract_bits(unsigned int value, int start, int count);
unsigned int rotate_left(unsigned int value, int shift);
unsigned int rotate_right(unsigned int value, int shift);
```

要件

- ビット位置は0から31まで
- 範囲外のビット位置の場合は何もしない
- 循環シフトは32ビットの範囲で行う

期待される出力例

```
=== 基本的なビット操作 ===
元の値: 0xA5 (10100101)

ビット3をセット: 0xAD (10101101)
ビット5をクリア: 0x85 (10000101)
ビット1を反転: 0xA7 (10100111)
ビット6のテスト: 1 (セットされている)

ビット2から4ビット抽出: 0x9 (1001)
左に3ビット循環シフト: 0x2D4 (1010110100)
右に2ビット循環シフト: 0x40000029 (01000000000000000000000000101001)
```

問題12-2：フラグ管理システム

ゲームキャラクターの状態を管理するフラグシステムを実装してください：

```
/* 状態フラグの定義 */
#define STATUS_ALIVE     0x0001
#define STATUS_POISONED  0x0002
#define STATUS_PARALYZED 0x0004
#define STATUS_CONFUSED  0x0008
#define STATUS_SLEEPING  0x0010
#define STATUS_BURNING   0x0020
#define STATUS_FROZEN    0x0040
#define STATUS_INVISIBLE 0x0080
#define STATUS_INVINCIBLE 0x0100
#define STATUS_BUFFED    0x0200
```

実装する機能：

1. 複数の状態を同時に設定/解除

2. 特定の状態の組み合わせをチェック
3. 状態の一覧を表示
4. 相反する状態の自動解除(例:凍結と燃焼)

期待される出力例

```
=== キャラクター状態管理 ===
初期状態: ALIVE

状態を追加: POISONED, CONFUSED
現在の状態: ALIVE | POISONED | CONFUSED

BURNINGを追加...
FROZENが自動的に解除されました
現在の状態: ALIVE | POISONED | CONFUSED | BURNING

状態チェック:

- 生存している: はい
- 毒と混乱の両方: はい
- 無敵状態: いいえ

全状態一覧:
[x] ALIVE
[x] POISONED
[ ] PARALYZED
[x] CONFUSED
[ ] SLEEPING
[x] BURNING
[ ] FROZEN
[ ] INVISIBLE
[ ] INVINCIBLE
[ ] BUFFED
```

応用問題

問題12-3:ビットフィールドでメモリ節約

学生の成績データを効率的に格納する構造体を設計してください:

```
/* 成績データ構造体(ビットフィールド使用) */
struct StudentGrade {
    unsigned int student_id : 20;    /* 学生ID (0-1048575) */
    unsigned int subject_code : 10;  /* 科目コード (0-1023) */
    unsigned int grade : 7;          /* 成績 (0-100) */
    unsigned int semester : 2;       /* 学期 (1-4) */
    unsigned int year : 7;           /* 年度 (0-99, 2000年基準) */
    unsigned int attendance : 1;     /* 出席状況 (合格/不合格) */
    unsigned int retake : 1;         /* 再履修フラグ */
};
```

実装する機能:

1. 成績データの入力と表示
2. 通常の構造体とのサイズ比較
3. 成績の統計情報(平均点、最高点、最低点)
4. 条件に合う学生の検索

期待される出力例

```
=== 成績データ管理 ===
構造体サイズ比較:

- ビットフィールド版: 8 バイト
- 通常版: 32 バイト
- 節約率: 75.0%

成績データ入力:
学生ID: 12345
科目コード: 101
成績: 85
学期: 2
年度: 24 (2024年)
出席: 合格
再履修: いいえ

登録された成績一覧:
ID      科目   成績  学期  年度  出席  再履修
12345   101    85    2    2024   yes   no
12346   101    92    2    2024   yes   no
12347   101    68    2    2024   no    yes

統計情報:
平均点: 81.7
最高点: 92 (学生ID: 12346)
最低点: 68 (学生ID: 12347)
```

問題12-4:RGB色操作

RGB565形式(16ビット)の色を操作するプログラムを作成してください:

```
/* RGB565色構造体 */
typedef struct {
    unsigned int blue  : 5;
    unsigned int green : 6;
    unsigned int red   : 5;
} RGB565;

/* または16ビット整数として扱う */
typedef unsigned short RGB565_int;
```

実装する機能:

1. RGB888(24ビット)⇔ RGB565の相互変換
2. 色の明度調整(明るく/暗く)
3. グレースケール変換
4. 2つの色の混合(アルファブレンド風)

期待される出力例

```
=== RGB565色操作 ===
RGB888 (255, 128, 64) → RGB565: 0xFC20
RGB565: R=31, G=32, B=0

RGB565 0xFC20 → RGB888: (248, 129, 0)
 (精度損失あり)

明度調整:
元の色: 0x7BEF (R=15, G=31, B=15)
50%明るく: 0xBDF7 (R=23, G=47, B=23)
50%暗く: 0x39E7 (R=7, G=15, B=7)

グレースケール変換:
カラー: 0xF800 (赤) → グレー: 0x7BEF

色の混合:
色1: 0xF800 (赤)
色2: 0x001F (青)
混合(50:50): 0x780F (紫)
```

問題12-5：ビットベクタで集合演算

ビットベクタを使って集合演算を実装してください：

```
#define SET_SIZE 256  /* 0-255の整数を扱う */

typedef struct {
    unsigned char bits[SET_SIZE / 8];
} BitSet;
```

実装する機能：

1. 集合の初期化、要素の追加/削除
2. 和集合、積集合、差集合、対称差
3. 部分集合の判定
4. 集合の要素数カウント
5. 集合の内容表示

期待される出力例

```
=== ビットベクタ集合演算 ===
集合A: {1, 3, 5, 7, 9}
集合B: {2, 3, 5, 8, 9}

和集合 (A □ B): {1, 2, 3, 5, 7, 8, 9}
積集合 (A □ B): {3, 5, 9}
差集合 (A - B): {1, 7}
対称差 (A □ B): {1, 2, 7, 8}

集合C: {3, 5}
C □ A: true (CはAの部分集合)
C □ B: true (CはBの部分集合)

要素数:
|A| = 5
|B| = 5
|A □ B| = 7
|A □ B| = 3
```

チャレンジ問題

問題12-6：CRC計算の実装

簡単なCRC-8チェックサムを計算する関数を実装してください：

```
/* CRC-8計算 (多項式: x^8 + x^2 + x + 1)  */
unsigned char calculate_crc8(const unsigned char *data, int length);
```

実装内容：

1. ビット単位でCRCを計算する基本実装
2. テーブルを使った高速化実装
3. データの破損検出テスト
4. 性能比較

期待される出力例

```
=== CRC-8計算 ===
データ: "Hello"
CRC-8 (ビット単位): 0x1A
CRC-8 (テーブル): 0x1A

破損検出テスト:
元データ: "Hello" (CRC: 0x1A)
破損データ: "Hallo" (CRC: 0x4D)
破損が検出されました。

性能比較 (10000回実行):
ビット単位: 125ms
テーブル方式: 15ms
高速化: 8.3倍
```

問題12-7:ビットボードでオセロ

8×8のオセロ盤面をビットボードで表現してください:

```
typedef struct {
    unsigned long long black;  /* 黒石の位置 */
    unsigned long long white;  /* 白石の位置 */
} OthelloBoard;
```

実装する機能:

1. 盤面の初期化と表示
2. 石を置ける位置の判定
3. 石を置いた時の反転処理
4. 勝敗判定

期待される出力例

```
=== オセロゲーム ===
  A B C D E F G H
1 . . . . . . . .
2 . . . . . . . .
3 . . . * . . . .
4 . . * O X . . .
5 . . . X O * . .
6 . . . . * . . .
7 . . . . . . . .
8 . . . . . . . .

黒番です
置ける場所(*): D3, C4, F5, E6

D3に置きます...

  A B C D E F G H
1 . . . . . . . .
2 . . . . . . . .
3 . . . X . . . .
4 . . * X X . . .
5 . . . X O * . .
6 . . . . * . . .
7 . . . . . . . .
8 . . . . . . . .

石の数: 黒=4, 白=1
```

問題12-8:ハフマン符号化(上級)

簡単なハフマン符号化を実装してください:

1. 文字の出現頻度をカウント
2. ハフマン木を構築

3　各文字のビット列を決定

4　テキストをビット列に圧縮

5　ビット列から元のテキストに復元

問題12-9：ブルームフィルター（上級）

確率的データ構造であるブルームフィルタを実装してください：

1　複数のハッシュ関数の実装

2　要素の追加

3　要素の存在確認（偽陽性あり）

4　偽陽性率の測定

演習のヒント

ビット操作の基本パターン

```
/* nビット目をセット */
value |= (1 << n);

/* nビット目をクリア */
value &= ~(1 << n);

/* nビット目を反転 */
value ^= (1 << n);

/* nビット目をテスト */
if (value & (1 << n)) { /* ... */ }
```

ビットマスクの作成

```
/* 下位nビットのマスク */
mask = (1 << n) - 1;

/* startからcountビット分のマスク */
mask = ((1 << count) - 1) << start;
```

ビットカウント

```
/* Brian Kernighanのアルゴリズム */
int count = 0;
while (n) {
    n &= n - 1;
    count++;
}
```

提出物

- 問題ごとにex12_1.c、ex12_2.cのような名前でCファイルを作成してください。
- 各ファイルに動作確認用のmain関数を含めてください。
- ビットマスクの設計と境界条件をコメントで説明してください。

評価基準

- **正確性**：要求された機能が正しく動作するか
- **効率性**：ビット操作の利点を活かした実装か
- **可読性**：コードが理解しやすく整理されているか
- **完成度**：エラー処理やエッジケースへの対応

第13章：構造体の演習問題

演習の目的

- 構造体の定義と使い方を理解する
- 構造体のメンバーアクセス（ドット演算子とアロー演算子）を習得する
- 構造体配列と構造体ポインターの操作を身につける
- ネストした構造体の設計と実装を経験する

基礎問題

問題13-1：学生情報管理

学生の情報（ID、名前、年齢、成績）を格納する構造体を定義し、ポインターを使って情報を表示・更新するプログラムを作成してください。

要件

- 学生情報を表す構造体を定義
- 学生情報を入力する関数を作成
- 学生情報を表示する関数を作成（ポインター引数）
- 成績を更新する関数を作成（ポインター引数）

実装のヒント

- 構造体ポインターのメンバーアクセスには->演算子を使用
- 文字列のコピーにはstrcpy()を使用

期待される出力例

```
=== 学生情報管理システム ===
学生情報を入力してください:
ID: 1001
名前: 山田太郎
年齢: 20
成績: 85.5

入力された学生情報:
ID: 1001
名前: 山田太郎
年齢: 20歳
成績: 85.50点

成績を更新します...
新しい成績: 92.0

更新後の学生情報:
ID: 1001
名前: 山田太郎
年齢: 20歳
成績: 92.00点
```

問題13-2：座標計算

2D座標を表す構造体を定義し、2点間の距離を計算する関数をポインターを使って実装してください。

要件

- 座標を表す構造体（x, y）を定義
- 2点間の距離を計算する関数を作成（ポインター引数）

- 座標を移動する関数を作成(ポインターで更新)
- math.hのsqrt関数を使用

実装のヒント

- 距離の公式: sqrt((x2-x1)^2 + (y2-y1)^2)
- コンパイル時に-lmオプションが必要

期待される出力例

```
=== 座標計算プログラム ===
点A の座標:
x: 0
y: 0

点B の座標:
x: 3
y: 4

点A(0.00, 0.00) と 点B(3.00, 4.00) の距離: 5.00

点Aを移動します:
x方向の移動量: 2
y方向の移動量: 1

移動後の点A: (2.00, 1.00)
新しい距離: 2.24
```

問題13-3:商品管理

商品情報(コード、名前、価格、在庫)の構造体を作成し、構造体配列で複数商品を管理するプログラムを作成してください。

要件

- 商品情報を表す構造体を定義
- 最大10個の商品を管理できる配列を用意
- 商品を追加する関数を作成
- 全商品を表示する関数を作成
- 在庫を更新する関数を作成(商品コードで検索)

実装のヒント

- 構造体配列の初期化に注意
- 商品数のカウンタを別途管理

期待される出力例

```
=== 商品管理システム ===

1. 商品追加
2. 商品一覧表示
3. 在庫更新
4. 終了

選択: 1
商品コード: P001
商品名: ノート
価格: 150
在庫数: 50
商品を追加しました。

選択: 1
商品コード: P002
商品名: ペン
価格: 100
在庫数: 100
商品を追加しました。

選択: 2
--- 商品一覧 ---
コード: P001
商品名: ノート
価格: 150円
在庫: 50個

コード: P002
商品名: ペン
価格: 100円
在庫: 100個

選択: 3
更新する商品コード: P001
新しい在庫数: 45
在庫を更新しました。
```

応用問題

問題13-4:従業員データベース

従業員情報と部署情報をネストした構造体で管理し、部署別の給与統計を算出するプログラムを作成してください。

要件

- 部署情報を表す構造体(部署名、部署コード)
- 従業員情報を表す構造体(ID、名前、部署情報、給与)
- 部署別の平均給与を計算する関数
- 最高給与の従業員を検索する関数
- 構造体ポインター配列を使用

実装のヒント

- ネストした構造体のアクセス方法を理解
- 部署コードで従業員をグループ化

期待される出力例

```
=== 従業員データベース ===
従業員データ:
ID: 1001, 名前: 田中太郎
部署: 営業部 (コード: SALES)
給与: 350000円

ID: 1002, 名前: 鈴木花子
部署: 営業部 (コード: SALES)
給与: 320000円

ID: 1003, 名前: 佐藤次郎
部署: 開発部 (コード: DEV)
給与: 450000円

=== 部署別統計 ===
営業部 (SALES):

- 人数: 2名
- 平均給与: 335000円

開発部 (DEV):

- 人数: 1名
- 平均給与: 450000円

最高給与の従業員:
佐藤次郎 (開発部) - 450000円
```

問題13-5:図書管理システム

本の情報(タイトル、著者、出版年、貸出状況)を管理し、検索・貸出・返却機能を実装してください。

要件

- 書籍情報を表す構造体を定義
- 貸出状況を管理(貸出中フラグ、借りた人のID)
- タイトルで検索する関数
- 貸出処理を行う関数
- 返却処理を行う関数
- 貸出中の本一覧を表示する関数

実装のヒント

- 部分一致検索にはstrstr()を使用
- 貸出状況は構造体内のフラグで管理

期待される出力例

```
=== 図書管理システム ===

1. 図書検索
2. 貸出処理
3. 返却処理
4. 貸出中一覧
5. 終了

選択: 1
検索するタイトル: プログラミング

検索結果:

1. 「C言語プログラミング入門」 - 山田太郎 (2023年)

   状態: 貸出可能

2. 「プログラミング言語C」 - カーニハン&リッチー (1988年)

   状態: 貸出中 (借主ID: U001)

選択: 2
貸出する本の番号: 1
借りる人のID: U002
「C言語プログラミング入門」を貸出しました。

選択: 4
--- 貸出中の本一覧 ---

1. 「プログラミング言語C」

   借主ID: U001

2. 「C言語プログラミング入門」

   借主ID: U002
```

チャレンジ問題

問題13-6:成績管理システム

学生と科目の構造体を使って、学生別・科目別の成績統計を管理するプログラムを作成してください。

要件

- 科目情報を表す構造体(科目名、科目コード、単位数)
- 成績情報を表す構造体(学生ID、科目コード、点数、評価)
- 学生別の平均点を計算する関数
- 科目別の平均点を計算する関数
- GPA計算機能(A=4.0, B=3.0, C=2.0, D=1.0, F=0.0)

追加要件

- CSVファイルからのデータ読み込み機能
- 成績分布のヒストグラム表示

期待される出力例

```
=== 成績管理システム ===
データ読み込み完了: 15件

学生別統計:
学生ID 1001:

- 受講科目数: 3
- 平均点: 83.3
- GPA: 3.33

科目別統計:
プログラミング基礎 (CS101):

- 受講者数: 5
- 平均点: 78.4
- 成績分布:

  A (90-100): ** (2名)
  B (80-89): * (1名)
  C (70-79): * (1名)
  D (60-69): * (1名)
  F (0-59): - (0名)

全体GPA順位:

1. 学生ID 1003: GPA 3.80
2. 学生ID 1001: GPA 3.33
3. 学生ID 1002: GPA 2.95
```

問題13-7:連結リストの実装

構造体を使って単方向連結リストを実装してください。

要件

- ノード構造体(データと次のノードへのポインター)
- リストの先頭に要素を追加
- リストの末尾に要素を追加
- 特定の値を持つノードを削除
- リスト全体を表示
- メモリリークを防ぐ適切な解放処理

実装のヒント

- 自己参照構造体の定義方法を理解
- malloc/freeを使った動的メモリ管理

期待される出力例

```
=== 連結リスト操作 ===

1. 先頭に追加
2. 末尾に追加
3. 削除
4. 表示
5. 終了

選択: 1
値: 10
先頭に10を追加しました。

選択: 2
値: 20
末尾に20を追加しました。

選択: 1
値: 5
先頭に5を追加しました。

選択: 4
リスト: 5 -> 10 -> 20 -> NULL

選択: 3
削除する値: 10
10を削除しました。

選択: 4
リスト: 5 -> 20 -> NULL

選択: 5
メモリを解放しています...
終了しました。
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- 構造体のメンバー構成と関数へ渡す方法をコメントで説明してください。
- 動的メモリを使用する場合は、確保と解放を対応させてください。

学習のポイント

- 構造体は関連するデータをまとめる強力な機能
- ポインター経由のアクセスでは->演算子を使用
- 大きな構造体や変更対象の構造体は、必要に応じてポインターで渡す
- ネストした構造体で複雑なデータ構造を表現可能

第14章：関数ポインターの演習問題

演習の目的

- 関数ポインターの基本概念を理解する
- コールバック関数の実装方法を習得する
- 関数ポインター配列を活用した動的関数選択を学ぶ
- 実行時関数切り替えシステムの設計を理解する

基礎問題

問題14-1：関数ポインターの基本操作

関数ポインターを使って複数の関数を動的に呼び出すプログラムを作成してください。

要件

- 2つの整数を受け取り、結果を返す関数を複数定義する
- 関数ポインターを使ってこれらの関数を呼び出す
- 関数ポインター配列を使った実装も含める

実装すべき関数

- `int maximum(int a, int b)` - 大きい方の値を返す
- `int minimum(int a, int b)` - 小さい方の値を返す
- `int power(int a, int b)` - aのb乗を返す（簡単な実装で可）

期待される出力例

```
=== 関数ポインターの基本操作 ===
a = 10, b = 5

関数ポインターで直接呼び出し:
maximum(10, 5) = 10
minimum(10, 5) = 5
power(10, 5) = 100000

関数ポインター配列で呼び出し:
操作0: maximum(10, 5) = 10
操作1: minimum(10, 5) = 5
操作2: power(10, 5) = 100000

関数アドレス表示:
maximum: 0x400650
minimum: 0x400670
power: 0x400690
```

ファイル名： `ex14_1_basic_function_pointer.c`

問題14-2：関数選択システム

文字に基づいて関数を選択し実行するシステムを実装してください。

要件

- 文字（'+'，'-'，'*'，'/' など）に基づいて関数を選択
- 選択された関数を実行して結果を表示
- 無効な文字が指定された場合のエラー処理
- 構造体を使った関数ポインター管理

期待される出力例

```
=== 関数選択システム ===
利用可能な演算: + - * / % ^

第1の数: 15
第2の数: 3
演算子: +
結果: 15 + 3 = 18

第1の数: 20
第2の数: 4
演算子: /
結果: 20 / 4 = 5

第1の数: 10
第2の数: 0
演算子: /
エラー: ゼロ除算はできません

第1の数: 8
第2の数: 3
演算子: @
エラー: 無効な演算子です
```

ファイル名: `ex14_2_function_selector.c`

応用問題

問題14-3:配列処理のコールバック

コールバック関数を使って配列の各要素に異なる処理を適用するプログラムを作成してください。

要件

- 整数配列を処理する関数を複数定義
- コールバック関数として配列処理関数に渡す
- 処理前後の配列の状態を表示
- 動的な処理選択機能

実装すべき処理

- 各要素を倍にする
- 各要素から1を引く
- 各要素の符号を反転する

期待される出力例

```
=== 配列処理のコールバック ===
元の配列: [1, 2, 3, 4, 5]

処理選択:

1. 各要素を倍にする
2. 各要素から1を引く
3. 各要素の符号を反転

選択: 1

処理後: [2, 4, 6, 8, 10]

別の配列: [5, -3, 8, -2, 0]
処理選択: 3

処理後: [-5, 3, -8, 2, 0]

複数処理の連続実行:
初期配列: [10, 20, 30]
 倍にする: [20, 40, 60]
 1を引く: [19, 39, 59]
最終結果: [19, 39, 59]
```

ファイル名: `ex14_3_array_callback.c`

問題14-4:関数ポインター配列を使った計算機

関数ポインター配列を使用した計算機プログラムを作成してください。

要件

- 関数ポインターの配列を定義
- インデックスを指定して演算を選択
- 複数の演算を連続で実行可能
- 演算履歴を表示する機能
- 統計情報の収集と表示

実装すべき演算

- 加算、減算、乗算、除算
- ゼロ除算のエラー処理

期待される出力例

```
=== 関数ポインター配列計算機 ===
演算メニュー:
0: 加算 (+)
1: 減算 (-)
2: 乗算 (*)
3: 除算 (/)
4: 終了

第1の数: 100
第2の数: 25
演算番号: 0
結果: 100 + 25 = 125

演算番号: 3
結果: 100 / 25 = 4

=== 演算履歴 ===

1. 100 + 25 = 125
2. 100 / 25 = 4

=== 統計情報 ===
総演算回数: 2

- 加算: 1回 (50.0%)
- 減算: 0回 (0.0%)
- 乗算: 0回 (0.0%)
- 除算: 1回 (50.0%)
```

ファイル名: `ex14_4_calculator_function_array.c`

チャレンジ問題

問題14-5:ソートアルゴリズム選択システム

異なるソートアルゴリズムを関数ポインターで切り替えるシステムを作成してください。

要件

- バブルソート、選択ソート、挿入ソートを実装
- 比較関数も関数ポインターで指定(昇順/降順)
- ソート前後の配列状態を表示
- アルゴリズムの性能比較機能

期待される出力例

```
=== ソートアルゴリズム選択システム ===
配列: [34, 7, 23, 32, 5, 62]

ソートアルゴリズム:

1. バブルソート
2. 選択ソート
3. 挿入ソート

選択: 1

順序:

1. 昇順
2. 降順

選択: 1

バブルソート (昇順) 実行中...
結果: [5, 7, 23, 32, 34, 62]
比較回数: 15, 交換回数: 8

=== 性能比較 (配列サイズ: 100) ===
アルゴリズム    比較回数   交換回数   時間(ms)
バブルソート     4950       2475       0.125
選択ソート       4950       99         0.098
挿入ソート       2525       2426       0.087
```

ファイル名: `ex14_5_sort_algorithms.c`

問題14-6:イベント駆動システム

コールバック関数を使ったイベント駆動システムを実装してください。

要件

- 複数種類のイベント(開始、停止、エラー、警告)
- イベントタイプごとに異なるハンドラーを登録
- イベント発生時に適切なハンドラーを呼び出し
- イベントの優先度機能

期待される出力例

```
=== イベント駆動システム ===
イベントハンドラーを登録中...

イベント発生シミュレーション:

[開始] システムが起動しました
[警告] メモリ使用率が80%を超えました
[エラー] ファイルが見つかりません: config.txt
[停止] システムをシャットダウンします

優先度付きイベント処理:
優先度3: [エラー] 重大なエラーが発生
優先度1: [警告] 軽微な警告
優先度2: [開始] サービス開始

=== イベント統計 ===
開始イベント: 2回
停止イベント: 1回
エラーイベント: 2回
警告イベント: 2回
総イベント数: 7回
```

ファイル名: `ex14_6_event_system.c`

提出物

- 問題ごとにC90版とC99版を作成し、C99版には_c99.cを付けてください。
- ファイルの冒頭に問題番号と関数ポインターの型をコメントで記載してください。
- それぞれの規格を指定し、警告を有効にしてコンパイルしてください。

コンパイル例

C90準拠

```
gcc -std=c90 -Wall -Wextra -pedantic ex14_1_basic_function_pointer.c -o ex14_1_basic_function_pointer
```

C99準拠

```
gcc -std=c99 -Wall -Wextra -pedantic ex14_1_basic_function_pointer_c99.c -o ex14_1_basic_function_pointer_c99
```

学習のポイント

関数ポインターの基本

1. **宣言**:`int (*func_ptr)(int, int);`
2. **初期化**:`func_ptr = function_name;`
3. **呼び出し**:`result = func_ptr(a, b);`
4. **配列**:`int (*operations[])(int, int) = {add, sub, mul};`

コールバック関数

1. **概念**:関数を引数として渡す仕組み
2. **活用**:動的な処理選択、カスタマイズ可能な処理
3. **設計**:関数ポインターを引数に取る関数の実装

実用的なテクニック

1. **関数テーブル**:構造体による関数ポインター管理
2. **エラー処理**:無効な関数ポインターの検出
3. **状態管理**:関数ポインターによる状態遷移
4. **性能**:関数ポインター使用時のオーバーヘッド考慮

注意事項

- 関数ポインターは初期化前に使用しない
- 型の一致を厳密に確認する
- NULLポインターチェックを忘れずに
- デバッグ時は関数名の確認が困難な場合がある

実用的な応用

これらの技術は以下のような場面で使用されます。

- **GUIフレームワーク**:イベントハンドラ
- **組み込みシステム**:割り込みハンドラ
- **ゲーム開発**:状態管理、AI行動パターン

- **システムプログラミング**：プラグインシステム

第15章：分割コンパイルと発展技術の演習問題

演習の目的

- プリプロセッサの高度な使い方を習得する
- メモリ管理の最適化技術を学ぶ
- キャッシュ効率を考慮したプログラミングを理解する
- システムプログラミングの基礎を身につける

基礎問題

問題15-1：プリプロセッサマクロの基本

以下の仕様を満たすプログラムを作成してください。

要件

- 数学的な定数をマクロで定義する（PI、E、黄金比など）
- 基本的な計算マクロを定義する（面積、体積など）
- 条件付きコンパイルでデバッグ機能を切り替える
- 文字列化マクロと連結マクロを活用する

実装すべき機能

- 円の面積・周長計算
- 球の体積・表面積計算
- デバッグ情報の出力（条件付き）
- マクロによる型安全な最大値・最小値関数

期待される出力例

```
=== プリプロセッサマクロの基本 ===
円の計算（半径 = 5.0）：

- 面積: 78.54
- 周長: 31.42

球の計算（半径 = 3.0）：

- 体積: 113.10
- 表面積: 113.10

[DEBUG] 計算完了: 円の面積 = 78.54（半径: 5.0）
[DEBUG] 計算完了: 球の体積 = 113.10（半径: 3.0）

型安全な最大値・最小値:
MAX(10, 20) = 20
MIN(3.14, 2.71) = 2.71

黄金比を使った計算:
黄金長方形（短辺 = 10）: 長辺 = 16.18
```

ファイル名： `ex15_1_macro_basics.c`

問題15-2：安全なメモリ操作マクロ

メモリ操作の安全性を向上させるマクロ群を作成してください。

要件

- NULLチェック付きメモリ割り当て

- 配列境界チェック付きアクセス
- 自動的なメモリ解放
- メモリリーク検出機能

実装すべきマクロ

- `SAFE_MALLOC(size)` - NULLチェック付きmalloc
- `SAFE_FREE(ptr)` - NULLクリア付きfree
- `ARRAY_BOUNDS_CHECK(arr, index, size)` - 境界チェック
- `MEMORY_LEAK_TRACKER` - 簡単なリーク検出

期待される出力例

```
=== 安全なメモリ操作マクロ ===

通常のメモリ割り当て:
100バイトを割り当て: 成功 (0x7fff5fbff8c0)
解放完了 (ポインターはNULLにリセット)

メモリ不足シミュレーション:
SAFE_MALLOC失敗: メモリ割り当てエラー (行: 45)

配列境界チェック:
配列[0-9]へのアクセステスト:
index 5: OK (値: 50)
index 10: エラー - 境界外アクセス検出。 (行: 67)

=== メモリリーク検出レポート ===
総割り当て: 1024 バイト
総解放: 768 バイト
リーク検出: 256 バイト (1ブロック)

  - ブロック#3: 256バイト (行: 89で割り当て)
```

ファイル名: `ex15_2_safe_memory.c`

問題15-3:基本的なメモリプール

固定サイズオブジェクト用の簡単なメモリプールを実装してください。

要件

- 事前に確保されたメモリ領域からオブジェクトを割り当て
- フリーリストによる高速な割り当て・解放
- プールの使用状況を表示する機能
- 初期化・終了処理

実装すべき機能

- プールの初期化と終了処理
- オブジェクトの取得と返却
- 使用状況の表示
- エラー処理

期待される出力例

```
=== 基本的なメモリプール ===
プール初期化: 100個のオブジェクト（各64バイト）

10個のオブジェクトを割り当て...
割り当て成功: 10個

=== プール使用状況 ===
総容量: 100 オブジェクト
使用中: 10 オブジェクト (10.0%)
空き: 90 オブジェクト (90.0%)
総メモリ: 6400 バイト

5個のオブジェクトを返却...

=== 更新後の使用状況 ===
使用中: 5 オブジェクト (5.0%)
空き: 95 オブジェクト (95.0%)

パフォーマンステスト（1000回の割り当て/解放）:

- メモリプール: 0.15ms
- 通常のmalloc/free: 2.34ms
- 高速化: 15.6倍
```

ファイル名: `ex15_3_memory_pool.c`

応用問題

問題15-4:汎用的なプリプロセッサライブラリ

さまざまな用途に使える汎用的なマクロライブラリを作成してください。

要件

- 型判定マクロ(コンパイル時)
- 汎用的なスワップマクロ
- ループ展開マクロ
- アサーション機能付きマクロ
- ベンチマーク計測マクロ

実装すべき機能

- `TYPE_CHECK(a, b)` - 型の一致確認
- `GENERIC_SWAP(a, b)` - 任意の型のスワップ
- `REPEAT(n, code)` - コードの繰り返し展開
- `BENCHMARK_BLOCK(name)` - ブロックの実行時間測定
- `STATIC_ASSERT(condition, message)` - コンパイル時アサーション

期待される出力例

```
=== 汎用的なプリプロセッサライブラリ ===

型チェック:
TYPE_CHECK(int, int): OK
TYPE_CHECK(int, double): 型不一致 (コンパイル時警告)

汎用スワップ:
整数: a=10, b=20 → a=20, b=10
文字列: s1="Hello", s2="World" → s1="World", s2="Hello"
構造体: p1=(1,2), p2=(3,4) → p1=(3,4), p2=(1,2)

ループ展開 (5回) :
展開0: i=0, value=0
展開1: i=1, value=1
展開2: i=2, value=4
展開3: i=3, value=9
展開4: i=4, value=16

=== ベンチマーク結果 ===
[BENCHMARK] 配列初期化: 0.234ms
[BENCHMARK] 行列演算: 1.567ms
[BENCHMARK] ソート処理: 3.890ms

スタティックアサート:
コンパイル時チェック: すべてOK
```

ファイル名: `ex15_4_generic_macros.c`

問題15-5:高性能メモリアロケーター

パフォーマンスを重視したカスタムメモリアロケーターを実装してください。

要件

- サイズ別メモリプール(小・中・大オブジェクト)
- メモリの断片化を最小限に抑える仕組み
- アロケーション統計情報の収集
- スレッドセーフ対応(簡易版)

実装すべき機能

- 複数サイズのメモリプール管理
- First Fit / Best Fitアルゴリズム
- メモリ使用統計とレポート機能
- デバッグモードでの詳細トレース

期待される出力例

```
=== 高性能メモリアロケーター ===

アロケーターを初期化中...

- 小オブジェクトプール: 0-64 バイト
- 中オブジェクトプール: 65-512 バイト
- 大オブジェクトプール: 513+ バイト

割り当てテスト:
32バイト割り当て: 成功 (小プールから)
256バイト割り当て: 成功 (中プールから)
1024バイト割り当て: 成功 (大プールから)

=== メモリ使用統計 ===
小オブジェクトプール:
  割り当て回数: 150
  解放回数: 120
  現在使用中: 30 (1920 バイト)
  断片化率: 5.2%

中オブジェクトプール:
  割り当て回数: 50
  解放回数: 45
  現在使用中: 5 (1280 バイト)
  断片化率: 8.7%

大オブジェクトプール:
  割り当て回数: 10
  解放回数: 8
  現在使用中: 2 (2048 バイト)
  断片化率: 12.3%

総メモリ使用量: 5248 バイト
平均割り当て時間: 0.0034ms
```

ファイル名: `ex15_5_allocator.c`

チャレンジ問題

問題15-6:キャッシュフレンドリーなデータ構造

CPUキャッシュ効率を考慮したデータ構造を実装してください。

要件

- 配列ベースの動的配列(vector風)
- キャッシュラインを意識したメモリレイアウト
- プリフェッチを活用した高速アクセス
- メモリプールとの連携

実装すべき機能

- 動的な要素追加・削除
- キャッシュ効率的な反復処理
- バルク操作(一括挿入・削除)
- パフォーマンス測定機能

期待される出力例

```
=== キャッシュフレンドリーなデータ構造 ===

ベクター初期化（初期容量: 16）
キャッシュライン境界でアラインメント済み

要素追加テスト:
1000要素を追加中...
容量自動拡張: 16 → 32 → 64 → 128 → 256 → 512 → 1024

=== パフォーマンス比較 ===
順次アクセス（キャッシュ効率的）:

- キャッシュベクター: 0.089ms
- 通常の配列: 0.156ms
- 高速化: 1.75倍

ランダムアクセス:

- キャッシュベクター: 0.234ms
- 通常の配列: 0.312ms
- 高速化: 1.33倍

バルク操作（100要素一括挿入）:

- キャッシュベクター: 0.012ms
- 個別挿入: 0.089ms
- 高速化: 7.42倍

=== キャッシュ統計 ===
L1キャッシュヒット率: 94.3%
L2キャッシュヒット率: 98.7%
プリフェッチ効果: 23.4%の改善
```

ファイル名: `ex15_6_cache_vector.c`

問題15-7:プリプロセッサベースのDSL

プリプロセッサを使ってドメイン固有言語(DSL)を作成してください。

要件

- 状態機械を記述するDSL
- イベント駆動システムの記述
- 自動的なコード生成
- コンパイル時検証機能

実装例

```
STATE_MACHINE(TrafficLight)
    STATE(Red)    TIMEOUT() -> Yellow
    STATE(Yellow) TIMEOUT() -> Green
    STATE(Green)  TIMEOUT() -> Red
END_STATE_MACHINE
```

期待される出力例

```
=== プリプロセッサベースのDSL ===

信号機状態機械の定義:
状態数: 3
遷移数: 3

状態機械を実行中...
[00:00] 状態: Red
[00:05] タイムアウト → Yellow へ遷移
[00:06] 状態: Yellow
[00:08] タイムアウト → Green へ遷移
[00:09] 状態: Green
[00:14] タイムアウト → Red へ遷移
[00:15] 状態: Red

=== コード生成結果 ===
生成された関数:

- void TrafficLight_init()
- void TrafficLight_process_event(Event)
- State TrafficLight_get_state()
- const char* TrafficLight_state_name(State)

生成されたコード行数: 87行
マクロ展開前: 12行
圧縮率: 7.25倍

=== 検証結果 ===
すべての状態から遷移可能: PASS
デッドロックなし: PASS
未定義遷移なし: PASS
```

ファイル名: `ex15_7_dsl.c`

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- 高度な言語機能や最適化を使用した箇所には、採用理由をコメントで記載してください。
- 性能測定を求める問題には、測定条件と結果を含めてください。

学習のポイント

- プリプロセッサは強力だが、使いすぎると可読性が低下する
- メモリ管理の設計は、実行性能とメモリ使用量に影響する
- キャッシュ効率を考慮すると大幅な性能向上が可能
- アロケーターやキャッシュなど、実行環境の仕組みを踏まえて設計する

第16章：C23の機能の演習問題

演習の目的

- C23標準で導入された新機能を理解する
- bool型、2進数リテラル、typeof演算子、nullptrを活用する
- モダンなCプログラミングスタイルを習得する
- より安全で表現力豊かなコードを書く

基礎問題

問題16-1：bool型の活用

bool型を使用して、簡単な状態管理システムを作成してください。

要件

- システムの各種状態をbool型で管理
- 状態の組み合わせによる判定ロジック
- 状態表示関数の実装
- 状態遷移の履歴記録

実装のヒント

```
typedef struct {
    bool is_connected;
    bool is_authenticated;
    bool has_permission;
    bool is_busy;
} SystemStatus;
```

期待される出力例

```
=== bool型を使った状態管理システム ===

初期状態:
接続: false
認証: false
権限: false
ビジー: false

システム接続中...
接続: true
認証: false (認証が必要です)

ユーザー認証...
接続: true
認証: true
権限: false (権限の付与待ち)

管理者権限を付与...
接続: true
認証: true
権限: true
ビジー: false

操作可能状態: Yes

=== 状態遷移履歴 ===
[00:00:00] システム起動
[00:00:01] 接続確立
[00:00:03] 認証成功
[00:00:05] 権限付与
[00:00:10] 処理開始 (ビジー: true)
[00:00:15] 処理完了 (ビジー: false)
```

問題16-2：ビット操作と2進数リテラル

2進数リテラルを使用して、8ビットのフラグ管理システムを作成してください。

要件

- 各ビットが異なる機能のON/OFFを表す
- ビットの設定、クリア、トグル、チェック関数
- 現在の状態を2進数で表示
- 複数フラグの一括操作

実装例

```
#define FLAG_READ    0b00000001
#define FLAG_WRITE   0b00000010
#define FLAG_EXECUTE 0b00000100
```

期待される出力例

```
=== 2進数リテラルを使ったフラグ管理 ===

権限フラグの定義:
FLAG_READ    = 0b00000001 (0x01)
FLAG_WRITE   = 0b00000010 (0x02)
FLAG_EXECUTE = 0b00000100 (0x04)
FLAG_DELETE  = 0b00001000 (0x08)

初期フラグ: 0b00000000

読み取り権限を設定:
現在のフラグ: 0b00000001 (READ)

書き込み権限を追加:
現在のフラグ: 0b00000011 (READ | WRITE)

すべての権限を設定:
現在のフラグ: 0b00001111 (READ | WRITE | EXECUTE | DELETE)

実行権限をトグル:
現在のフラグ: 0b00001011 (READ | WRITE | DELETE)

権限チェック:

- 読み取り: yes
- 書き込み: yes
- 実行: no
- 削除: yes

一括操作（読み取り専用に設定）:
現在のフラグ: 0b00000001 (READ)
```

応用問題

問題16-3：typeof演算子の応用

typeof演算子を使用して、汎用的なデータ構造操作マクロを作成してください。

要件

- 配列の最大値・最小値を求めるマクロ
- 2つの変数を安全に交換するマクロ
- 配列要素の合計を計算するマクロ
- 型安全な比較関数マクロ

実装例

```
#define ARRAY_MAX(arr, size) ({ \
    typeof(arr[0]) max = arr[0]; \
    /* 実装を完成させる */ \
    max; \
})
```

期待される出力例

```
=== typeof演算子の汎用マクロ ===

整数配列のテスト:
配列: [34, 67, 12, 89, 45, 23, 78, 56]
最大値: 89
最小値: 12
合計: 404

浮動小数点配列のテスト:
配列: [3.14, 2.71, 1.41, 1.73, 2.23]
最大値: 3.14
最小値: 1.41
合計: 11.22

安全な変数交換:
交換前: a=10, b=20
交換後: a=20, b=10

文字列の交換:
交換前: s1="Hello", s2="World"
交換後: s1="World", s2="Hello"

型安全な比較:
compare(10, 20) = -1
compare(3.14, 3.14) = 0
compare('Z', 'A') = 1
```

問題16-4：nullptr安全プログラミング

nullptrを活用して、安全なリンクリスト操作関数を作成してください。

要件

- ノードの追加、削除、検索
- すべてのポインター操作でnullptrチェック
- エラー処理の実装
- メモリリークの防止

実装のヒント

- 従来のNULLの代わりにnullptrを使用
- 戻り値でのエラー通知

期待される出力例

```
=== nullptr安全リンクリスト ===

要素を追加: 10
要素を追加: 20
要素を追加: 30

リストの内容: 10 -> 20 -> 30 -> nullptr

要素20を検索... 見つかりました
要素40を検索... 見つかりません

要素20を削除...
リストの内容: 10 -> 30 -> nullptr

空のリストに対する操作:
削除試行: エラー - リストが空です
検索試行: エラー - リストが空です

nullptrチェック統計:

- nullptrチェック総数: 42
- nullptr検出回数: 7
- 安全に処理された操作: 100%

メモリリーク検査: OK (すべてのメモリが解放されました)
```

問題16-5:デジット区切り記号の活用

デジット区切り記号を使って、大きな数値を扱うプログラムを作成してください。

要件

- 10進数、16進数、2進数での表記
- 可読性の高い定数定義
- ビットマスクの定義
- メモリサイズの計算

実装例

```
#define MEMORY_SIZE 16'777'216  // 16MB
#define MASK_32BIT  0xFFFF'FFFF
#define PATTERN     0b1010'1010'1010'1010
```

期待される出力例

```
=== デジット区切り記号を使った数値表現 ===

大きな数値の定義:
人口: 7'825'000'000 人
メモリサイズ: 16'777'216 バイト (16MB)
国家予算: 106'600'000'000'000 円

16進数表記:
アドレス空間: 0xFFFF'FFFF'FFFF'FFFF
カラーコード: 0x00FF'00FF (緑とマゼンタ)

2進数表記:
ビットパターン: 0b1010'1010'1010'1010
マスク: 0b1111'1111'0000'0000

計算例:
16MB = 16'777'216 バイト
     = 16'384 KB
     = 16 MB

ビット演算:
0b1111'0000'1111'0000 AND 0b1010'1010'1010'1010
= 0b1010'0000'1010'0000
```

チャレンジ問題

問題16-6:C23総合演習:設定管理システム

C23の複数の新機能を組み合わせた設定管理システムを作成してください。

要件

- bool型で各種設定のON/OFF管理
- 2進数リテラルでフラグ管理
- nullptrで安全なポインター操作
- typeof演算子でジェネリックな操作
- 設定の保存と読み込み機能

機能要件

- 設定項目の動的追加・削除
- 設定のグループ化
- デフォルト値の管理
- 設定変更の通知機能

期待される出力例

```
=== C23総合設定管理システム ===

設定グループ: Display

- brightness: true (bool)
- dark_mode: false (bool)
- resolution: 1920x1080
- refresh_rate: 60

設定グループ: Network

- wifi_enabled: true (bool)
- bluetooth_enabled: false (bool)
- airplane_mode: false (bool)

設定フラグ (2進数表示):
現在: 0b0000'0101 (WiFi | Brightness)

設定変更: dark_mode = true
通知: DisplaySettingsChanged -> dark_mode

設定の保存...
ファイル: settings.conf
保存完了 (チェックサム: 0xA5C3)

=== typeof演算子による汎用操作 ===
設定値の型安全な更新:

- 整数値: 60 -> 144 (成功)
- bool値: false -> true (成功)
- 文字列: "1920x1080" -> "2560x1440" (成功)
```

問題16-7:ビット演算デバッガ

教育的なビット演算デバッガを作成してください。

要件

- 2進数リテラルでの入力サポート
- ビット演算の可視化
- ステップ実行機能

- 演算結果の詳細説明

実装すべき演算

- AND, OR, XOR, NOT
- ビットシフト(左右)
- ビット回転
- ビットカウント

期待される出力例

```
=== ビット演算デバッガ ===

入力方式を選択:

1. 2進数 (0b...)
2. 16進数 (0x...)
3. 10進数

選択: 1
値A: 0b1010'1100
値B: 0b1100'1010

=== ビット演算の可視化 ===

AND演算:
  1010 1100
& 1100 1010
-----------
  1000 1000 (0x88)

OR演算:
  1010 1100
| 1100 1010
-----------
  1110 1110 (0xEE)

XOR演算:
  1010 1100
^ 1100 1010
-----------
  0110 0110 (0x66)

NOT演算:
~ 1010 1100
-----------
  0101 0011 (0x53)

左シフト (2ビット):
1010 1100 << 2 = 1011 0000 (0xB0)
  ↑↑ 消失

右シフト (2ビット):
1010 1100 >> 2 = 0010 1011 (0x2B)
            消失 ↑↑

ビット数: A=5個, B=5個
```

提出物

- 問題ごとに独立したCファイルを作成し、本文に示したファイル名を使用してください。
- C23の機能を使用する箇所には、従来の書き方との差をコメントで記載してください。
- 使用するコンパイラーが対象機能を実装していることを確認してください。

学習のポイント

- C23の新機能は従来のCコードをより安全で読みやすくする
- bool型により意図が明確なコードが書ける

- 2進数リテラルはビット操作を直感的にする
- nullptrはポインターの安全性を向上させる
- typeof演算子により型安全なマクロが作成可能

補章2：開発環境の演習問題

演習の目的

開発環境の詳細な設定と診断、コンパイル過程の理解を深めます。

基礎問題

問題1：環境情報の確認

`examples/environment_check.c` を各C規格でコンパイルして実行し、結果を比較してください。

```
# C90準拠

gcc -std=c90 -pedantic environment_check.c -o check_c90
./check_c90

# C99準拠

gcc -std=c99 -pedantic environment_check.c -o check_c99
./check_c99

# C11準拠

gcc -std=c11 -pedantic environment_check.c -o check_c11
./check_c11
```

問題2：コンパイル過程の確認

以下のコマンドで各段階の出力を確認してください。

```
# プリプロセッサ出力

gcc -E examples/compiler_test.c > preprocessed.i

# アセンブリコード

gcc -S examples/compiler_test.c

# オブジェクトファイル

gcc -c examples/compiler_test.c

# 最終的な実行ファイル

gcc compiler_test.o -o compiler_test
```

応用問題

問題3：環境診断ツールの作成

`solutions/env_diagnostic.c` を作成し、以下の情報を表示するプログラムを作成してください。

表示項目

- コンパイラーの種類とバージョン
- C規格のバージョン
- システムのエンディアン
- 各データ型のサイズと範囲
- ポインターのサイズ（32bit/64bit判定）

問題4：コンパイラーオプションの比較

同じプログラムを異なるオプションでコンパイルし、違いを確認してください。

```
# 最適化なし
gcc -O0 test.c -o test_O0
# 最適化レベル2
gcc -O2 test.c -o test_O2
# サイズ最適化
gcc -Os test.c -o test_Os
# ファイルサイズを比較
ls -l test_*
```

チャレンジ問題

問題5：マルチ規格対応プログラム

異なるC規格で異なる動作をするプログラムを作成してください。

要求仕様

- プリプロセッサマクロで規格を判定
- 各規格特有の機能を使用
- 実行時に使用している規格を表示

問題6：包括的環境レポート生成ツール

開発環境の完全なレポートを生成するツールを作成してください。

機能要件

- HTMLまたはMarkdown形式でレポート出力
- 問題がある場合は警告と対処法を表示
- 推奨される設定を提案

参考資料

- GCC公式ドキュメント
- Clang公式ドキュメント
- GNU Makeマニュアル

解答例

第1章：C言語入門の解答例

この章では、C言語の開発環境構築の確認と基本的なプログラムの作成練習を行いました。

解答ファイル一覧

補足解答：名前入力と挨拶

ユーザーに名前の入力を求め、挨拶メッセージを表示するプログラムです。

- ex1_3_hello_name.c - C90準拠版
- ex1_3_hello_name_c99.c - C99準拠版（//コメント、変数宣言位置の自由化）

学習ポイント

- 標準入出力（printf、scanf）の基本的な使い方
- 文字配列（文字列）の扱い方
- 入力エラーの処理
- C90とC99のコーディングスタイルの違い

発展解答：システム情報調査プログラム

実行環境の詳細情報を調査・表示するプログラムです。

- ex1_6_system_info.c - C90準拠版
- ex1_6_system_info_c99.c - C99準拠版（bool型、stdint.h、複合リテラル使用）

学習ポイント

- プリプロセッサディレクティブによる条件コンパイル
- エンディアン判定の実装方法
- データ型のサイズとlimits.hの使用
- C99で追加された機能の確認方法

コンパイルと実行

各解答例は以下のようにコンパイルできます。

```
# C90版のコンパイル例

gcc -Wall -Wextra -pedantic -std=c90 ex1_3_hello_name.c -o ex1_3_hello_name

# C99版のコンパイル例

gcc -Wall -Wextra -pedantic -std=c99 ex1_3_hello_name_c99.c -o ex1_3_hello_name_c99

# 実行

./ex1_3_hello_name
```

親ディレクトリのMakefileを使用する場合。

```
# すべての解答例をコンパイル

make solutions

# 個別にコンパイル

make ex1_3_hello_name
make ex1_6_system_info_c99

# 実行
```

```
make run-ex1_3_hello_name
```

学習のポイント

1. C90とC99の主な違い

各解答例にはC90版とC99版の両方を用意しています。主な違いは次のとおりです。

- **コメント**:C90は/* */のみ、C99は//も使用可能
- **変数宣言**:C90は関数の先頭、C99は使用箇所で宣言可能
- **新しい型**:C99ではbool、long long、stdint.hの固定幅整数型が追加
- **その他**:可変長配列、複合リテラル、designated initializerなど

2. 入力処理の安全性

ex1_3_hello_name.cでは、バッファーオーバーフローを防ぐため。

```
scanf("%99s", name);  /* 最大99文字に制限 */
```

3. システム情報の取得方法

ex1_6_system_info.cでは、以下の技術を使用。

- **エンディアン判定**:unionを使用したバイト順の確認
- **アーキテクチャ判定**:ポインターサイズによる判定
- **OS判定**:プリプロセッサマクロの利用
- **コンパイラー判定**:コンパイラー固有のマクロ

4. エラー処理の重要性

すべてのプログラムで適切なエラー処理を実装。

- 入力エラーのチェック
- 戻り値による状態の通知
- エラーメッセージの表示

発展的な学習

これらの基本的な実装を理解したら、以下に挑戦してみましょう。

1. **入力の拡張**:複数の単語を含む名前の入力(fgetsの使用)
2. **国際化**:日本語以外の言語への対応
3. **クロスプラットフォーム**:より多くのOSやコンパイラーへの対応
4. **ビルドシステム**:CMakeなどの高度なビルドツールの使用

注意事項

- 実装例はあくまで一例です。他の実装方法も検討してみましょう
- 実際のプロジェクトでは、より堅牢なエラー処理が必要です
- システム固有の情報は環境によって異なる結果となります

第2章：基本構文の解答例

解答例一覧

演習2-1：自己紹介プログラム

- **ファイル**：`ex2_1_hello_intro.c`, `ex2_1_hello_intro_c99.c`
- **学習内容**：printf関数の基本的な使い方、改行文字の使用
- **ポイント**：複数行の出力とフォーマットの基本

演習2-2：個人データ表示

- **ファイル**：`ex2_2_personal_data.c`, `ex2_2_personal_data_c99.c`
- **学習内容**：変数の宣言と初期化、各種データ型の使用
- **ポイント**：int, float, char型の実用的な使い方

演習2-3：年齢計算プログラム

- **ファイル**：`ex2_3_age_calculator.c`, `ex2_3_age_calculator_c99.c`
- **学習内容**：scanf関数による入力、計算処理
- **ポイント**：ユーザー入力を受け取って計算結果を表示

演習2-4：フォーマット指定子の実践

- **ファイル**：`ex2_4_format_demo.c`, `ex2_4_format_demo_c99.c`
- **学習内容**：printf関数の書式指定子、桁数指定
- **ポイント**：%d, %f, %c, %sの実用的な使い方

演習2-5：簡単な計算機

- **ファイル**：`ex2_5_simple_calculator.c`, `ex2_5_simple_calculator_c99.c`
- **学習内容**：四則演算、浮動小数点数の処理
- **ポイント**：複数の入力と計算結果の表示

演習2-6：ASCII文字探索

- **ファイル**：`ex2_6_ascii_explorer.c`, `ex2_6_ascii_explorer_c99.c`
- **学習内容**：文字と数値の変換、ASCIIコードの理解
- **ポイント**：char型と整数型の関係性

C90版とC99版の違い

C90版（基本ファイル）

- すべての変数を関数の先頭で宣言
- /* */ 形式のコメントを使用
- 従来のC言語の書き方に準拠

C99版（_c99.cファイル）

- 変数を使用する箇所の近くで宣言可能

- // 形式のコメントに対応
- より現代的なC言語の書き方

コンパイルと実行

C90版

```
gcc -Wall -Wextra -pedantic -std=c90 hello_intro.c -o hello_intro
./hello_intro
```

C99版

```
gcc -Wall -Wextra -pedantic -std=c99 hello_intro_c99.c -o hello_intro_c99
./hello_intro_c99
```

Makefileを使用

```
make all           # すべての解答例をコンパイル
make clean         # 実行ファイルを削除
make run-hello     # hello_introを実行
```

学習のポイント

1. **基本的な出力**:printf関数の正しい使い方を習得
2. **変数の扱い**:各種データ型の特徴を理解
3. **入力処理**:scanf関数の安全な使用方法
4. **書式指定**:適切なフォーマット指定子の選択
5. **エラー処理**:基本的な入力検証の考え方

注意事項

- すべての解答例は教育目的で作成されています
- 実際の開発では、より厳密なエラー処理が必要です
- C90版を基本として学習し、C99版で現代的な書き方を確認してください

次のステップ

この章をマスターしたら、次は以下の章に進みましょう。

- 第3章:データ型・変数・スコープ
- 第5章:制御構造(条件分岐)

第3章：データ型と変数の解答例

この章の演習問題の解答例を示します。各解答にはC90準拠版とC99準拠版があり、両者の違いを確認できます。

演習解答一覧

演習3-1：変数宣言と初期化

ファイル： `ex3_1_variable_declaration.c` | C99版

さまざまなデータ型の変数を宣言し、初期化して出力するプログラムです。

学習内容

- 各データ型の適切な宣言方法
- 初期化の重要性
- printf関数での型に応じた書式指定子の使用
- sizeof演算子によるサイズ確認
- 定数の定義と使用

演習3-2：四則演算計算機

ファイル： `ex3_2_calculator.c` | C99版

2つの数値を変数に格納し、四則演算の結果を表示するプログラムです。

学習内容

- 整数演算と実数演算の違い
- 四則演算子（+, -, *, /, %）の使い方
- 整数除算では小数点以下が切り捨てられる
- 剰余演算子(%)は整数のみで使用可能
- 混合演算での自動型変換
- 明示的型変換（キャスト）の使用

演習3-3：データ型サイズの確認

ファイル： `ex3_3_sizeof_demo.c` | C99版

sizeof演算子を使って、各データ型のサイズを表示するプログラムです。

学習内容

- sizeof演算子でデータ型や変数のサイズを確認
- limits.hヘッダで整数型の範囲定数を取得
- float.hヘッダで浮動小数点型の特性を取得
- 配列のサイズと要素数の計算方法
- ポインターのサイズはシステム依存
- C99の固定幅整数型(stdint.h)

演習3-4：型変換の理解

ファイル： `ex3_4_type_conversion.c` | C99版

整数除算と実数除算の違いを確認し、型変換について学習するプログラムです。

学習内容

- 整数除算は小数点以下が切り捨てられる
- 暗黙の型変換は自動的に行われる
- 明示的な型変換(キャスト)で強制的に変換できる
- 型変換には優先順位がある
- charとintの間でASCII値による変換が可能
- オーバーフローは上位ビットの切り捨てで発生

演習3-5:スコープの実験

ファイル: `ex3_5_scope_experiment.c` | C99版

グローバル変数とローカル変数を使い分け、スコープの概念を学習するプログラムです。

学習内容

- グローバル変数はファイル全体でアクセス可能
- ローカル変数は宣言されたブロック内でのみ有効
- 静的ローカル変数は関数呼出間で値を保持
- static指定子でファイル内限定のグローバル変数を作成可能
- 内側のスコープで同名変数を宣言すると外側を隠蔽
- ブロックスコープは {} で区切られた範囲

演習3-6:定数の活用

ファイル: `ex3_6_constants_demo.c` | C99版

constと#defineを使って定数を定義し、物理計算に使用するプログラムです。

学習内容

- #defineによるマクロ定数の定義
- const変数による定数の定義
- マクロ関数の作成と使用
- 物理計算での定数の活用
- 定数の命名規則(大文字、アンダースコア)
- #define vs constの違い

C90とC99の主な違い

C90の特徴

- 変数宣言は関数やブロックの先頭で行う必要がある
- /* */形式のコメントのみ使用可能
- 配列サイズは定数式である必要がある
- long long型は使用できない

C99の新機能

- 変数を使用箇所で宣言可能
- // 形式のコメントが使用可能
- for文内で変数宣言可能（for（int i = 0; …））
- 可変長配列（VLA）をサポート
- long long型の追加（64ビット整数）
- 固定幅整数型（int8_t, int16_t, int32_t, int64_t等）

コンパイル方法

C90準拠でコンパイル

```
gcc -std=c90 -Wall -Wextra -pedantic filename.c -o output
```

C99準拠でコンパイル

```
gcc -std=c99 -Wall -Wextra -pedantic filename.c -o output
```

数学ライブラリが必要な場合

```
gcc -std=c90 -Wall -Wextra -pedantic filename.c -o output -lm
```

重要な学習ポイント

データ型の選択基準

- **char**：文字データ、小さな整数
- **short**：小さな整数（メモリ節約）
- **int**：一般的な整数計算
- **long**：大きな整数
- **unsigned**：負の値を扱わない場合
- **float**：精度をそれほど要求しない実数
- **double**：高精度な実数計算

変数のスコープと生存期間

- **自動変数**：ブロック開始時に作成、終了時に破棄
- **静的変数**：プログラム開始時に作成、終了まで存続
- **グローバル変数**：プログラム全体で存続

定数の使い分け

- **#define**：プリプロセッサによる文字列置換、型情報なし
- **const**：型付き定数、デバッガーで値確認可能、スコープあり

チャレンジ問題

演習3-7：温度変換プログラム

ファイル： `ex3_7_temperature_converter.c` | C99版

摂氏・華氏・ケルビンの温度単位を相互変換するプログラムです。

学習内容

- #defineマクロによる定数の定義
- 浮動小数点演算の精度管理
- 物理的な意味を持つ計算の実装
- 複数の単位系の相互変換
- 科学技術計算での定数活用

よくある間違いと対策

1. 初期化忘れ

```
/* NG: 初期化せずに使用 */
int count;
printf("%d\n", count);  /* 未定義値が出力される */

/* OK: 初期化してから使用 */
int count = 0;
printf("%d\n", count);
```

2. 整数除算の結果

```
/* NG: 期待した結果にならない */
int result = 5 / 2;     /* 結果: 2 */

/* OK: 実数除算を使用 */
double result = 5.0 / 2.0;  /* 結果: 2.5 */
```

3. 型の範囲超過

```
/* NG: charの範囲を超える */
char big_num = 300;     /* オーバーフロー */

/* OK: 適切な型を使用 */
int big_num = 300;
```

これらの演習を通じて、C言語の基本データ型、変数のスコープ、定数の活用方法を習得してください。

第4章：演算子の解答例

このディレクトリには演習問題の解答例が含まれています。

解答ファイル一覧

基礎問題

1. **演習4-1：四則演算計算機**
   - ex4_1_calculator.c - C90準拠版
   - ex4_1_calculator_c99.c - C99準拠版
   - 2つの整数に対する基本的な算術演算を実行
2. **演習4-2：比較と論理演算**
   - ex4_2_comparison.c - C90準拠版
   - ex4_2_comparison_c99.c - C99準拠版
   - 条件演算子のみで最大値・最小値を求める
3. **演習4-3：インクリメント・デクリメント**
   - ex4_3_increment.c - C90準拠版
   - ex4_3_increment_c99.c - C99準拠版
   - 前置と後置の違いを詳細に確認

応用問題

1. **演習4-4：ビット操作**
   - ex4_4_bitwise.c - C90準拠版
   - ex4_4_bitwise_c99.c - C99準拠版
   - ビット演算子を使った各種操作の実装
2. **演習4-5：条件演算子の活用**
   - ex4_5_conditional.c - C90準拠版
   - ex4_5_conditional_c99.c - C99準拠版
   - 3つの数値を条件演算子のみで昇順に並び替え
3. **演習4-6：演算子優先順位**
   - ex4_6_precedence.c - C90準拠版
   - ex4_6_precedence_c99.c - C99準拠版
   - 複雑な式での演算子優先順位の確認

学習のポイント

C90版とC99版の主な違い

1. **変数宣言位置**
   - C90: ブロックの先頭でのみ宣言可能
   - C99: 使用箇所で宣言可能

2 **bool型の使用**
   - C90: int型で真偽値を表現
   - C99: stdbool.hによるbool型の使用
3 **inline関数**
   - C90: 未対応
   - C99: inline関数による効率化
4 **コメントスタイル**
   - C90:/* */のみ
   - C99: // も使用可能
5 **固定幅整数型**
   - C90: 標準整数型のみ
   - C99: stdint.hによるuint8_t等の使用

各演習の重要概念

演習4-1:四則演算計算機

- ゼロ除算のエラー処理
- 整数除算と実数除算の違い
- 代入演算子の使い方

演習4-2:比較と論理演算

- 関係演算子の評価結果(0または1)
- 論理演算子による条件の組み合わせ
- 条件演算子によるif文なしの条件分岐

演習4-3:インクリメント・デクリメント

- 前置と後置の評価タイミングの違い
- 式の中での副作用
- 未定義動作の回避

演習4-4:ビット操作

- ビットマスクの作成と使用
- 特定ビットの設定・クリア・反転
- ビットシフトによる乗除算

演習4-5:条件演算子の活用

- ネストした条件演算子の使い方
- 複数条件の組み合わせ
- 可読性とのバランス

演習4-6:演算子優先順位

- 演算子の優先順位と結合規則
- 括弧による評価順序の制御

- よくある間違いとその回避方法

コンパイルと実行

```
# C90準拠でコンパイル

gcc -std=c90 -Wall -Wextra -pedantic ex4_1_calculator.c -o ex4_1_calculator

# C99準拠でコンパイル

gcc -std=c99 -Wall -Wextra -pedantic ex4_1_calculator_c99.c -o ex4_1_calculator_c99

# 実行

./ex4_1_calculator
```

注意事項

- すべての解答例は教育目的で書かれており、実用的なコードよりも理解しやすさを重視しています
- エラー処理は基本的なものに留めており、実際のプロダクションコードではより堅牢な処理が必要です
- 各解答ファイルには詳細なコメントが含まれているので、コードと合わせて読むことを推奨します

第5章：条件分岐の解答例

解答例一覧

高度な解答例

成績計算システム

- **ファイル**：`grade_calculator.c`, `grade_calculator_c99.c`
- **学習内容**：if-else if連鎖、switch文、複合条件判定
- **ポイント**：実用的な評価システムの実装
- **機能**：テスト点数、出席率、宿題提出状況による総合評価

演習問題の解答

演習5-1：年齢による料金計算

- **ファイル**：`ex5_1_ticket_pricing.c`, `ex5_1_ticket_pricing_c99.c`
- **学習内容**：if-else if文による段階的判定
- **ポイント**：境界値の適切な処理

演習5-2：電卓プログラム

- **ファイル**：`ex5_2_calculator.c`, `ex5_2_calculator_c99.c`
- **学習内容**：switch文による多分岐処理
- **ポイント**：ゼロ除算エラーの処理

演習5-3：BMI判定プログラム

- **ファイル**：`ex5_3_bmi_judge.c`, `ex5_3_bmi_judge_c99.c`
- **学習内容**：浮動小数点数の計算と範囲判定
- **ポイント**：単位変換と健康アドバイス

演習5-4：うるう年判定

- **ファイル**：`ex5_4_leap_year.c`, `ex5_4_leap_year_c99.c`
- **学習内容**：複雑な論理演算子の組み合わせ
- **ポイント**：複合条件式の正確な実装

演習5-5：成績評価システム

- **ファイル**：`ex5_5_grade_system.c`, `ex5_5_grade_system_c99.c`
- **学習内容**：条件演算子と複合条件の活用
- **ポイント**：複数データの統合的な評価

チャレンジ5-7：じゃんけんゲーム

- **ファイル**：`ex5_6_janken_game.c`, `ex5_6_janken_game_c99.c`
- **学習内容**：ランダム数生成と複雑な勝敗判定
- **ポイント**：ゲームロジックの実装と統計分析

C90版とC99版の違い

C90版(基本ファイル)

- すべての変数を関数の先頭で宣言
- /* */ 形式のコメントを使用
- 従来のC言語の制御構造に準拠

C99版(_c99.cファイル)

- 変数を使用する箇所の近くで宣言可能
- // 形式のコメントに対応
- より現代的な制御構造の書き方

コンパイルと実行

基本的なコンパイル

```
gcc -Wall -Wextra -pedantic -std=c90 ex5_1_ticket_pricing.c -o ex5_1_ticket_pricing
./ex5_1_ticket_pricing
```

C99版のコンパイル

```
gcc -Wall -Wextra -pedantic -std=c99 ex5_1_ticket_pricing_c99.c -o ex5_1_ticket_pricing_c99
./ex5_1_ticket_pricing_c99
```

Makefileを使用

```
make all                    # すべてをコンパイル
make solutions              # 解答例のみコンパイル
make ex5_1                  # 個別コンパイル
make run-solutions          # 解答例を実行
make clean                  # 実行ファイルを削除
```

学習のポイント

条件分岐の基本

1. **if文**:単純な条件判定
2. **if-else文**:2分岐の処理
3. **if-else if文**:多段階の条件判定
4. **switch文**:値による多分岐処理
5. **条件演算子**:簡潔な条件分岐

実用的なテクニック

1. **入力検証**:ユーザー入力の安全性確保
2. **エラー処理**:適切なエラー処理
3. **境界値処理**:範囲指定の正確な実装
4. **複合条件**:論理演算子の効果的な使用
5. **可読性**:理解しやすいコードの書き方

設計パターン

1. **段階的判定**:if-else if連鎖による階層的処理

2 **値による分岐**：switch文による効率的な分岐

3 **状態管理**：フラグ変数による状態の管理

4 **統合的判定**：複数条件による総合的な評価

注意事項

- 条件式では等価演算子（==）と代入演算子（=）を混同しない
- switch文ではbreak文を忘れずに記述
- 浮動小数点数の比較では誤差を考慮
- 論理演算子の優先順位に注意

実用的な応用

これらの制御構造は以下のような実用的なプログラムで使用されます。

- **Webアプリケーション**：ユーザー認証、フォーム検証
- **ゲーム**：プレイヤーの行動による分岐処理
- **データ分析**：条件による分類・集計処理
- **システム管理**：設定による動作の制御

次のステップ

この章をマスターしたら、次の章に進みましょう。

- 第6章：制御構造（ループ）
- 第7章：配列（基本編）
- 第10章：文字列操作
- 第11章：関数

第6章：繰り返し処理の解答例

解答例一覧

演習6-1：フィボナッチ数列

- **ファイル**：ex6_1_fibonacci.c, ex6_1_fibonacci_c99.c
- **学習内容**：for文による繰り返し処理、フィボナッチ数列のアルゴリズム
- **ポイント**：変数の値の更新と循環、初期値の特別処理

演習6-2：数当てゲーム（改良版）

- **ファイル**：ex6_2_number_guess.c, ex6_2_number_guess_c99.c
- **学習内容**：do-while文による繰り返し、ランダム数生成
- **ポイント**：複数の終了条件、デモ版として自動実行

演習6-4：素数リスト

- **ファイル**：ex6_3_prime_list.c, ex6_3_prime_list_c99.c
- **学習内容**：ネストしたループ、break文、効率的な素数判定
- **ポイント**：数学的最適化、統計情報の計算

演習6-5：図形の描画

- **ファイル**：ex6_4_draw_shapes.c, ex6_4_draw_shapes_c99.c
- **学習内容**：ネストしたループによる2次元パターン生成
- **ポイント**：空白とアスタリスクの配置制御、複雑な図形の分解

演習6-3：掛け算表

- **ファイル**：ex6_5_multiplication_table.c, ex6_5_multiplication_table_c99.c
- **学習内容**：ネストしたfor文、表形式データの生成
- **ポイント**：書式指定子による桁揃え、ヘッダーと区切り線

補足解答：階乗計算

- **ファイル**：ex6_6_factorial.c, ex6_6_factorial_c99.c
- **学習内容**：while文による繰り返し、オーバーフロー対策
- **ポイント**：long long型の使用、計算過程の表示

C90版とC99版の違い

C90版（基本ファイル）

- すべての変数を関数の先頭で宣言
- /* */ 形式のコメントを使用
- 従来のCの制御構造に準拠

C99版（_c99.cファイル）

- 変数を使用する箇所の近くで宣言可能

- for文内でのループ変数宣言
- // 形式のコメントに対応
- より現代的な制御構造の書き方

コンパイルと実行

基本的なコンパイル

```
gcc -Wall -Wextra -pedantic -std=c90 ex6_1_fibonacci.c -o ex6_1_fibonacci
./ex6_1_fibonacci
```

C99版のコンパイル

```
gcc -Wall -Wextra -pedantic -std=c99 ex6_1_fibonacci_c99.c -o ex6_1_fibonacci_c99
./ex6_1_fibonacci_c99
```

数学関数を使用する場合(素数判定)

```
gcc -Wall -Wextra -pedantic -std=c90 ex6_3_prime_list.c -o ex6_3_prime_list -lm
./ex6_3_prime_list
```

学習のポイント

ループの基本

1. **for文**:回数が決まった繰り返し処理
2. **while文**:条件が真の間の繰り返し処理
3. **do-while文**:最低1回は実行される繰り返し処理
4. **ネストしたループ**:2次元的な処理パターン

制御文の活用

1. **break文**:ループの早期終了
2. **continue文**:次の繰り返しへのスキップ
3. **条件分岐との組み合わせ**:複雑な制御フロー

実用的なテクニック

1. **計算アルゴリズム**:フィボナッチ、階乗、素数判定
2. **パターン生成**:図形描画、表形式データ
3. **入力検証**:エラー処理とユーザビリティ
4. **最適化**:効率的なアルゴリズムの実装

プログラム設計

1. **段階的な実装**:簡単な処理から複雑な処理へ
2. **モジュール化**:機能ごとの分割と整理
3. **デバッグ**:ループの動作確認と修正
4. **可読性**:理解しやすいコードの書き方

注意事項

- ループの無限ループに注意

- 配列の境界を超えないよう注意
- オーバーフローの可能性を考慮
- 効率的なアルゴリズムの選択

対話型プログラムについて

一部のサンプルコードは、処理の流れを確認しやすい非対話型のデモとして実装しています。対話型に変更する場合は、ユーザー入力の検証とエラー処理を追加してください。

実用的な応用

これらの制御構造は以下のような実用的なプログラムで使用されます。

- **データ処理**：大量データの反復処理
- **ゲーム**：ゲームループと状態管理
- **シミュレーション**：反復計算による数値解析
- **グラフィックス**：パターン生成と描画処理

次のステップ

この章をマスターしたら、次の章に進みましょう。

- 第7章：配列（基本編）
- 第10章：文字列操作
- 第11章：関数

第7章：配列の解答例

解答例一覧

解答例7-1：数値の統計処理

- **ファイル**：ex7_1.c, ex7_1_c99.c
- **学習内容**：1次元配列の操作、最大値・最小値・平均値の計算
- **ポイント**：配列の初期化、ループによる要素の処理

解答例7-2：配列内の検索

- **ファイル**：ex7_2.c, ex7_2_c99.c
- **学習内容**：線形検索アルゴリズム、見つかった要素のインデックス取得
- **ポイント**：配列の走査、条件判定による検索処理

解答例7-3：配列のソート

- **ファイル**：ex7_3.c, ex7_3_c99.c
- **学習内容**：バブルソートアルゴリズム、配列要素の交換
- **ポイント**：ネストしたループ、要素の入れ替え処理

解答例7-4：2次元配列による行列演算

- **ファイル**：ex7_4.c, ex7_4_c99.c
- **学習内容**：2次元配列の操作、行列の加算・乗算
- **ポイント**：多次元配列のインデックス操作、行列計算アルゴリズム

解答例7-5：文字列の操作

- **ファイル**：ex7_5.c, ex7_5_c99.c
- **学習内容**：文字配列による文字列処理、文字列関数の実装
- **ポイント**：文字配列と文字列の関係、ヌル終端文字の扱い

チャレンジ問題

発展解答7-1：ソートアルゴリズムの実装と比較

- **ファイル**：ex7_challenge1.c, ex7_challenge1_c99.c
- **学習内容**：バブルソート、選択ソート、挿入ソートの実装と性能比較
- **ポイント**：アルゴリズムの性能測定、実行時間と比較・交換回数の計測

発展解答7-2：ライフゲーム（Conway's Game of Life）

- **ファイル**：ex7_challenge2.c, ex7_challenge2_c99.c
- **学習内容**：2次元配列を使ったセルオートマトンの実装
- **ポイント**：複雑なルールの実装、境界条件の処理、世代管理

C90版とC99版の違い

C90版（基本ファイル）

- すべての変数を関数の先頭で宣言
- 配列のサイズは定数で指定
- /* */ 形式のコメントを使用
- 従来のC言語の配列操作に準拠

C99版(_c99.cファイル)

- 変数を使用する箇所の近くで宣言可能
- 可変長配列(VLA)に対応(一部の演習で使用)
- // 形式のコメントに対応
- より現代的な配列操作

コンパイルと実行

C90版

```
gcc -Wall -Wextra -pedantic -std=c90 ex7_1.c -o ex7_1
./ex7_1
```

C99版

```
gcc -Wall -Wextra -pedantic -std=c99 ex7_1_c99.c -o ex7_1_c99
./ex7_1_c99
```

Makefileを使用

```
make all              # すべての解答例をコンパイル
make solutions        # 解答例のみコンパイル
make ex7_1            # 個別の解答例をコンパイル
make ex7_1_c99        # C99版の個別コンパイル
make run-solutions    # 解答例を実行
make clean            # 実行ファイルを削除
```

学習のポイント

1. **配列の基本操作**:宣言、初期化、要素アクセス
2. **配列の走査**:forループによる全要素の処理
3. **アルゴリズムの実装**:検索、ソート、統計処理
4. **多次元配列**:2次元、3次元配列の効果的な使用
5. **文字列処理**:文字配列による文字列操作
6. **メモリ効率**:配列サイズとメモリ使用量の関係

配列の重要概念

インデックス

- 配列の要素は0から始まるインデックスでアクセス
- 配列の境界を超えないよう注意(バッファーオーバーラン対策)

メモリレイアウト

- 配列要素は連続したメモリ領域に配置
- 多次元配列は行優先順序で格納

効率的な処理

- キャッシュ効率を考慮したメモリアクセスパターン
- アルゴリズムの時間計算量を意識した実装

注意事項

- 配列の境界チェックは必須(C言語は自動チェックしない)
- 初期化されていない配列要素は不定値
- 文字列配列では ヌル終端文字 ('\0') を忘れずに

次のステップ

この章をマスターしたら、次の章に進みましょう。

- 第10章:文字列操作
- 第11章:関数
- 第8章:ポインター基礎

第8章：ポインターの解答例

この章の演習問題の解答例です。各解答にはC90版とC99版の両方を用意しています。

演習問題の解答

演習8-1：ポインターの基本操作

2つの整数変数の値をポインターを使って交換する演習です。

- ex8_1_pointer_swap.c - C90準拠版
- ex8_1_pointer_swap_c99.c - C99準拠版（//コメント、bool型、インライン関数、restrict修飾子）

学習ポイント

- ポインターを渡して呼び出し元のオブジェクトを変更する仕組み
- 値渡しとの違いの理解
- アドレス演算子（&）と間接参照演算子（*）の使用
- ポインターの安全な使用方法

演習8-2：配列とポインター

ポインター演算を使って配列の要素を逆順に表示する演習です。

- ex8_2_array_reverse.c - C90準拠版
- ex8_2_array_reverse_c99.c - C99準拠版（可変長配列、複合リテラル、指定初期化子）

学習ポイント

- 配列添字とポインター演算の対応
- ポインター演算による配列要素アクセス
- メモリレイアウトの理解
- 安全な境界チェック

演習8-3：文字列操作

ポインターを使った文字列の長さ計算、複写、連結を行う演習です。

- ex8_3_string_manipulation.c - C90準拠版
- ex8_3_string_manipulation_c99.c - C99準拠版（//コメント、restrict修飾子、bool型）

学習ポイント

- 文字列とポインターの関係
- 文字列の終端（ヌル文字）の処理
- ポインターを使った文字列操作関数の実装
- バッファーオーバーフローの防止

演習8-4：配列操作関数

ポインターを使って配列の最大値、最小値、平均値を求める演習です。

- ex8_4_array_functions.c - C90準拠版
- ex8_4_array_functions_c99.c - C99準拠版（//コメント、bool型、restrict修飾子）

学習ポイント

- 配列をポインターとして関数に渡す方法
- 配列サイズの適切な管理
- ポインターを使った効率的な配列操作
- 関数の戻り値としてのポインターの活用

C90とC99の主な違い

1. コメントスタイル

- C90: /* */ のみ
- C99: // 単一行コメントも使用可能

2. 変数宣言

- C90: ブロックの先頭でのみ宣言可能
- C99: 使用箇所で宣言可能(forループ内での宣言など)

3. bool型

- C90: 存在しない(intで代用)
- C99: <stdbool.h>でbool、true、falseが使用可能

4. インライン関数

- C90: 存在しない
- C99: inlineキーワードで関数のインライン化が可能

5. 可変長配列(VLA)

- C90: 存在しない
- C99: 実行時にサイズが決まる配列を作成可能

6. 複合リテラル

- C90: 存在しない
- C99: (Type){初期化子}の形式で一時オブジェクトを作成可能

7. 指定初期化子

- C90: 存在しない
- C99: {[index] = value}の形式で配列の特定要素を初期化可能

8. restrict修飾子

- C90: 存在しない
- C99: ポインターが重複しないことをコンパイラーに伝える最適化ヒント

9. 固定幅整数型

- C90: 存在しない
- C99: <stdint.h>でint8_t、int16_t、int32_tなどが使用可能

実行方法

```
# C90版のコンパイルと実行

gcc -std=c90 -Wall -Wextra -pedantic ex8_1_pointer_swap.c -o ex8_1_pointer_swap
./ex8_1_pointer_swap

# C99版のコンパイルと実行

gcc -std=c99 -Wall -Wextra -pedantic ex8_1_pointer_swap_c99.c -o ex8_1_pointer_swap_c99
./ex8_1_pointer_swap_c99

# その他の演習例

gcc -std=c90 -Wall -Wextra -pedantic ex8_2_array_reverse.c -o ex8_2_array_reverse
gcc -std=c90 -Wall -Wextra -pedantic ex8_3_string_manipulation.c -o ex8_3_string_manipulation
gcc -std=c90 -Wall -Wextra -pedantic ex8_4_array_functions.c -o ex8_4_array_functions

# Makefileを使用した場合

make run-ex8_1_pointer_swap      # C90版
make run-ex8_1_pointer_swap_c99  # C99版
make run-ex8_2_array_reverse     # C90版
make run-ex8_2_array_reverse_c99 # C99版
make run-ex8_3_string_manipulation     # C90版
make run-ex8_3_string_manipulation_c99 # C99版
make run-ex8_4_array_functions     # C90版
make run-ex8_4_array_functions_c99 # C99版
```

ポインター使用時の注意事項

1. 初期化の重要性

```
// NG: 未初期化ポインターの使用
int *ptr;
*ptr = 10;  // 危険。未初期化ポインターの参照

// OK: 適切な初期化
int value = 0;
int *ptr = &value;
*ptr = 10;  // 安全
```

2. NULLポインターのチェック

```
// 関数でポインターを受け取る際は必ずチェック
void process_data(int *ptr)
{
    if (ptr == NULL) {
        printf("Error: received a null pointer\n");
        return;
    }
    // 処理を続行
}
```

3. 配列の境界チェック

```
// 配列のサイズを超えたアクセスは未定義動作
int arr[5] = {1, 2, 3, 4, 5};
int *ptr = arr;

// NG: 境界を超えたアクセス
printf("%d\n", *(ptr + 10));  // 危険。

// OK: 境界内でのアクセス
for (int i = 0; i < 5; i++) {
    printf("%d ", *(ptr + i));
}
```

4. ローカル変数のアドレスを返さない

```
// NG: ローカル変数のアドレスを返す
int* dangerous_function(void)
{
    int local_var = 100;
    return &local_var;  // 危険。関数終了後に無効になる
}

// OK: 静的変数または呼び出し元から受け取ったポインターを返す
int* safe_function(int *input)
{
    static int static_var = 100;
    return &static_var;  // 安全（静的変数）
    // または return input;  （引数として受け取ったポインター）
}
```

デバッグのヒント

1. **アドレスの確認**：`printf("%p\n", (void*)ptr)` でポインターの値を確認
2. **値の確認**：`printf("%d\n", *ptr)` でポインターが指す値を確認
3. **境界チェック**：配列アクセス前に必ず範囲をチェック
4. **コンパイラー警告**：`-Wall -Wextra -pedantic` を使用して警告を有効化

これらの解答例を参考に、ポインターの安全で効果的な使用方法を身につけてください。

第10章：文字列の解答例

このディレクトリには文字列操作に関する演習問題の解答例が含まれています。

解答ファイル一覧

演習10-1：基本的な文字列操作関数の実装

標準ライブラリの関数を使わずに、基本的な文字列操作関数を実装する問題です。

- **C90版**：ex10_1_string_operations.c
- **C99版**：ex10_1_string_operations_c99.c

実装する関数

- `my_strlen()` - 文字列の長さを計算
- `my_strcpy()` - 文字列をコピー
- `my_strcmp()` - 文字列を比較
- `my_strcat()` - 文字列を連結
- `my_strchr()` - 文字を検索

学習ポイント

- 文字列がヌル終端であることの理解
- ポインターと配列の操作
- 文字列処理の基本アルゴリズム

演習10-3：文字列配列の管理システム

学生名を格納する文字列配列の管理システムを実装する問題です。

- **C90版**：ex10_2_string_arrays.c
- **C99版**：ex10_2_string_arrays_c99.c

実装する機能

- 学生名の追加・削除
- 名前による検索
- アルファベット順ソート
- 一覧表示

学習ポイント

- 二次元配列の操作
- 文字列の動的管理
- 配列要素の挿入・削除
- 文字列ソートアルゴリズム

演習10-6：高度な文字列検索関数

標準ライブラリを使わずに、高度な文字列検索機能を実装する問題です。

- **C90版**：ex10_3_string_search.c

- **C99版**：ex10_3_string_search_c99.c

実装する機能

- 大小文字を区別しない検索
- 単語境界での検索
- ワイルドカード検索（*、？）
- 複数パターンの同時検索

学習ポイント

- 高度な文字列マッチングアルゴリズム
- パターンマッチング
- 検索結果の詳細情報管理
- メモリ効率的な実装

C90版とC99版の違い

C90版の特徴

- ANSI C（C90）に準拠
- 従来の配列とポインター操作
- 関数の先頭での変数宣言
- int型による真偽値表現

C99版の特徴

- C99の新機能を活用
- **bool型**：明確な真偽値表現
- **可変長配列（VLA）**：実行時サイズ決定
- **複合リテラル**：一時的なデータ構造
- **指定初期化子**：特定要素の初期化
- **restrict修飾子**：最適化ヒント
- **inline関数**：関数呼び出しコスト削減
- **固定幅整数型**：明確なデータサイズ

コンパイルと実行

基本的なコンパイル

```
# C90版

gcc -Wall -Wextra -pedantic -std=c90 ex10_1_string_operations.c -o ex10_1

# C99版

gcc -Wall -Wextra -pedantic -std=c99 ex10_1_string_operations_c99.c -o ex10_1_c99
```

Makefileを使用

```
# すべての解答をコンパイル

make solutions

# 特定の解答を実行

make run-ex10-1
make run-ex10-1-c99
```

学習の進め方

1. **基礎から応用へ**:演習10-1から順番に取り組む
2. **C90版を先に学習**:基本的な実装を理解
3. **C99版で機能拡張**:新しい言語機能を活用
4. **コードの比較**:両バージョンの違いを理解
5. **実行とテスト**:実際に動かして動作を確認

各解答の詳細説明

演習10-1:基本的な文字列操作

C90版の重要ポイント

- 基本的なポインター操作
- ヌル終端文字列の理解
- エラー処理の基礎

C99版の追加機能

- `restrict`修飾子による最適化
- `bool`型による明確な戻り値
- `inline`関数による性能向上
- 詳細な統計情報と分析機能

演習10-3:文字列配列管理

C90版の重要ポイント

- 二次元配列の操作
- 配列要素の動的管理
- 基本的なソートアルゴリズム

C99版の追加機能

- 可変長配列による柔軟な処理
- 構造体による詳細情報管理
- 複数の操作モード(検索・削除)
- 統計分析機能

演習10-6:高度な文字列検索

C90版の重要ポイント

- 基本的なパターンマッチング

- 文字列検索アルゴリズム
- メモリ効率的な実装

C99版の追加機能

- 動的プログラミングによるワイルドカード検索
- 複数パターン同時検索
- 詳細な検索結果情報
- 可変長配列によるメモリ最適化

実践的な応用例

これらの解答では、以下のような実践的な応用例も示しています。

1. **ログ解析システム** - 文字列検索を使用
2. **学生管理システム** - 配列操作を使用
3. **文字列統計分析** - 基本操作を組み合わせ
4. **パフォーマンステスト** - 大量データでの動作確認

トラブルシューティング

よくある問題

1. **セグメンテーション違反**
   - ヌル終端の確認
   - 配列の境界チェック
   - ポインターの初期化
2. **コンパイルエラー**
   - 標準の指定(`-std=c90` または `-std=c99`)
   - 必要なヘッダファイルのインクルード
3. **予期しない動作**
   - デバッグ情報の有効化(`-g`フラグ)
   - テストケースでの段階的確認

参考資料

- C言語の文字列操作基礎
- 演習問題の詳細
- 実行例とサンプルコード

これらの解答例では、C言語の文字列操作を基礎から応用まで順に確認できます。C90版で基本を確認し、C99版で追加された機能と実装方法の違いを学びます。

第11章：関数の解答例

この章では、関数の基本から応用まで、様々な演習問題の解答例を提供しています。

解答ファイル一覧

演習11-1：基本的な関数作成

素数判定や簡単な計算を行う基本的な関数群の実装例です。

- ex11_1_prime_functions.c - C90準拠版
- ex11_1_prime_functions_c99.c - C99準拠版（bool型、//コメント使用）

学習ポイント

- 関数の基本的な定義と呼び出し
- 引数と戻り値の扱い方
- ループと条件分岐の組み合わせ

演習11-2：配列操作関数

配列に対する各種操作（合計、平均、ソート等）を行う関数群の実装例です。

- ex11_2_array_operations.c - C90準拠版
- ex11_2_array_operations_c99.c - C99準拠版（可変長配列、inline関数使用）

学習ポイント

- 配列を引数として受け取る方法
- ポインターを使った配列操作
- 動的メモリ割り当て（calloc/free）

演習11-3：文字列処理関数

文字列の操作（コピー、連結、変換等）を行う関数群の実装例です。

- ex11_3_string_processing.c - C90準拠版
- ex11_3_string_processing_c99.c - C99準拠版（bool型、inline関数使用）

学習ポイント

- 文字配列としての文字列操作
- 標準ライブラリ関数の再実装
- 文字列の解析と変換

演習11-4：複数戻り値を持つ関数

ポインター引数を使って複数の値を返す関数群の実装例です。

- ex11_4_multiple_returns.c - C90準拠版
- ex11_4_multiple_returns_c99.c - C99準拠版（構造体戻り値、複合リテラル使用）

学習ポイント

- ポインター引数による複数値の返却
- 構造体を使った複数値のまとめ方

- 数学的計算と座標変換

演習11-5:エラー処理付き関数

エラー処理を含む堅牢な関数群の実装例です。

- ex11_5_error_handling.c - C90準拠版
- ex11_5_error_handling_c99.c - C99準拠版(enum型、可変長引数マクロ使用)

学習ポイント

- エラーコードの設計と管理
- 安全な数値計算(オーバーフロー対策)
- 境界チェックとNULLポインターチェック

演習11-6:構造体を使った関数

構造体を引数や戻り値として使用する関数群の実装例です。

- ex11_6_struct_functions.c - C90準拠版
- ex11_6_struct_functions_c99.c - C99準拠版(designated initializer、複合リテラル使用)

学習ポイント

- 構造体の値渡しとポインター渡し
- 構造体を返す関数の実装
- 複雑なデータ構造の操作

コンパイルと実行

各解答例は以下のようにコンパイルできます。

```
# C90版のコンパイル例

gcc -Wall -Wextra -pedantic -std=c90 ex11_1_prime_functions.c -o ex11_1_prime

# C99版のコンパイル例 (数学関数を使う場合は-lmを追加)

gcc -Wall -Wextra -pedantic -std=c99 ex11_4_multiple_returns_c99.c -lm -o ex11_4_c99

# 実行

./ex11_1_prime
```

学習のポイント

1. C90とC99の違い

各演習にはC90版とC99版の両方を用意しています。主な違いは次のとおりです。

- **変数宣言**:C90は関数の先頭、C99は使用箇所で宣言可能
- **コメント**:C90は/* */のみ、C99は//も使用可能
- **bool型**:C90は整数で代用、C99は`stdbool.h`の`bool`型
- **その他**:C99では可変長配列、複合リテラル、designated initializerなど

2. 関数設計の原則

- **単一責任の原則**:1つの関数は1つの機能に集中
- **適切な名前付け**:関数名から機能が推測できる

- **エラー処理**：異常系を考慮した堅牢な実装
- **const修飾子**：変更しない引数には`const`を付ける

3. テストの重要性

各解答例にはテスト用のmain関数が含まれています。

- 正常系のテスト
- 異常系のテスト（エラーケース）
- 境界値のテスト
- 実行結果の表示

4. メモリ管理

動的メモリを使用する場合。

- `malloc/calloc`で確保
- 使用後は必ず`free`で解放
- NULLチェックを忘れない
- メモリリークに注意

5. 移植性の考慮

- 標準ライブラリのみを使用
- プラットフォーム依存のコードを避ける
- 適切な型の使用（size_t、int64_tなど）

発展的な学習

これらの基本的な実装を理解したら、以下に挑戦してみましょう。

1. **最適化**：より効率的なアルゴリズムの実装
2. **拡張**：新しい機能の追加
3. **汎用化**：より汎用的な関数の設計
4. **エラー処理の改善**：より詳細なエラー情報の提供

注意事項

- 実装例はあくまで一例です。他の実装方法も検討してみましょう
- 実際のプロジェクトでは、標準ライブラリの使用を推奨します
- セキュリティを考慮した実装を心がけましょう

第12章:ビット操作の解答例

解答の前に

ビット操作は低レベルプログラミングの基本です。ここでは、各解答例を通じてビット操作の手法を確認します。

演習12-1解答:基本的なビット操作

ex12_1_bit_operations.c

実装のポイント

- ビット位置の範囲チェックが重要
- シフト演算は32ビット以上シフトすると未定義動作
- 循環シフトは通常のシフトと論理和で実現

```
/* ビットの循環シフト例 */
unsigned int rotate_left(unsigned int value, int shift) {
    shift %= 32;  /* 32ビット以上のシフトを防ぐ */
    return (value << shift) | (value >> (32 - shift));
}
```

演習12-2解答:フラグ管理システム

ex12_2_flag_system.c

実装のポイント

- 複数フラグの同時操作は論理和/論理積を活用
- 相反する状態は自動的にクリアする設計
- フラグの名前と値の対応表を用意すると便利

```
/* 相反する状態の処理例 */
if (status & STATUS_FROZEN) {
    status &= ~STATUS_BURNING;  /* 凍結時は燃焼を解除 */
}
```

演習12-3解答:ビットフィールドでメモリ節約

ex12_3_student_grades.c

実装のポイント

- ビットフィールドのサイズは慎重に設計
- パディングの影響を考慮
- 移植性を考慮した実装

通常の構造体では約16バイト必要なデータを、ビットフィールドで約6バイトに圧縮できます。

演習12-4解答:RGB色操作

ex12_4_rgb_colors.c

実装のポイント

- RGB565⇔RGB888変換時の精度損失に注意
- 5ビット→8ビット変換は上位ビットを下位に複製
- 色の混合は各成分を個別に計算

```
/* RGB565からRGB888への変換例 */
r8 = (r5 << 3) | (r5 >> 2);  /* 5ビットを8ビットに拡張 */
```

演習12-5解答：ビットベクタで集合演算

ex12_5_bit_set.c

実装のポイント

- 集合演算はビット演算と直接対応
- 要素数カウントはBrian Kernighanのアルゴリズムを使用
- 大きな集合では複数のワードに分割

集合演算とビット演算の対応：

- 和集合 = OR演算
- 積集合 = AND演算
- 差集合 = AND NOT演算
- 対称差 = XOR演算

演習12-6解答：CRC計算の実装

ex12_6_crc_checksum.c

実装のポイント

- ビット単位の実装は理解しやすいが遅い
- テーブル方式は256エントリで大幅に高速化
- CRC多項式の選択が重要

CRCは単純なチェックサムより優れたエラー検出能力を持ちます。

演習12-7解答：ビットボードでオセロ

ex12_7_othello_bitboard.c

実装のポイント

- 64ビット整数で8×8盤面を表現
- ビットシフトで方向を表現
- マスク演算で端の処理

ビットボードを使うと、複数の石の反転を一度に処理できます。

チャレンジ問題解答

チャレンジ12-A解答：ハフマン符号化

ex12_a_huffman.c

ハフマン符号化は可変長符号化の代表例です。頻出文字に短いビット列を割り当てることで圧縮を実現します。

チャレンジ12-B解答：ブルームフィルター

ex12_b_bloom_filter.c

ブルームフィルターは、空間効率の高い確率的データ構造です。偽陽性は発生しますが、偽陰性は発生しません。

発展学習

さらなる学習のために

1. **SIMD命令**：現代のCPUはビット操作を並列化できる
2. **ビット並列アルゴリズム**：複数のデータを同時処理
3. **圧縮アルゴリズム**：LZ77、LZW等の実装
4. **暗号化**：ビット操作は暗号化の基礎

参考図書

- “Hacker’s Delight” by Henry S. Warren Jr.
- “ビット演算超入門”（日本語書籍）

まとめ

ビット操作は以下の場面で威力を発揮します：

- メモリ制約が厳しい組み込みシステム
- 高速処理が必要なゲームエンジン
- ネットワークプロトコルの実装
- データ圧縮・暗号化

これらの技術を習得することで、より効率的なプログラムが書けるようになります。

第13章：構造体の解答例

このディレクトリには演習問題の解答例が含まれています。

解答ファイル一覧

基礎問題

1. **演習13-1：学生情報管理**
   - ex13_1_student.c - C90準拠版
   - ex13_1_student_c99.c - C99準拠版
   - 構造体とポインターを使った基本的なデータ管理
2. **演習13-2：座標計算**
   - ex13_2_coordinate.c - C90準拠版
   - ex13_2_coordinate_c99.c - C99準拠版
   - 構造体で座標を表現し、距離計算を実装
3. **演習13-3：商品管理**
   - ex13_3_product.c - C90準拠版
   - ex13_3_product_c99.c - C99準拠版
   - 構造体配列による商品在庫管理システム

応用問題

1. **演習13-4：従業員データベース**
   - ネストした構造体と統計計算
   - （未実装）
2. **演習13-5：図書管理システム**
   - 検索・貸出・返却機能の実装
   - （未実装）
3. **演習13-6：成績管理システム**
   - 学生別・科目別の成績統計とGPA計算
   - （未実装）

発展問題

1. **発展解答A：動的配列システム**
   - 動的メモリ管理による拡張可能な配列
   - （未実装）
2. **発展解答B：データソート**
   - 複数条件でのソート機能
   - （未実装）
3. **発展解答C：階層データ構造**

- 会社組織の階層構造表現
- （未実装）

学習のポイント

C90版とC99版の主な違い

1. **構造体の初期化**
   - C90: 順序通りの初期化のみ
   - C99: 指定イニシャライザが使用可能
2. **bool型**
   - C90: intで代用
   - C99: stdbool.hのbool型
3. **変数宣言**
   - C90: ブロックの先頭でのみ
   - C99: 使用箇所で宣言可能
4. **可変長配列（VLA）**
   - C90: 未対応
   - C99: 実行時にサイズ決定可能
5. **inline関数**
   - C90: 未対応
   - C99: inline修飾子で最適化

各演習の重要概念

演習13-1～13-3（基礎）

- 構造体の定義と初期化
- ドット演算子とアロー演算子
- 構造体配列の操作
- ポインターによる効率的なデータ渡し

演習13-4～13-6（応用）

- ネストした構造体
- 構造体ポインター配列
- 複雑なデータ構造の管理
- 統計処理とデータ分析

演習13-7～13-9（発展）

- 動的メモリ管理
- qsortによる汎用ソート
- 階層的なデータ構造
- より高度なポインター操作

コンパイルと実行

```
# C90準拠でコンパイル

gcc -std=c90 -Wall -Wextra -pedantic ex13_1_student.c -o ex13_1_student

# C99準拠でコンパイル

gcc -std=c99 -Wall -Wextra -pedantic ex13_1_student_c99.c -o ex13_1_student_c99

# 座標計算（数学ライブラリをリンク）

gcc -std=c90 -Wall -Wextra -pedantic ex13_2_coordinate.c -o ex13_2_coordinate -lm

# 実行

./ex13_1_student
```

注意事項

- 構造体のサイズはパディングの影響を受けることがある
- 構造体の直接比較はできない（メンバーごとに比較）
- 文字列メンバーへの代入にはstrcpyを使用
- 大きな構造体は値渡しより、ポインター渡しが効率的
- 動的メモリを使用する場合は、必ず解放処理を実装

第14章：関数ポインターの解答例

このディレクトリには、第14章の演習問題の解答例が含まれています。各解答例にはC90準拠版とC99版があり、いずれも非対話型のデモとして実装しています。

解答例一覧

演習14-1：関数ポインターの基本操作

- **C90版**：ex14_1_basic_function_pointer.c
- **C99版**：ex14_1_basic_function_pointer_c99.c
- **内容**：関数ポインターを使った動的関数呼び出しのデモ（maximum, minimum, power関数）
- **学習ポイント**：関数ポインターの宣言、初期化、呼び出し、配列での管理

演習14-2：関数選択システム

- **C90版**：ex14_2_function_selector.c
- **C99版**：ex14_2_function_selector_c99.c
- **内容**：文字に基づく関数選択システム（構造体による関数ポインター管理）
- **学習ポイント**：構造体と関数ポインターの組み合わせ、エラー処理

演習14-3：配列処理のコールバック

- **C90版**：ex14_3_array_callback.c
- **C99版**：ex14_3_array_callback_c99.c
- **内容**：コールバック関数を使った配列要素処理（倍加、減算、符号反転）
- **学習ポイント**：コールバック関数の概念、関数ポインターを引数として渡す方法

演習14-4：関数ポインター配列を使った計算機

- **C90版**：ex14_4_calculator_function_array.c
- **C99版**：ex14_4_calculator_function_array_c99.c
- **内容**：関数ポインター配列による計算機（履歴管理、統計機能付き）
- **学習ポイント**：関数ポインター配列、履歴管理、連続計算

実装上の特徴

非対話型デモ版について

すべての解答例は、クラッシュを避けるためscanf() による対話型入力を排除し、あらかじめ定義された値を使用するデモ版として実装されています。

教育的配慮

- 各ファイルには詳細なコメントと学習ポイントを記載
- C90とC99の違いを明確に示すコード例
- 実用的な応用例を含むデモンストレーション

セキュリティ対策

- ゼロ除算エラーのハンドリング

- 配列境界の適切なチェック
- 無効な関数ポインターの検出

C90版とC99版の主な違い

C90版の特徴

- 変数宣言は関数やブロックの先頭で行う
- /* */ 形式のコメントのみ使用
- for文の初期化部で変数宣言不可
- bool型は独自定義または整数型で代替

C99版の特徴

- 変数宣言を使用箇所で可能
- // 形式のコメントも使用可能
- for文内での変数宣言が可能
- <stdbool.h>のbool型を使用
- 指定初期化子による構造体初期化
- 可変長配列(VLA)のサポート

コンパイル方法

C90版のコンパイル

```
gcc -Wall -Wextra -pedantic -std=c90 -o program_name source_file.c
```

C99版のコンパイル

```
gcc -Wall -Wextra -pedantic -std=c99 -o program_name source_file_c99.c
```

学習のポイント

1. **関数ポインターの基本**
   - 関数ポインターの宣言方法
   - 関数アドレスの取得と呼び出し
   - 関数ポインターの型定義(typedef)
2. **関数ポインターの応用**
   - コールバック関数の実装
   - 関数ポインター配列の活用
   - 動的な関数選択
3. **設計パターン**
   - イベント駆動プログラミング
   - プラグインアーキテクチャ
   - 状態マシンパターン
4. **実践的な活用**

- エラー処理
- 拡張性の高い設計
- 保守性を考慮したコード構造

注意事項

- これらの解答例は一つの実装方法を示したものです
- 実際の開発では、要件に応じて適切な実装を選択してください
- エラー処理は基本的なレベルに留めています
- メモリ管理が必要な場合は、適切な解放処理を追加してください

第15章：分割コンパイルと発展技術の解答例

演習15-1：高度なプリプロセッサマクロ

ファイル：ex15_1_macro_basics.c, ex15_1_macro_basics_c99.c

基本的なプリプロセッサマクロの実装です。

- 数学定数、計算マクロ、ビット操作
- デバッグ支援マクロ（条件付きコンパイル）
- C90版：標準準拠のマクロ実装
- C99版：可変引数マクロ、__func__マクロ活用

演習15-2：安全なメモリ操作マクロ

ファイル：ex15_2_safe_memory.c, ex15_2_safe_memory_c99.c

メモリ安全性を高めるマクロセットの実装です。

- NULLチェック付きメモリ操作
- 配列境界チェック
- メモリリーク追跡システム
- C90版：基本的な安全性チェック
- C99版：inline関数、可変引数マクロによる拡張

演習15-3：シンプルなメモリプール

ファイル：ex15_3_memory_pool.c, ex15_3_memory_pool_c99.c

固定サイズオブジェクト用メモリプールの実装です。

- O(1)での割り当て・解放
- フリーリスト管理
- フラグメンテーション対策
- C90版：基本的なプール実装
- C99版：型安全性の向上、統計機能追加

演習15-4：汎用的なプリプロセッサライブラリ

ファイル：ex15_4_generic_macros.c, ex15_4_generic_macros_c99.c

高度なマクロプログラミング技法の実装です。

- 型チェック、汎用操作マクロ
- ループ展開、ベンチマークマクロ
- C90版：memcpyベースの汎用swap
- C99版：_Generic、typeof、複合リテラル活用

演習15-5：高性能メモリアロケーター

ファイル：ex15_5_allocator.c, ex15_5_allocator_c99.c

マルチプール型カスタムアロケーターの実装です。

- サイズ別プール管理
- 高速割り当て・解放
- 詳細な統計情報
- C90版：基本的なプール選択アルゴリズム
- C99版：inline最適化、restrict修飾子活用

演習15-6：キャッシュフレンドリーなデータ構造

ファイル：`ex15_6_cache_vector.c`, `ex15_6_cache_vector_c99.c`

キャッシュ効率を考慮した動的配列の実装です。

- キャッシュラインアライメント
- プリフェッチヒント
- バルク操作の最適化
- C90版：基本的なアライメント対応
- C99版：フレキシブル配列メンバー、VLA活用

演習15-7：プリプロセッサベースのDSL

ファイル：`ex15_7_dsl.c`, `ex15_7_dsl_c99.c`

ドメイン固有言語の実装例です。

- 状態機械DSL
- イベント駆動システムDSL
- ワークフローDSL
- C90版：基本的なマクロDSL
- C99版：可変引数マクロによる表現力向上

発展解答A：ガベージコレクション機能付きメモリ管理

ファイル：`ex15_8_gc_framework.c`, `ex15_8_gc_framework_c99.c`

3つのGCアルゴリズムの実装です。

- マーク&スイープGC
- 参照カウント方式
- 世代別GC
- C90版：基本的なGCアルゴリズム
- C99版：インクリメンタルGC、クロージャ対応

発展解答B：リアルタイムメモリアロケーター

ファイル：`ex15_9_realtime.c`, `ex15_9_realtime_c99.c`

決定的時間保証を持つアロケーターの実装です。

- O(1)時間保証
- デッドライン管理
- WCET(最悪実行時間)追跡
- C90版:基本的なリアルタイム保証
- C99版:キャッシュカラリング、高度な統計

ビルド方法

```
# 特定の解答をビルド
make ex15_1_macro_basics
# C99版をビルド
make ex15_1_macro_basics_c99
# すべての解答をビルド
make all
# 実行
./ex15_1_macro_basics
```

学習のポイント

1. **プリプロセッサの限界と可能性**
   - マクロの適切な使用場面
   - 型安全性の確保方法
   - デバッグの考慮
2. **メモリ管理の最適化**
   - カスタムアロケーターの設計
   - キャッシュ効率の考慮
   - リアルタイム性の保証
3. **C90とC99の違い**
   - 言語機能の進化
   - より安全で表現力豊かなコード
   - パフォーマンスの改善
4. **実践的な応用**
   - 組み込みシステム
   - ゲームエンジン
   - リアルタイムシステム

第16章：C23の機能の解答例

各演習問題の解答例と解説です。

演習16-1：bool型の活用

ファイル：`ex16_1_status_system.c`

bool型を使って複数のシステム状態を管理し、状態の組み合わせを判定する実装例です。

学習ポイント

- bool型の基本的な使用法
- 構造体でのbool型メンバー
- 論理演算の活用

演習16-2：ビット操作と2進数リテラル

ファイル：`ex16_2_bit_flags.c`

2進数リテラルを使った8ビットフラグシステムの実装例。

学習ポイント

- 2進数リテラルの実用的な使用
- ビット演算の基本操作
- マクロでのビット操作

演習16-3：typeof演算子の応用

ファイル：`ex16_3_generic_macros.c`

typeof演算子を使った汎用マクロの実装例。

学習ポイント

- typeof演算子による型の取得
- 型安全なマクロの作成
- Statement Expression（GCC拡張）の活用

演習16-4：nullptr安全プログラミング

ファイル：`ex16_4_safe_linkedlist.c`

nullptrを使った安全なリンクリスト実装例。

学習ポイント

- nullptrによる明示的なポインターチェック
- エラー処理のベストプラクティス
- メモリ管理の安全性

演習16-6：C23総合演習

ファイル：`ex16_5_config_manager.c`

C23の複数機能を組み合わせた設定管理システムの実装例。

学習ポイント

- 複数のC23機能の統合
- 実用的なプログラム設計
- 可読性と保守性の両立

解答例の実行方法

```
# 個別の解答例をコンパイル・実行

make ex16_1_status_system
./solutions/ex16_1_status_system

# すべての解答例を実行

make run-solutions
```

注意事項

- これらは解答例の一つです。他にも様々な実装方法があります
- C23のサポート状況により、一部の機能が動作しない可能性があります
- エラー処理は教育目的で詳細に実装しています